Laporan Tugas Besar 1
IF3170 Intelegensi Artifisial 2026/2027



Kelompok 38

AI_is_Love

Muhammad Nafis Habibi	13524018
Billy Ontoseno Irawan	13524121
Muhammad Rafi Akbar	13524125

Institut Teknologi Bandung

Sekolah Teknik Elektro dan Informatika - Komputasi

Bandung

2026

Daftar Isi

Daftar Isi **3**
Deskripsi Persoalan **4**
Daftar Variabel Permasalahan 4
Ketentuan Representasi 4
Objective Function & Constraints 5
Pembahasan **7**
Implementasi Algoritma 7
1. Struktur Data Umum 7
2. Hill Climbing 7
3. Simulated Annealing 7
4. Genetic Algorithm 7
Hasil dan Analisis Eksperimen 7
1. Hill Climbing 7
Hasil Eksperimen 7
Analisis Hasil 8
2. Simulated Annealing 8
Hasil Eksperimen 8
Analisis Hasil 8
3. Genetic Algorithm 8
Hasil Eksperimen 8
Analisis Hasil 10
Kesimpulan dan Saran **11**
Pembagian Tugas **12**
Referensi **13**
SURAT PERNYATAAN PENGGUNAAN AI **14**

Deskripsi Persoalan

Tugas ini bertujuan untuk membuat konfigurasi pengisian paket paling optimal untuk sebuah truk peti kemas. Optimal disini berarti memaksimalkan fungsi objektif.

Daftar Variabel Permasalahan

Entitas	**Atribut**	**Contoh**	**Keterangan**
Package	id	“RusdiPackage”	Identifier
	Dimensions	{w:3, l:4, h:1}	Ukuran sebuah paket, yaitu width, length, dan height
	Orientation	{w:x, l:y, h:z}	Sumbu yang sejajar dengan setiap sisi paket. Terdapat enam kemungkinan orientasi, misalnya {w:y, l:x, h:z} setelah rotasi 90° terhadap sumbu z.
	Value	60	Nilai harga sebuah paket (satuan unit)
	Weight	5	Berat paket (satuan unit berat)
	IsFragile	False	Benda pecah-belah
	ETA	3	Estimated Time of Arrival, perkiraan waktu kedatangan paket (dalam sisa hari)
Truck	Dimensions	{w:10, l:20, h:5}	Ukuran sebuah peti kemas pada truk, yaitu width, length, dan height
	MaxCapacity	500	Kapasitas angkut maksimum pada suatu peti kemas (satuan unit berat)

Ketentuan Representasi

Berikut prosedur bagaimana dalam merepresentasikan permasalahan ini:

- Paket hanya berbentuk kubus / balok. Paket dapat berbeda ukuran satu sama lain dan tidak semua harus dimasukkan ke dalam peti kemas pada truk sehingga suatu paket dapat berada di luar (seluruh bagian paket di luar)
- Koordinat berupa bilangan bulat. Cara representasi koordinat dibebaskan.

- Struktur data yang digunakan untuk representasi dibebaskan. Misalnya menggunakan key-value seperti JSON.
- Inisialisasi (initial state) random.
- Langkah neighbor (move) yang diperbolehkan tiap iterasi:
    1. Menukar dua paket, baik di dalam maupun di luar truk
    2. Memindahkan sebuah paket ke suatu koordinat yang berbeda
    3. Merotasi paket 90° terhadap sumbu x, y, atau z
- Representasi solusi (final state) adalah pemetaan semua paket yang ada dan juga pemetaan semua koordinat dalam peti kemas pada truk. Secara detail yang harus dapat ditunjukkan oleh state dan visualisasi adalah:
    1. Bagaimana keadaan semua paket yang di dalam maupun yang di luar
    2. Bagaimana keadaan peti kemas pada truk
    3. Value pada state tersebut

Objective Function & Constraints

Agar permasalahan ini tidak menjadi kompleks, berikut arahan yang diberikan

- **Objective function**

  Permasalahan ini bertujuan untuk memaksimalkan value peti kemas pada truk.
- **Kardus tidak bisa melayang dan menembus dinding peti kemas**

  Melayang disini didefinisikan dengan misalkan paket i setelah orientasi tertentu memiliki posisi sudut kiri-bawah-belakang

  $$(x_i, y_i, z_i)$$

  dan dimensi

  $$(w_i, d_i, h_i)$$

  sehingga alas paket i adalah

  $$B_i = \{(x, y, z_i) \mid x_i \leq x < x_i + w_i,\ y_i \leq y < y_i + d_i\})$$

  Sebuah titik di alas memiliki support apabila:

  1. $Z_{i=0}$, artinya di alas truk, atau
  2. Terdapat paket lain j yang mempunyai bagian atas tepat di bawah titik tersebut

Maka melayang adalah kondisi tidak ada satupun titik alas yang memiliki support.

- **Barang pecah-belah tidak boleh di bawah paket lain**

  Barang pecah-belah tidak boleh menjadi support
- **Truk tidak boleh mengangkut melebihi kapasitasnya**

Pembahasan

Implementasi Algoritma

Algoritma yang diimplementasikan yaitu Hill Climbing beserta semua variasinya, Simulated Annealing, dan Genetic Algorithm. Algoritma bisa menangani lebih dari satu truk sekaligus. Penamaan variabel, kelas, dan fungsi mungkin terasa norak. Kami ingin bersenang-senang dengan namanya.

1. Struktur Data

Implementasi struktur data dilakukan di file data.py.

Position, Dimensions, dan Orientation

Position menandakan posisi dalam ruang tiga dimensi. Dimensions menandakan ukuran dari truk dan paket. Orientation menandakan sumbu mana yang sejajar dengan atribut dari Dimensions. Method __str__ dan copy ditambahkan untuk memudahkan debugging dan penggunaan.

```
class Position:
    def __init__(self, x, y, z):
        self.x = x
        self.y = y
        self.z = z

    def __str__(self):
        return f"({self.x},{self.y},{self.z})"

    def copy(self):
        return Position(self.x, self.y, self.z)

class Dimensions:
    def __init__(self, width, length, height):
        self.width = width
        self.length = length
        self.height = height

    def __str__(self):
        return f"{self.width}x{self.length}x{self.height}"

    def copy(self):
        return Dimensions(self.width, self.length, self.height)

class Orientation:
    def __init__(self, width_orientation = "x", length_orientation = "z",
```

```
height_orientation = "y"):
        self.width_orientation = width_orientation
        self.length_orientation = length_orientation
        self.height_orientation = height_orientation

    def __str__(self):
        return
f"(w->{self.width_orientation},l->{self.length_orientation},h->{self.height_orientat
ion})"

    def copy(self):
        return Orientation(self.width_orientation, self.length_orientation,
self.height_orientation)
```

Truk dan Paket

Truk memiliki dimensi peti kemas dan kapasitas maksimal. Orientasi di sini agar tidak ambigu saja. Tinggi truk dianggap sejajar dengan sumbu y. Paket memiliki dimensi, orientasi, weight, is_fragile, dan estimated_time_of_arrival. Orientasi paket tidak terlalu terpakai karena ujung-ujungnya berubah saat keberjalanan algoritma. estimated_time_of_arrival tidak terpakai sama sekali.

```
class TruckKun:
    id_count = 0
    def __init__(self, id = "truck_" + str(id_count), dimensions = Dimensions(10,
10, 10), max_capacity = 1000):
        self.id = id
        self.dimensions = dimensions
        self.orientation = Orientation() # use the default
        self.max_capacity = max_capacity

        if id == "truck_" + str(self.id_count):
            TruckKun.id_count += 1

    def oriented_dimentions(self) -> Dimensions:
        oriented_dimensions = Dimensions(0, 0, 0)
        match self.orientation.width_orientation:
            case "x": oriented_dimensions.width += self.dimensions.width
            case "y": oriented_dimensions.height += self.dimensions.width
            case "z": oriented_dimensions.length += self.dimensions.width
        match self.orientation.height_orientation:
            case "x": oriented_dimensions.width += self.dimensions.height
            case "y": oriented_dimensions.height += self.dimensions.height
            case "z": oriented_dimensions.length += self.dimensions.height
        match self.orientation.length_orientation:
            case "x": oriented_dimensions.width += self.dimensions.length
            case "y": oriented_dimensions.height += self.dimensions.length
            case "z": oriented_dimensions.length += self.dimensions.length
```

```
class LovePackage:
    id_count = 0

    def __init__(self, id = "package_" + str(id_count),
                 dimensions = Dimensions(10, 10, 10),
                 orientation = Orientation(),
                 value = 10,
                 weight = 10,
                 is_fragile = False,
                 estimated_time_of_arrival = 1):
        self.id = id
        self.dimensions = dimensions
        self.orientation = orientation
        self.value = value
        self.weight = weight
        self.is_fragile = is_fragile
        self.estimated_time_of_arrival = estimated_time_of_arrival

        if id == "package_" + str(self.id_count):
            LovePackage.id_count += 1
```

Konfigurasi Peletakan Paket dalam Truk

Truk dan paket di-list di dalam satu kelas Isekai dan hasil peletakannya disimpan dalam IsekaiConfiguration. List truk dan paket dipisah dari konfigurasinya karena dalam satu algoritma, konfigurasi yang dibuat pasti menggunakan list truk dan paket yang sama. Jadi tidak perlu di-copy saat ingin membuat konfigurasi baru.

Adapun method yang disediakan di IsekaiConfiguration dapat digunakan untuk mengatur posisi, orientasi, dan di dalam truk mana, serta mengecek kevalidan konfigurasi.

```
class Isekai:
    def __init__(self, trucks: list[TruckKun], packages: list[LovePackage]):
        self.trucks = trucks
        self.packages = packages

class IsekaiConfiguration:
    def __init__(self, isekai: Isekai):
        self.isekai = isekai

        self.package_truck = dict[LovePackage, TruckKun]() # package -> truck
        self.package_position = dict[LovePackage, Position]() # package -> position
        self.package_orientation = dict[LovePackage, Orientation]() # package ->
orientation

    def configure_package(self, package: LovePackage, truck: TruckKun, position:
Position = Position(0, 0, 0), orientation: Orientation = Orientation()):
```

```
        self.package_truck[package] = truck
        self.package_position[package] = position
        self.package_orientation[package] = orientation

    def remove_package(self, package: LovePackage):
        self.package_truck.pop(package)
        self.package_position.pop(package)
        self.package_orientation.pop(package)

    def set_package_truck(self, package: LovePackage, truck: TruckKun):
        self.package_truck[package] = truck

    def set_package_position(self, package: LovePackage, position: Position):
        self.package_position[package] = position

    def set_package_orientation(self, package: LovePackage, orientation:
Orientation):
        self.package_orientation[package] = orientation

    def get_package_truck(self, package: LovePackage) -> TruckKun:
        return self.package_truck.get(package)

    def get_package_position(self, package: LovePackage) -> Position:
        position = self.package_position.get(package)
        if position is None:
            return None
        else:
            return position.copy()

    def get_package_orientation(self, package: LovePackage) -> Orientation:
        orientation = self.package_orientation.get(package)
        if orientation is None:
            return None
        else:
            return orientation.copy()

    def get_package_dimensions(self, package: LovePackage) -> Dimensions:
        package_orientation = self.get_package_orientation(package)
        if package_orientation is None:
            package_orientation = package.orientation
        package_dimensions = Dimensions(0, 0, 0)
        match package_orientation.width_orientation:
            case "x": package_dimensions.width += package.dimensions.width
            case "y": package_dimensions.height += package.dimensions.width
            case "z": package_dimensions.length += package.dimensions.width
        match package_orientation.height_orientation:
            case "x": package_dimensions.width += package.dimensions.height
            case "y": package_dimensions.height += package.dimensions.height
            case "z": package_dimensions.length += package.dimensions.height
        match package_orientation.length_orientation:
            case "x": package_dimensions.width += package.dimensions.length
            case "y": package_dimensions.height += package.dimensions.length
            case "z": package_dimensions.length += package.dimensions.length
```

```
        return package_dimensions

    def package_is_outside_truck(self, package:LovePackage) -> bool:
        truck = self.get_package_truck(package)

        if truck is None:
            return False

        bounds = self.calculate_bound(package)

        return (
            bounds[0].x < 0
            or bounds[0].y < 0
            or bounds[0].z < 0
            or bounds[1].x > truck.dimensions.width
            or bounds[1].y > truck.dimensions.height
            or bounds[1].z > truck.dimensions.length
        )

    def is_valid(self, debug=False) -> bool:
        valid = True

        for truck in self.isekai.trucks:
            if (self.total_weight(truck) > truck.max_capacity):
                if debug: print(truck.id, "has too much weight")
                valid = False
            if not valid: break

        for package in self.isekai.packages:
            if self.get_package_truck(package) is None:
                continue
            if self.package_is_outside_truck(package):
                if debug: print(package.id, "is outside truck")
                valid = False
            if self.package_is_intersecting(package):
                if debug: print(package.id, "is intersecting")
                valid = False
            if not self.package_is_supported(package):
                if debug: print(package.id, "is not supported")
                valid = False
            if package.is_fragile and self.package_is_supporting(package):
                if debug: print(package.id, "is destroyed")
                valid = False
            if not valid: break

        return valid

    def is_truck_valid(self, truck: TruckKun) -> bool:
        valid = True

        if (self.total_weight(truck) > truck.max_capacity):
            valid = False
```

```
        for package in self.isekai.packages:
            if self.get_package_truck(package) is not truck:
                continue
            if self.package_is_outside_truck(package):
                valid = False
            if self.package_is_intersecting(package):
                valid = False
            if not self.package_is_supported(package):
                valid = False
            if package.is_fragile and self.package_is_supporting(package):
                valid = False
            if not valid: break

        return valid


    def total_weight(self, truck: TruckKun):
        total = 0
        for package, truckKun in self.package_truck.items():
            if truck is truckKun:
                total += package.weight
        return total

    def package_is_supported(self, package: LovePackage) -> bool:
        if self.get_package_truck(package) is None:
            return True

        bounds = self.calculate_bound(package)
        if bounds[0].y == 0:
            return True

        for other_package in self.isekai.packages:
            if package is other_package:
                            continue
            if self.get_package_truck(package) is not
self.get_package_truck(other_package):
                continue
            other_bounds = self.calculate_bound(other_package)
            intersecting_area = not (
                bounds[1].x <= other_bounds[0].x
                or bounds[0].x >= other_bounds[1].x
                or bounds[1].z <= other_bounds[0].z
                or bounds[0].z >= other_bounds[1].z
            )
            if intersecting_area and bounds[0].y == other_bounds[1].y:
                return True

        return False

    def package_is_supporting(self, package: LovePackage) -> bool:
        if self.get_package_truck(package) is None:
            return False
```

```
        bounds = self.calculate_bound(package)
        for other_package in self.isekai.packages:
            if package is other_package:
                            continue
            if self.get_package_truck(package) is not
self.get_package_truck(other_package):
                continue
            other_bounds = self.calculate_bound(other_package)
            intersecting_area = not (bounds[1].x <= other_bounds[0].x
                                     or bounds[0].x >= other_bounds[1].x
                                     or bounds[1].z <= other_bounds[0].z
                                     or bounds[0].z >= other_bounds[1].z)
            if intersecting_area and bounds[1].y == other_bounds[0].y:
                return True

        return False

    def calculate_bound(self, package: LovePackage) -> list[Position]:
        first_point = self.get_package_position(package)
        other_point = first_point.copy()
        package_dimensions = self.get_package_dimensions(package)
        other_point.x += package_dimensions.width
        other_point.y += package_dimensions.height
        other_point.z += package_dimensions.length
        return [first_point, other_point]

    def package_is_intersecting(self, package: LovePackage, other_package:
LovePackage = None) -> bool:
        if self.get_package_truck(package) is None:
            return False

        intersecting = False
        bounds = self.calculate_bound(package)
        if other_package is None:
            for other_package in self.isekai.packages:
                if package is other_package:
                    continue
                if self.get_package_truck(package) is not
self.get_package_truck(other_package):
                    continue
                other_bounds = self.calculate_bound(other_package)
                if not (bounds[1].x <= other_bounds[0].x
                        or bounds[0].x >= other_bounds[1].x
                        or bounds[1].y <= other_bounds[0].y
                        or bounds[0].y >= other_bounds[1].y
                        or bounds[1].z <= other_bounds[0].z
                        or bounds[0].z >= other_bounds[1].z):
                    intersecting = True
                    break
        else:
            truck = self.get_package_truck(package)
            other_truck = self.get_package_truck(other_package)
            if package is not other_package and truck is not other_truck:
```

```
                other_bounds = self.calculate_bound(other_package)
                intersecting = not (bounds[1].x <= other_bounds[0].x
                                    or bounds[0].x >= other_bounds[1].x
                                    or bounds[1].y <= other_bounds[0].y
                                    or bounds[0].y >= other_bounds[1].y
                                    or bounds[1].z <= other_bounds[0].z
                                    or bounds[0].z >= other_bounds[1].z)

        return intersecting

    def copy(self) -> IsekaiConfiguration:
        config = IsekaiConfiguration(self.isekai)
        config.package_truck = self.package_truck.copy()
        config.package_position = self.package_position.copy()
        config.package_orientation = self.package_orientation.copy()
        return config
```

Write dan Read Data dari JSON

Kurang lebih alur pengubahan dari JSON ke dalam bentuk kelas yaitu string JSON, objek python biasa (gabungan dictionary dan array), lalu menjadi kelas.

```
def truck_from_obj(obj):
    if obj["type"] != "truck":
        return None
    else:
        return TruckKun(obj["id"],
                        Dimensions(obj["dimensions"]["w"], obj["dimensions"]["l"],
obj["dimensions"]["h"]),
                        obj["max_capacity"])

def truck_to_obj(truck: TruckKun):
    return {
        "id" : truck.id,
        "dimensions" : {"w": truck.dimensions.width, "l": truck.dimensions.length,
"h": truck.dimensions.height},
        "max_capacity" : truck.max_capacity
    }

def package_from_obj(obj):
    if obj["type"] != "package":
        return None
    else:
        return LovePackage(obj["id"],
                        Dimensions(obj["dimensions"]["w"], obj["dimensions"]["l"],
obj["dimensions"]["h"]),
                        Orientation(obj["orientation"]["w"],
obj["orientation"]["l"], obj["orientation"]["h"]),
                        obj["value"],
                        obj["weight"],
```

```
                        obj["is_fragile"],
                        obj["estimated_time_of_arrival"])

def package_to_obj(package: LovePackage):
    return {
        "id" : package.id,
        "dimensions" : {
            "w": package.dimensions.width,
            "l": package.dimensions.length,
            "h": package.dimensions.height
        },
        "orientation" : {
            "w": package.orientation.width_orientation,
            "l": package.orientation.length_orientation,
            "h": package.orientation.height_orientation
        },
        "value": package.value,
        "weight": package.weight,
        "is_fragile": package.is_fragile,
        "estimated_time_of_arrival": package.estimated_time_of_arrival
    }

def parse_isekai(json_string: str):
    truckKuns = []
    lovePackages = []

    objs = json.loads(json_string)
    for obj in objs:
        if obj["type"] == "package":
            lovePackages.append(package_from_obj(obj))
        elif obj["type"] == "truck":
            truckKuns.append(truck_from_obj(obj))

    return Isekai(truckKuns, lovePackages)

def load_isekai_from_file(filepath: str):
    with open(filepath, "r") as file:
        result = parse_isekai(file.read())
    return result

def isekai_to_str(isekai: Isekai):
    objs = []
    for truck in isekai.trucks:
        objs.append(truck)
    for package in isekai.packages:
        objs.append(package)
    return json.dumps(objs)

def save_isekai_to_file(isekai: Isekai, filepath: str):
    with open(filepath, "w") as file:
        file.write(isekai_to_str(isekai))
```

2. Hill Climbing

Variasi hill climbing yang diimplementasikan yaitu Steepest Ascend, Sideways Move, Random Restart, dan Stochastic. Implementasi hill climbing disatukan pada satu fungsi yang sama. Parameter mode dan max_sideways yang akan membedakan apa yang dilakukan algoritmanya.

```
import itertools
import random
import time

from data import *

AXES = ("x", "y", "z")

#ukuran pada orientasi berbeda
def love_size(package, orientation):
    dimensions = {
        orientation.width_orientation: package.dimensions.width,
        orientation.height_orientation: package.dimensions.height,
        orientation.length_orientation: package.dimensions.length,
    }
    return dimensions["x"], dimensions["y"], dimensions["z"]

# semua orientasi yg bisa
def heavy_rotation(package):
    result, seen = [], set()

    for axes in itertools.permutations(AXES):
        orientation = Orientation(*axes)
        size = love_size(package, orientation)
        if size not in seen:
            result.append(orientation)
            seen.add(size)
    return result

# bisa masuk truck atau tidak
def will_it_work(truck, package, pos, orientation):
    x, y, z = love_size(package, orientation)
    d = truck.dimensions
    return (
        pos.x >= 0 and pos.y >= 0 and pos.z >= 0
        and pos.x + x <= d.width
        and pos.y + y <= d.height
        and pos.z + z <= d.length
    )

# hindarin state duplikat
def my_identity(state):
```

```
    result = []
    for package in state.isekai.packages:
        truck = state.get_package_truck(package)

        if truck is None:
            result.append((package.id, None))
            continue

        p = state.get_package_position(package)
        o = state.get_package_orientation(package)

        result.append((package.id, truck.id, p.x, p.y, p.z, o.width_orientation,
o.height_orientation, o.length_orientation,))

    return tuple(result)

# add package
def new_chapter(state, package, truck, pos, orientation):
    candidate = state.copy()
    candidate.package_truck[package] = truck
    candidate.package_position[package] = pos.copy()
    candidate.package_orientation[package] = orientation.copy()
    return candidate

# remove package
def breakup(state, package):
    candidate = state.copy()
    candidate.package_truck.pop(package, None)
    candidate.package_position.pop(package, None)
    candidate.package_orientation.pop(package, None)
    return candidate

#posisi kandidat
def heroine_candidate(state, truck, skip = None):
    anchors = {(0, 0, 0)}

    for package in state.isekai.packages:
        if package is skip or state.get_package_truck(package) is not truck:
            continue

        low, high = state.calculate_bound(package)
        anchors.update((
            (high.x, low.y, low.z),
            (low.x, high.y, low.z),
            (low.x, low.y, high.z),
        ))
    return [Position(*p) for p in anchors]

#value
def love_score(state):
    score = 0

    for p in state.isekai.packages:
```

```
        if state.get_package_truck(p) is not None:
            score += p.value

    return score

#config awal random
def generate_heroine(isekai, include_prob=0.5, rng = None):
    # initial config random
    if not 0 <= include_prob <= 1:
        raise ValueError("probabilitas harus di antara 0 dan 1")

    rng = rng or random.Random()
    state = IsekaiConfiguration(isekai)
    packages = list(isekai.packages)
    rng.shuffle(packages)

    for package in packages:
        if rng.random() > include_prob:
            continue

        trucks = list(isekai.trucks)
        rng.shuffle(trucks)
        orientations = heavy_rotation(package)
        rng.shuffle(orientations)

        for truck in trucks:
            positions = heroine_candidate(state, truck)
            rng.shuffle(positions)

            for pos in positions:
                for orientation in orientations:
                    if not will_it_work(truck, package, pos, orientation):
                        continue

                    candidate = new_chapter(state, package, truck, pos, orientation)

                    if candidate.is_valid():
                        state = candidate
                        break
                else:
                    continue
                break
            else:
                continue
            break
    return state

# Semua neighbor yang mungkin, diproduksi satu per satu agar HC tidak nyimpan
seluruh konfigurasi kandidat sekaligus.
def iter_childhood_neighbors(state):
    seen = {my_identity(state)}

    def add(candidate):
```

```
        key = my_identity(candidate)
        if key not in seen and candidate.is_valid():
            seen.add(key)
            return candidate
        return None

    packages = state.isekai.packages

    for package in packages:
        old_truck = state.get_package_truck(package)
        old_pos = state.get_package_position(package)
        old_orientation = state.get_package_orientation(package)

        if old_truck is not None:
            base = breakup(state, package)
        else:
            base = state

        if old_truck is not None:
            neighbor = add(base)
            if neighbor is not None:
                yield neighbor

        if old_orientation is None:
            orientations = heavy_rotation(package)
        else:
            orientations = [old_orientation]

        for truck in state.isekai.trucks:
            for pos in heroine_candidate(base, truck):
                if old_truck is truck and old_pos is not None:
                    if (pos.x, pos.y, pos.z) == (old_pos.x, old_pos.y, old_pos.z):
                        continue

                for orientation in orientations:
                    if will_it_work(truck, package, pos, orientation):
                        neighbor = add(new_chapter(base, package, truck, pos,
orientation))
                        if neighbor is not None:
                            yield neighbor

        if old_truck is not None:
            old_size = love_size(package, old_orientation)

            for orientation in heavy_rotation(package):
                if love_size(package, orientation) == old_size:
                    continue

                if will_it_work(old_truck, package, old_pos, orientation):
                    neighbor = add(
                        new_chapter(state, package, old_truck, old_pos, orientation)
                    )
                    if neighbor is not None:
```

```
                        yield neighbor

    # tuker posisi / ganti paket baru
    for a in packages:
        if state.get_package_truck(a) is not None:
            continue

        for b in packages:
            truck_b = state.get_package_truck(b)
            if truck_b is None:
                continue

            pos_b = state.get_package_position(b)
            base = breakup(state, b)

            for orientation_a in heavy_rotation(a):
                if not will_it_work(truck_b, a, pos_b, orientation_a):
                    continue

                replaced = new_chapter(base, a, truck_b, pos_b, orientation_a)
                neighbor = add(replaced)
                if neighbor is not None:
                    yield neighbor

                for pos_b_new in heroine_candidate(replaced, truck_b, skip=a):
                    for orientation_b in [state.get_package_orientation(b)]:
                        if not will_it_work(truck_b, b, pos_b_new, orientation_b):
                            continue
                        candidate = new_chapter(replaced, b, truck_b, pos_b_new,
orientation_b)
                        neighbor = add(candidate)
                        if neighbor is not None:
                            yield neighbor

    # swap posisi paket
    for i, a in enumerate(packages):
        truck_a = state.get_package_truck(a)
        if truck_a is None:
            continue
        for b in packages[i + 1:]:
            truck_b = state.get_package_truck(b)
            if truck_b is None:
                continue
            candidate = state.copy()
            candidate.package_truck[a], candidate.package_truck[b] = truck_b,
truck_a
            candidate.package_position[a] = state.get_package_position(b)
            candidate.package_position[b] = state.get_package_position(a)
            candidate.package_orientation[a] = state.get_package_orientation(b)
            candidate.package_orientation[b] = state.get_package_orientation(a)
            neighbor = add(candidate)
            if neighbor is not None:
                yield neighbor
```

```
def get_childhood_neighbors(state):
    return list(iter_childhood_neighbors(state))

def total_value(isekai):
    return sum(p.value for p in isekai.packages)

def climbing(initial, mode = "Steepest", max_iterations = 500, rng = None, detail =
False, max_sideways = 0, reset_sideways_on_improve = True, isekai = None):
    if mode not in ("Steepest", "Stochastic"):
        raise ValueError("Mode harus 'Steepest' atau 'Stochastic'")
    if max_iterations < 0 or max_sideways < 0:
        raise ValueError("Limit tidak boleh negatif")
    if not initial.is_valid():
        raise ValueError("Initial state tidak valid")

    rng = rng or random.Random()
    isekai = isekai or initial.isekai
    start_time = time.perf_counter()
    upper_bound = total_value(isekai)
    current = initial.copy()
    current_score = initial_score = love_score(current)
    our_history = [current_score]

    visited = {my_identity(current)} if max_sideways > 0 else None

    sideway_streak = sideway_count = 0
    stop_reason = "max iteration count"

    for iteration in range(max_iterations):
        if current_score >= upper_bound:
            stop_reason = "Semua paket sudah masuk truck"
            break

        best_score = None
        best_neighbor = None
        best_tie_count = 0
        improving_neighbor = None
        improving_count = 0

        for neighbor in iter_childhood_neighbors(current):
            neighbor_key = my_identity(neighbor)
            if visited is not None and neighbor_key in visited:
                continue

            score = love_score(neighbor)
            if best_score is None or score > best_score:
                best_score = score
                best_neighbor = neighbor
                best_tie_count = 1
            elif score == best_score:
                best_tie_count += 1
                if rng.randrange(best_tie_count) == 0:
```

```
                    best_neighbor = neighbor

            if mode == "Stochastic" and score > current_score:
                improving_count += 1
                if rng.randrange(improving_count) == 0:
                    improving_neighbor = neighbor

        if best_neighbor is None:
            stop_reason = "no more neighbors"
            break

        if best_score < current_score:
            stop_reason = "sudah Local Optimum"
            break

        if best_score == current_score:
            if max_sideways == 0:
                stop_reason = "sudah Local Optimum"
                break
            if sideway_streak >= max_sideways:
                stop_reason = "sideway limit tercapai"
                break
            sideway_streak += 1
            sideway_count += 1
            current = best_neighbor
            current_score = best_score

        else:
            if reset_sideways_on_improve:
                sideway_streak = 0
            if mode == "Steepest":
                current = best_neighbor
                current_score = best_score
            else:
                current = improving_neighbor
                current_score = love_score(current)

        if visited is not None:
            visited.add(my_identity(current))
        our_history.append(current_score)

        if detail:
            print(f"Iterasi {iteration + 1}: "  f"score={current_score}
(sideways={sideway_streak})")

    return {
            "initial_state": initial,
            "final_state": current,
            "initial_value": initial_score,
            "final_value": current_score,
            "history": our_history,
            "iterations": len(our_history) - 1,
            "sideway_count": sideway_count,
```

```
            "max_sideways": max_sideways,
            "stop_reason": stop_reason,
            "duration": time.perf_counter() - start_time,
        }

def steepest_hill_climbing(initial, max_iterations = 500, rng = None, detail =
False):
    return climbing(initial, "Steepest", max_iterations, rng, detail)

def stochastic_hill_climbing(initial, max_iterations = 500, rng = None, detail =
False):
    return climbing(initial, "Stochastic", max_iterations, rng, detail)

def sideway_hill_climbing(initial, max_sideways = 20, max_iterations = 500, rng =
None, reset_sideways_on_improve=True, detail=False):
    return climbing(initial, "Steepest", max_iterations, rng, detail, max_sideways,
reset_sideways_on_improve)

def random_restart_hc(isekai, max_restarts = 5, max_iterations = 500, include_prob =
0.5, mode = "Steepest",max_sideways = 0, rng = None, detail=False):
    if max_restarts < 1:
        raise ValueError("max_restarts harus >= 1")

    rng = rng or random.Random()
    upper_bound = total_value(isekai)
    start_time = time.perf_counter()
    best_state = None
    history = []
    iterations_per_restart = []
    restart_value = []
    stop_reason = "restart limit tercapai"

    for i in range(max_restarts):
        initial_state = generate_heroine(isekai, include_prob=include_prob, rng=rng)
        result = climbing(initial_state, mode, max_iterations, rng, detail,
max_sideways)
        history.extend(result["history"])
        iterations_per_restart.append(result["iterations"])
        restart_value.append(result["final_value"])

        if best_state is None or result["final_value"] > best_state["final_value"]:
            best_state = result
        if best_state["final_value"] >= upper_bound:
            stop_reason = "Semua paket sudah masuk di Truck"
            break

    return {
            "initial_state": best_state["initial_state"],
            "final_state": best_state["final_state"],
            "initial_value": best_state["initial_value"],
            "final_value": best_state["final_value"],
            "history": history,
            "iterations": sum(iterations_per_restart),
```

```
            "restarts": len(iterations_per_restart),
            "max_restarts": max_restarts,
            "iterations_per_restart": iterations_per_restart,
            "restart_value": restart_value,
            "stop_reason": stop_reason,
            "duration": time.perf_counter() - start_time,
        }

def run_steepest_hill_climbing(isekai, max_iterations=500, include_prob=0.5,
seed=None, detail=False):
    rng = random.Random(seed)
    initial_state = generate_heroine(isekai, include_prob = include_prob, rng = rng)
    return steepest_hill_climbing(initial_state, max_iterations, rng , detail)

def run_stochastic_hill_climbing(isekai, max_iterations=500, include_prob=0.5,
seed=None, detail=False):
    rng = random.Random(seed)
    initial_state = generate_heroine(isekai, include_prob = include_prob, rng = rng)
    return stochastic_hill_climbing(initial_state, max_iterations, rng , detail)

def run_sideway_hill_climbing(isekai, max_sideways=20, max_iterations=500,
include_prob=0.5, seed=None, detail=False):
    rng = random.Random(seed)
    initial_state = generate_heroine(isekai, include_prob = include_prob, rng = rng)
    return sideway_hill_climbing(initial_state, max_sideways=max_sideways,
max_iterations=max_iterations, rng=rng, detail=detail)

def run_random_restart_hc(isekai, max_restarts = 5, max_iterations = 500,
include_prob = 0.5, seed = None, detail = False):
    return random_restart_hc(isekai, max_restarts, max_iterations, include_prob,
"Steepest", 0, random.Random(seed), detail)

#contoh:
# isekai = load_isekai_from_file("../data/testcase_1.json")
# result = run_sideway_hill_climbing(isekai, max_sideways=20, max_iterations=500,
seed=42)

# print(result["final_value"])
```

3. Simulated Annealing

Implementasi Simulated Annealing mengukur kualitas solusi menggunakan fungsi evaluasi kesejahteraan. Representasi objek masalah dipetakan melalui kelas lingkungan Isekai dan kelas konfigurasi solusi IsekaiConfiguration. Pencarian tetangga dilakukan oleh fungsi random_successor yang bekerja dengan menyalin konfigurasi saat ini lalu menerapkan salah satu dari tiga langkah yang diperbolehkan pada tiap iterasi.

Annealing diatur oleh fungsi linear_schedule atau exponent_schedule (slides matkul tidak memberikan implementasi spesifik untuk fungsi schedule) yang menghasilkan fungsi penurunan suhu T, baik secara linier maupun eksponensial hingga mencapai nol (jumlah maksimum iterasi) .Inti eksekusi algoritma dijalankan oleh fungsi post_honeymoon yang mengawali proses dengan menentukan state awal baik dari konfigurasi yang sudah ada maupun dibangkitkan secara acak melalui ngasal(isekai). Pada setiap iterasi, algoritma menghitung selisih nilai evaluasi antara solusi tetangga dan solusi saat ini. Jika solusi baru menghasilkan nilai yang lebih tinggi, perubahan langsung diterima. Namun jika lebih buruk, algoritma tetap membuka peluang penerimaan jika nilai probabilitas lebih dari suatu nilai random(0,1). Mekanisme ini memungkinkan algoritma keluar dari jebakan optimum lokal yang juga dipantau secara berkala melalui deteksi plateau. Pada akhir proses, fungsi ini mengembalikan dictionary komprehensif yang memuat konfigurasi terbaik yang pernah ditemukan beserta riwayat metrik selama pencarian berlangsung

```
def random_successor(config: IsekaiConfiguration) -> IsekaiConfiguration:

    successor = config.copy()

    match random.randint(0, 2):
        case 0: kesenggol(successor)
        case 1: keguling(successor)
        case 2: ketukar(successor)

    return successor

#
https://tenor.com/view/thats-a-nice-argument-senator-why-dont-you-back-it-up-with-a-
source-gif-26196944
# https://algorithmafternoon.com/books/simulated_annealing/chapter02/

def linear_schedule(T0: float = 100.0, T_end: float = 0.0, steps: int = 1000):
    def schedule(t: int) -> float:
        if t >= steps:
            return 0.0
        return T0 - (T0 - T_end) * (t/steps)
    return schedule

def exponent_schedule(T0: float = 100.0, alpha: float = 0.67, steps: int = 1000):
    def schedule(t: int) -> float:
        if t >= steps:
            return 0.0
        return T0 * (alpha ** t)
    return schedule
```

```
def post_honeymoon(isekai: Isekai, schedule, initial_state: IsekaiConfiguration =
None, max_iterations: int = 1000, record_every: int = 1) -> dict:

    random.seed(67)
    start_time = time.perf_counter()

    if initial_state is not None:
        current = initial_state.copy()
    else:
        current = ngasal(isekai)

    current_value = kesejahteraan(current)
    initial_value = current_value
    initial_state = current.copy()

    best_state = current.copy()
    best_value = current_value

    value_history = [current_value]
    temperature_history = [schedule(0) if max_iterations > 0 else 0.0]
    deltaE_history = [0.0]
    acceptance_prob_history = [1.0]
    accepted_history = [True]
    stuck_history = [0]           # consecutive non-improving accepted steps
    stuck_count = 0               # consecutive iterations with no improvement
    local_optima_count = 0


    PLATEAU_LEN = 67
    plateau_value = current_value
    plateau_len = 0

    for t in range(1, max_iterations + 1):

        T = schedule(t)
        if T == 0:
            break

        next_state = random_successor(current)
        next_value = kesejahteraan(next_state)

        deltaE = next_value - current_value
        prob = math.exp(deltaE / T) if deltaE <= 0 else 1.0

        accepted = False
        if deltaE > 0:
            current = next_state
            current_value = next_value
            accepted = True
        else:
            if random.random() < prob:
                current = next_state
```

```
                current_value = next_value
                accepted = True


        if current_value > best_value:
            best_value = current_value
            best_state = current.copy()


        if current_value == plateau_value:
            plateau_len += 1
        else:
            plateau_value = current_value
            plateau_len = 0

        if plateau_len >= PLATEAU_LEN:
            stuck_count += 1
        else:
            stuck_count = 0

        if plateau_len >= PLATEAU_LEN and accepted and deltaE < 0:
            local_optima_count += 1

        # record
        if t % record_every == 0 or t == max_iterations:
            value_history.append(current_value)
            temperature_history.append(T)
            deltaE_history.append(deltaE)
            acceptance_prob_history.append(prob)
            accepted_history.append(accepted)
            stuck_history.append(stuck_count)

    final_value = kesejahteraan(best_state)

    duration = time.perf_counter() - start_time

    return {
        "initial_state": initial_state,
        "final_state": best_state,
        "initial_value": initial_value,
        "final_value": final_value,
        "value_history": value_history,
        "temperature_history": temperature_history,
        "deltaE_history": deltaE_history,
        "acceptance_prob_history": acceptance_prob_history,
        "accepted_history": accepted_history,
        "stuck_history": stuck_history,
        "local_optima_count": local_optima_count,
        "iterations": len(value_history) - 1,
        "max_iterations": max_iterations,
        "duration": duration,
    }
```

4. Genetic Algorithm

Genetic Algorithm versi 1

Adapun penjelasan fungsi yang ada pada algoritma ini:

- ngasal(isekai) untuk mendapatkan konfigurasi acak yang valid. Supaya algoritma cukup menyilang dan memutasi konfigurasi tersebut supaya lebih baik. Ngasal-nya dilakukan dengan cara menaroh paket pada truk acak dari atas ke bawah sampai menabrak salah satu paket atau lantai truk. Posisi x dan z pasti valid jika paket sendirian. Posisi y disesuaikan dengan tabrakan dengan paket lain. Paket yang menembus luar truk, yang tertumpuk sampai atas truk, dibuang (tidak diletakkan di truk sama sekali).
- kawin(config1, config2) untuk menyilang dua konfigurasi (*crossover*). Penyilangan dilakukan dengan cara membagi paket yang ada berdasarkan posisi y-nya. Misalkan pada suatu truk batasnya adalah Y, maka semua paket config1 yang berada di posisi (posisi pojok paketnya) di bawah Y akan masuk ke dalam offspring1 dan yang di atasnya akan masuk ke dalam offspring2, sedangkan semua paket config2 yang berada di posisi di bawah Y akan masuk ke dalam offspring2 dan yang di atasnya akan masuk ke dalam offspring1. Paket diletakkan jika dan hanya jika paket belum diletakkan sama sekali ke truk manapun.
- kesenggol(config), keguling(config), ketukar(config), kelempar(config) adalah fungsi mutasi yang fungsinya secara berurutan adalah: mutasi posisi, mutasi orientasi, menukar dua paket, serta meletakkan paket ke dalam atau ke luar truk acak.
- kelainan(config) digunakan untuk memanggil secara acak salah satu fungsi mutasi.
- dolan() adalah fungsi utama (main) untuk menjalankan algoritma genetik dengan parameter banyak populasi, jumlah iterasi, peluang mutasi, persentase elit, dan seed.

Secara garis besar, urutan pengerjaan pada algoritma ini yaitu:

1. Generate random populasi
2. Menghitung fitness dari populasi.
3. Melakukan penyilangan konfigurasi, menghasilkan offspring sebanyak 2 kali jumlah populasi awal. Pemilihan konfigurasi yang disilang dilakukan dengan Roulette Wheel Selection. Konfigurasi yang lebih fit memiliki peluang yang lebih besar untuk dipilih.
4. Melakukan mutasi pada populasi lama dan offspring.

5. Menghitung fitness dari offspring
6. Beberapa konfigurasi dari populasi awal yang paling fit dianggap sebagai elit dan langsung lolos seleksi ke generasi berikutnya. Elitism di sini diguanakan supaya hasil yang terbaik pasti tidak hilang.
7. Melakukan seleksi terhadap populasi awal dan offspring. Populasi awal juga ikut seleksi supaya hasil yang sudah bagus tidak menghilang begitu saja. Seleksi dilakukan dengan Roulette Wheel Selection. Konfigurasi yang lebih fit memiliki peluang yang lebih besar untuk dipilih.
8. Lakukan lagi langkah 2-7 untuk generasi berikutnya hingga jumlah generasi/iterasi yang ditentukan.

```
from data import *
import random
import math
import time

# random inital state
def ngasal(isekai: Isekai) -> IsekaiConfiguration:
    config = IsekaiConfiguration(isekai)

    for package in isekai.packages:
        kelempar(config, package)

    order = dict[TruckKun, list]()

    for truck in isekai.trucks:
        order[truck] = []
        for package in isekai.packages:
            if config.get_package_truck(package) is truck:
                order[truck].append(package)
        random.shuffle(order[truck])
        for package in order[truck]:
            position = config.get_package_position(package)
            position.y = 0
            for other_package in order[truck]:
                if package == other_package: break
                bound = config.calculate_bound(package)
                other_bound = config.calculate_bound(other_package)
                if not (bound[1].x <= other_bound[0].x
                        or bound[0].x >= other_bound[1].x
                        or bound[1].z <= other_bound[0].z
                        or bound[0].z >= other_bound[1].z):
                    position.y = max(position.y, other_bound[1].y)
            config.set_package_position(package, position)
```

```
    for package in isekai.packages:
        if config.package_is_outside_truck(package) or package.is_fragile and
config.package_is_supporting(package):
            config.remove_package(package)

    for truck in isekai.trucks:
        for package in order[truck]:
            if config.get_package_truck(package) is not truck:
                continue
            position = config.get_package_position(package)
            position.y = 0
            for other_package in order[truck]:
                if package == other_package: break
                if config.get_package_truck(other_package) is not truck:
                    continue
                bound = config.calculate_bound(package)
                other_bound = config.calculate_bound(other_package)
                if not (bound[1].x <= other_bound[0].x
                        or bound[0].x >= other_bound[1].x
                        or bound[1].z <= other_bound[0].z
                        or bound[0].z >= other_bound[1].z):
                    position.y = max(position.y, other_bound[1].y)
            config.set_package_position(package, position)

    return config

# crossover
def kawin(config1: IsekaiConfiguration, config2: IsekaiConfiguration):
    isekai = config1.isekai;
    other_world_1 = IsekaiConfiguration(isekai)
    other_world_2 = IsekaiConfiguration(isekai)

    for truck in isekai.trucks:
        point_of_no_return = random.randint(0, truck.dimensions.height)

        for package in isekai.packages:
            if config1.get_package_truck(package) is truck:
                position = config1.get_package_position(package)
                orientation = config1.get_package_orientation(package)
                if position.y < point_of_no_return:
                    if other_world_1.get_package_truck(package) is None:
                        other_world_1.configure_package(package, truck, position,
orientation)
                else:
                    if other_world_2.get_package_truck(package) is None:
                        other_world_2.configure_package(package, truck, position,
orientation)
            if config2.get_package_truck(package) is truck:
                position = config2.get_package_position(package)
                orientation = config2.get_package_orientation(package)
                if position.y < point_of_no_return:
                    if other_world_2.get_package_truck(package) is None:
                        other_world_2.configure_package(package, truck, position,
```

```
orientation)
                else:
                    if other_world_1.get_package_truck(package) is None:
                        other_world_1.configure_package(package, truck, position,
orientation)

    return [other_world_1, other_world_2]

# position mutation
def kesenggol(config: IsekaiConfiguration, package: LovePackage = None):
    if package is None:
        package = config.isekai.packages[random.randint(0,
len(config.isekai.packages) - 1)]

    truck = config.get_package_truck(package)
    if truck is None:
        return

    package_dimensions = config.get_package_dimensions(package)

    if (truck.dimensions.width >= package_dimensions.width and
        truck.dimensions.height >= package_dimensions.height and
        truck.dimensions.length >= package_dimensions.length):
        random_position = Position(random.randint(0, truck.dimensions.width -
package_dimensions.width),
                                   random.randint(0, truck.dimensions.height -
package_dimensions.height),
                                   random.randint(0, truck.dimensions.length -
package_dimensions.length))
        config.set_package_position(package, random_position)

# orientation mutation
def keguling(config: IsekaiConfiguration, package: LovePackage = None):
    if package is None:
        package = config.isekai.packages[random.randint(0,
len(config.isekai.packages) - 1)]
    orientation = config.get_package_orientation(package)

    if orientation is None:
        return

    match random.randint(0, 2):
        case 0:
            temp = orientation.width_orientation
            orientation.width_orientation = orientation.height_orientation
            orientation.height_orientation = temp
        case 1:
            temp = orientation.width_orientation
            orientation.width_orientation = orientation.length_orientation
            orientation.length_orientation = temp
        case 2:
            temp = orientation.length_orientation
            orientation.length_orientation = orientation.height_orientation
```

```
            orientation.height_orientation = temp
    config.set_package_orientation(package, orientation)

# menukar dua paket
def ketukar(config: IsekaiConfiguration):
    package1 = config.isekai.packages[random.randint(0, len(config.isekai.packages)
- 1)]
    package2 = config.isekai.packages[random.randint(0, len(config.isekai.packages)
- 1)]
    truck1 = config.get_package_truck(package1)
    truck2 = config.get_package_truck(package2)
    position1 = config.get_package_position(package1)
    position2 = config.get_package_position(package2)
    orientation1 = config.get_package_orientation(package1)
    orientation2 = config.get_package_orientation(package2)
    config.configure_package(package1, truck2, position2, orientation2)
    config.configure_package(package2, truck1, position1, orientation1)

# masuk keluar paket dari truck
def kelempar(config: IsekaiConfiguration, package: LovePackage = None):
    if package is None:
        package = config.isekai.packages[random.randint(0,
len(config.isekai.packages) - 1)]
    truck = config.get_package_truck(package)

    if truck is None:
        truck = config.isekai.trucks[random.randint(0, len(config.isekai.trucks) -
1)]
        orientation_value = ["x", "y", "z"]
        random.shuffle(orientation_value)
        orientation = Orientation(orientation_value[0], orientation_value[1],
orientation_value[2])
        config.set_package_orientation(package, orientation)
        package_dimensions = config.get_package_dimensions(package)
        if (truck.dimensions.width >= package_dimensions.width and
            truck.dimensions.height >= package_dimensions.height and
            truck.dimensions.length >= package_dimensions.length):
            random_position = Position(random.randint(0, truck.dimensions.width -
package_dimensions.width),
                                       random.randint(0, truck.dimensions.height -
package_dimensions.height),
                                       random.randint(0, truck.dimensions.length -
package_dimensions.length))
            config.configure_package(package, truck, random_position,
package.orientation)
    else:
        config.remove_package(package)

# mutasi
def kelainan(config: IsekaiConfiguration):
    match random.randint(0, 3):
        case 0: kesenggol(config)
        case 1: keguling(config)
```

```
        case 2: ketukar(config)
        case 3: kelempar(config)

# objective function
def kesejahteraan(config: IsekaiConfiguration, check_validity: bool = False):
    total = 0
    for truck in config.isekai.trucks:
        if not check_validity or config.is_truck_valid(truck):
            for package in config.isekai.packages:
                if config.get_package_truck(package) is truck:
                    total += package.value
    return total

# penalty
def kesengsaraan(config: IsekaiConfiguration):
    total = 0
    for truck in config.isekai.trucks:
        if (config.total_weight(truck) > truck.max_capacity):
            total -= config.total_weight(truck) - truck.max_capacity

    total_value = 0
    for package in config.isekai.packages:
        total_value += package.value
    avg_value = total_value // len(config.isekai.packages)

    for package in config.isekai.packages:
        if config.get_package_truck(package) is None:
            continue
        if config.package_is_outside_truck(package):
            total -= avg_value // 2
        if config.package_is_intersecting(package):
            total -= avg_value // 2
        if not config.package_is_supported(package):
            total -= avg_value // 2
        if package.is_fragile and config.package_is_supporting(package):
            total -= avg_value // 2

    return total

# main function
def dolan(isekai: Isekai, generation=1000, jumlah_penduduk=100,
peluang_kelainan=0.05, persen_kaum_elit=0.05, debug=False, seed:int=None) -> dict:
    random.seed(seed)

    start_time = time.perf_counter()

    orde_lama: list[IsekaiConfiguration] = []
    for i in range(jumlah_penduduk):
        orde_lama.append(ngasal(isekai))

    the_first_world: IsekaiConfiguration # best initial state
    the_first_kesejahteraan = 0 # best initial value
    the_perfect_world: IsekaiConfiguration # best final state
```

```
    the_perfect_kesejahteraan = 0
    list_termiskin = []
    list_termakmur = []
    list_terbiasa = []
    list_perfect = []

    the_perfect_world = IsekaiConfiguration(isekai)
    the_perfect_kesejahteraan = 0
    kesejahteraan_rakyat = dict[IsekaiConfiguration, int]()

    for i in range(generation):
        # Perhitungan statistik
        termiskin = 0
        termakmur = 0
        terbiasa = 0
        total_kesejahteraan = 0

        first = True
        for rakyat in orde_lama:
            if rakyat not in kesejahteraan_rakyat:
                ks = kesejahteraan(rakyat) + kesengsaraan(rakyat)
                kesejahteraan_rakyat[rakyat] = ks
            else:
                ks = kesejahteraan_rakyat[rakyat]
            total_kesejahteraan += ks
            if rakyat.is_valid() and ks > the_perfect_kesejahteraan:
                the_perfect_world = rakyat.copy()
                the_perfect_kesejahteraan = ks
            if first:
                first = False
                termiskin = ks
                termakmur = ks
            else:
                termiskin = min(termiskin, ks)
                termakmur = max(termakmur, ks)
        terbiasa = total_kesejahteraan / jumlah_penduduk

        # Buat initial state
        if i == 0:
            the_first_world = the_perfect_world
            the_first_kesejahteraan = the_perfect_kesejahteraan

        list_termiskin.append(termiskin)
        list_termakmur.append(termakmur)
        list_terbiasa.append(terbiasa)
        list_perfect.append(the_perfect_kesejahteraan)

        if debug:
            print(f"Generation {i}:")
            print(f" -> termakmur =", termakmur)
            print(f" -> terbiasa =", terbiasa)
            print(f" -> termiskin =", termiskin)
            print(f" -> varietas =", len(kesejahteraan_rakyat))
```

```
        # Anggapannya seperti menggeser semua kesejahteraan biar tidak ada yang
negatif dan nol
        if termiskin <= 0:
            total_kesejahteraan += (-termiskin + 1) * jumlah_penduduk

        # Generate offspring
        orde_baru = list[IsekaiConfiguration]()
        for _ in range(jumlah_penduduk):
            rakyat1: IsekaiConfiguration
            rakyat2: IsekaiConfiguration

            random_pilih1 = random.randint(0, total_kesejahteraan - 1)
            random_pilih2 = random.randint(0, total_kesejahteraan - 1)
            for rakyat in orde_lama:
                if random_pilih1 < kesejahteraan_rakyat[rakyat] + max(0, -termiskin
+ 1):
                    rakyat1 = rakyat
                    break
                random_pilih1 -= kesejahteraan_rakyat[rakyat] + max(0, -termiskin +
1)
            for rakyat in orde_lama:
                if random_pilih2 < kesejahteraan_rakyat[rakyat] + max(0, -termiskin
+ 1):
                    rakyat2 = rakyat
                    break
                random_pilih2 -= kesejahteraan_rakyat[rakyat] + max(0, -termiskin +
1)

            bocah = kawin(rakyat1, rakyat2)
            orde_baru.append(bocah[0])
            orde_baru.append(bocah[1])

        # Mutasi populasi lama dan offspring, kecuali kaum elit dari populasi lama
        # Sekalian hitung ulang kesejahteraan populasi lama jika perlu
        # Hitung juga kesejahteraan offspring karena belum
        orde_lama.sort(key = lambda rakyat: kesejahteraan_rakyat[rakyat],
reverse=True)
        for i in range(math.ceil(persen_kaum_elit * len(orde_lama)),
len(orde_lama)):
            if random.random() < peluang_kelainan:
                kelainan(orde_lama[i])
                kesejahteraan_rakyat[orde_lama[i]] = kesejahteraan(orde_lama[i]) +
kesengsaraan(orde_lama[i])
        for i in range(len(orde_baru)):
            if random.random() < peluang_kelainan:
                kelainan(orde_baru[i])
            kesejahteraan_rakyat[orde_baru[i]] = kesejahteraan(orde_baru[i]) +
kesengsaraan(orde_baru[i])

        # Hitung ulang kesejahteraan, buat seleksi nanti
        total_kesejahteraan = 0
        first = True
```

```
        for rakyat in orde_lama:
            total_kesejahteraan += kesejahteraan_rakyat[rakyat]
            if first:
                first = False
                termiskin = kesejahteraan_rakyat[rakyat]
            else:
                termiskin = min(termiskin, kesejahteraan_rakyat[rakyat])
        for rakyat in orde_baru:
            total_kesejahteraan += kesejahteraan_rakyat[rakyat]
            if first:
                first = False
                termiskin = kesejahteraan_rakyat[rakyat]
            else:
                termiskin = min(termiskin, kesejahteraan_rakyat[rakyat])

        # Anggapannya seperti menggeser semua kesejahteraan biar tidak ada yang
negatif dan nol
        if termiskin <= 0:
            total_kesejahteraan += (-termiskin + 1) * 3 * jumlah_penduduk

        # Seleksi dari populasi lama dan baru, kaum elit ngebayar biar langsung
lolos seleksi
        orde_terbaru = []
        for i in range(0, math.ceil(persen_kaum_elit * len(orde_lama))):
            orde_terbaru.append(orde_lama[i])
        while len(orde_terbaru) < jumlah_penduduk:
            sudah_pilih = False
            random_pilih = random.randint(0, total_kesejahteraan - 1)
            for rakyat in orde_lama:
                if random_pilih < kesejahteraan_rakyat[rakyat] + max(0, termiskin +
1):
                    orde_terbaru.append(rakyat)
                    sudah_pilih = True
                    break
                random_pilih -= kesejahteraan_rakyat[rakyat] + max(0, termiskin + 1)
            if sudah_pilih: continue
            for rakyat in orde_baru:
                if random_pilih < kesejahteraan_rakyat[rakyat] + max(0, termiskin +
1):
                    orde_terbaru.append(rakyat)
                    break
                random_pilih -= kesejahteraan_rakyat[rakyat] + max(0, termiskin + 1)

        # Hasil yang betulan valid, ga cuma dari fitness
        for rakyat, ks in kesejahteraan_rakyat.items():
            if rakyat.is_valid() and ks > the_perfect_kesejahteraan:
                the_perfect_world = rakyat.copy()
                the_perfect_kesejahteraan = ks

        # Hapus konfigurasi yang ga masuk jadi generasi berikutnya
        rakyat_mati = []
        for rakyat in kesejahteraan_rakyat.keys():
            if rakyat not in orde_terbaru:
```

```
                rakyat_mati.append(rakyat)

        for rakyat in rakyat_mati:
            kesejahteraan_rakyat.pop(rakyat)

        orde_lama = orde_terbaru

    # Hitung statistik akhir
    termiskin = 0
    termakmur = 0
    terbiasa = 0
    total_kesejahteraan = 0

    first = True
    for rakyat in orde_lama:
        if rakyat not in kesejahteraan_rakyat:
            ks = kesejahteraan(rakyat) + kesengsaraan(rakyat)
            kesejahteraan_rakyat[rakyat] = ks
        else:
            ks = kesejahteraan_rakyat[rakyat]
        total_kesejahteraan += ks
        if rakyat.is_valid() and ks > the_perfect_kesejahteraan:
            the_perfect_world = rakyat.copy()
            the_perfect_kesejahteraan = ks
        if first:
            first = False
            termiskin = ks
            termakmur = ks
        else:
            termiskin = min(termiskin, ks)
            termakmur = max(termakmur, ks)
    terbiasa = total_kesejahteraan / jumlah_penduduk

    list_termiskin.append(termiskin)
    list_termakmur.append(termakmur)
    list_terbiasa.append(terbiasa)
    list_perfect.append(the_perfect_kesejahteraan)

    if debug:
        print(f"Generation {generation}:")
        print(f" -> termakmur =", termakmur)
        print(f" -> terbiasa =", terbiasa)
        print(f" -> termiskin =", termiskin)
        print(f" -> varietas =", len(kesejahteraan_rakyat))

    return  {
        "initial_state": the_first_world,
        "final_state": the_perfect_world,
        "initial_value": the_first_kesejahteraan,
        "final_value": the_perfect_kesejahteraan,
        "best_value_history": list_perfect,
        "max_fitness_history": list_termakmur,
        "avg_fitness_history": list_terbiasa,
```

```
        "min_fitness_history": list_termiskin,
        "iterations": generation,
        "populations": jumlah_penduduk,
        "duration": time.perf_counter() - start_time
    }
```

Genetic Algorithm versi 2

Secara garis besar, algoritma genetik versi 2 ini sama. Perbedaannya hanya di bagian cara penyilangannya. Ide utama penyilangan kali ini adalah dengan menyilang urutan posisi y paket pada tiap truk. Posisi y tidak digenerate secara langsung, melainkan melihat dari urutan paket. Seperti pada algoritma pembuatan konfigurasi awal, posisi y akan ditentukan dengan menyimulasikan peletakan paket dari atas truk ke bawah. Karena versi ini membutuhkan urutan paket pada tiap truk, dibuatlah struktur data IsekaiOrder yang menginherit IsekaiConfiguration. Fungsi mutasi dan set yang ada harus memperhatikan urutan paket. Ada satu tambahan fungsi mutasi yaitu kejengkang() yang mengubah urutan satu paket untuk diselipkan ulang pada urutan di truk yang sedang ditempati.

Fungsi utama hampir tidak berubah sama sekali. Langkah-langkah algoritma masih sama dengan versi 1.

```
from __future__ import annotations
from data import *
import random
import math
import time

class IsekaiOrder(IsekaiConfiguration):
    def __init__(self, isekai: Isekai):
        super().__init__(isekai)
        self.package_order = dict[TruckKun, list[LovePackage]]()
        for truck in isekai.trucks:
            self.package_order[truck] = list[LovePackage]()


    def configure_package(self, package: LovePackage, truck: TruckKun, position:
Position = Position(0, 0, 0), orientation: Orientation = Orientation(), order: int =
-1):
        prev_truck = self.get_package_truck(package)
        if prev_truck is not None:
            self.package_order[prev_truck].remove(package)
        super().configure_package(package, truck, position, orientation)
        self.set_package_order(package, order)
```

```
    def remove_package(self, package: LovePackage):
        truck = self.get_package_truck(package)
        if truck is not None and package in self.package_order[truck]:
            self.package_order[truck].remove(package)
        super().remove_package(package)


    def set_package_truck(self, package: LovePackage, truck: TruckKun, order: int =
-1):
        prev_truck = self.get_package_truck(package)
        if prev_truck is not None:
            self.package_order[prev_truck].remove(package)

        self.package_truck[package] = truck

        if truck is not None:
            if order == -1:
                order = random.randint(0, len(self.package_order[truck]))
            self.package_order[truck].insert(order, package)


    def set_package_order(self, package: LovePackage, order: int) -> bool:
        truck = self.get_package_truck(package)
        if truck is not None:
            if package in self.package_order[truck]:
                self.package_order[truck].remove(package)
            if order == -1:
                order = random.randint(0, len(self.package_order[truck]))
            self.package_order[truck].insert(order, package)


    def get_package_order(self, package: LovePackage) -> bool:
        truck = self.get_package_truck(package)
        if truck is not None and package in self.package_order[truck]:
            return self.package_order[truck].index(package)
        else:
            return -1


    def recalculate_position(self):
        for packages in self.package_order.values():
            for package in packages:
                position = self.get_package_position(package)
                position.y = 0
                for other_package in packages:
                    if package == other_package: break
                    bound = self.calculate_bound(package)
                    other_bound = self.calculate_bound(other_package)
                    if not (bound[1].x <= other_bound[0].x
                            or bound[0].x >= other_bound[1].x
                            or bound[1].z <= other_bound[0].z
```

```
                            or bound[0].z >= other_bound[1].z):
                        position.y = max(position.y, other_bound[1].y)


    def copy(self) -> IsekaiOrder:
        config = IsekaiOrder(self.isekai)
        config.package_truck = self.package_truck.copy()
        config.package_position = self.package_position.copy()
        config.package_orientation = self.package_orientation.copy()
        for truck in self.isekai.trucks:
            config.package_order[truck] = self.package_order[truck].copy()
        return config

# random state
def ngasal(isekai: Isekai) -> IsekaiOrder:
    config = IsekaiOrder(isekai)

    for package in isekai.packages:
        kelempar(config, package)

    for truck in isekai.trucks:
        config.package_order[truck] = []
        for package in isekai.packages:
            if config.get_package_truck(package) is truck:
                config.package_order[truck].append(package)
        random.shuffle(config.package_order[truck])
        for package in config.package_order[truck]:
            position = config.get_package_position(package)
            position.y = 0
            for other_package in config.package_order[truck]:
                if package == other_package: break
                bound = config.calculate_bound(package)
                other_bound = config.calculate_bound(other_package)
                if not (bound[1].x <= other_bound[0].x
                        or bound[0].x >= other_bound[1].x
                        or bound[1].z <= other_bound[0].z
                        or bound[0].z >= other_bound[1].z):
                    position.y = max(position.y, other_bound[1].y)
            config.set_package_position(package, position)

    for package in isekai.packages:
        if config.package_is_outside_truck(package) or package.is_fragile and
config.package_is_supporting(package):
            config.remove_package(package)

    for truck in isekai.trucks:
        for package in config.package_order[truck]:
            if config.get_package_truck(package) is not truck:
                continue
            position = config.get_package_position(package)
            position.y = 0
            for other_package in config.package_order[truck]:
                if package == other_package: break
```

```
                if config.get_package_truck(other_package) is not truck:
                    continue
                bound = config.calculate_bound(package)
                other_bound = config.calculate_bound(other_package)
                if not (bound[1].x <= other_bound[0].x
                        or bound[0].x >= other_bound[1].x
                        or bound[1].z <= other_bound[0].z
                        or bound[0].z >= other_bound[1].z):
                    position.y = max(position.y, other_bound[1].y)
            config.set_package_position(package, position)

    return config

# crossover
def kawin(config1: IsekaiOrder, config2: IsekaiOrder):
    isekai = config1.isekai;
    other_world_1 = config1.copy()
    other_world_2 = config2.copy()

    configured_1 = dict()
    configured_2 = dict()

    for truck in isekai.trucks:
        point_of_no_return_1 = random.randint(0, len(config1.package_order[truck]))
        point_of_no_return_2 = random.randint(0, len(config2.package_order[truck]))
        other_world_1.package_order[truck].clear()
        other_world_2.package_order[truck].clear()

        for i in range(0, point_of_no_return_1):
            package = config1.package_order[truck][i]
            if package in configured_1: continue
            configured_1[package] = True
            other_world_1.package_truck[package] = truck
            other_world_1.package_position[package] =
config1.get_package_position(package)
            other_world_1.package_orientation[package] =
config1.get_package_orientation(package)
            other_world_1.package_order[truck].append(package)
        for i in range(0, point_of_no_return_2):
            package = config2.package_order[truck][i]
            if package in configured_2: continue
            configured_2[package] = True
            other_world_2.package_truck[package] = truck
            other_world_2.package_position[package] =
config2.get_package_position(package)
            other_world_2.package_orientation[package] =
config2.get_package_orientation(package)
            other_world_2.package_order[truck].append(package)
        for i in range(point_of_no_return_1, len(config1.package_order[truck])):
            package = config1.package_order[truck][i]
            if package in configured_2: continue
            configured_2[package] = True
            other_world_2.package_truck[package] = truck
```

```
            other_world_2.package_position[package] =
config1.get_package_position(package)
            other_world_2.package_orientation[package] =
config1.get_package_orientation(package)
            other_world_2.package_order[truck].append(package)
        for i in range(point_of_no_return_2, len(config2.package_order[truck])):
            package = config2.package_order[truck][i]
            if package in configured_1: continue
            configured_1[package] = True
            other_world_1.package_truck[package] = truck
            other_world_1.package_position[package] =
config2.get_package_position(package)
            other_world_1.package_orientation[package] =
config2.get_package_orientation(package)
            other_world_1.package_order[truck].append(package)

    for package in isekai.packages:
        if package not in configured_1 and other_world_1.get_package_truck(package)
is not None:
            other_world_1.remove_package(package)
        if package not in configured_2 and other_world_2.get_package_truck(package)
is not None:
            other_world_2.remove_package(package)

    return [other_world_1, other_world_2]

# position mutation
def kesenggol(config: IsekaiOrder, package: LovePackage = None):
    if package is None:
        package = config.isekai.packages[random.randint(0,
len(config.isekai.packages) - 1)]

    truck = config.get_package_truck(package)
    if truck is None:
        return

    package_dimensions = config.get_package_dimensions(package)

    if (truck.dimensions.width >= package_dimensions.width and
        truck.dimensions.height >= package_dimensions.height and
        truck.dimensions.length >= package_dimensions.length):
        random_position = Position(random.randint(0, truck.dimensions.width -
package_dimensions.width),
                                   random.randint(0, truck.dimensions.height -
package_dimensions.height),
                                   random.randint(0, truck.dimensions.length -
package_dimensions.length))
        config.set_package_position(package, random_position)

# orientation mutation
def keguling(config: IsekaiOrder, package: LovePackage = None):
    if package is None:
        package = config.isekai.packages[random.randint(0,
```

```
len(config.isekai.packages) - 1)]
    orientation = config.get_package_orientation(package)

    if orientation is None:
        return

    match random.randint(0, 2):
        case 0:
            temp = orientation.width_orientation
            orientation.width_orientation = orientation.height_orientation
            orientation.height_orientation = temp
        case 1:
            temp = orientation.width_orientation
            orientation.width_orientation = orientation.length_orientation
            orientation.length_orientation = temp
        case 2:
            temp = orientation.length_orientation
            orientation.length_orientation = orientation.height_orientation
            orientation.height_orientation = temp
    config.set_package_orientation(package, orientation)

# menukar dua paket
def ketukar(config: IsekaiOrder):
    package1 = config.isekai.packages[random.randint(0, len(config.isekai.packages)
- 1)]
    package2 = config.isekai.packages[random.randint(0, len(config.isekai.packages)
- 1)]
    if package1 is package2:
        return
    truck1 = config.get_package_truck(package1)
    truck2 = config.get_package_truck(package2)
    order1 = config.get_package_order(package1)
    order2 = config.get_package_order(package2)
    position1 = config.get_package_position(package1)
    position2 = config.get_package_position(package2)
    orientation1 = config.get_package_orientation(package1)
    orientation2 = config.get_package_orientation(package2)
    config.configure_package(package1, truck2, position2, orientation2, order2)
    config.configure_package(package2, truck1, position1, orientation1, order1)

# masuk keluar paket dari truck
def kelempar(config: IsekaiOrder, package: LovePackage = None):
    if package is None:
        package = config.isekai.packages[random.randint(0,
len(config.isekai.packages) - 1)]
    truck = config.get_package_truck(package)

    if truck is None:
        truck = config.isekai.trucks[random.randint(0, len(config.isekai.trucks) -
1)]
        orientation_value = ["x", "y", "z"]
        random.shuffle(orientation_value)
        orientation = Orientation(orientation_value[0], orientation_value[1],
```

```
orientation_value[2])
        config.set_package_orientation(package, orientation)
        package_dimensions = config.get_package_dimensions(package)
        if (truck.dimensions.width >= package_dimensions.width and
            truck.dimensions.height >= package_dimensions.height and
            truck.dimensions.length >= package_dimensions.length):
            random_position = Position(random.randint(0, truck.dimensions.width -
package_dimensions.width),
                                       random.randint(0, truck.dimensions.height -
package_dimensions.height),
                                       random.randint(0, truck.dimensions.length -
package_dimensions.length))
            config.configure_package(package, truck, random_position,
package.orientation)
    else:
        config.remove_package(package)

# acak urutan
def kejengkang(config: IsekaiOrder, package: LovePackage = None):
    if package is None:
        package = config.isekai.packages[random.randint(0,
len(config.isekai.packages) - 1)]

    truck = config.get_package_truck(package)
    if truck is not None:
        order = random.randint(0, len(config.package_order[truck]))
        config.set_package_order(package, order)

# mutasi
def kelainan(config: IsekaiOrder):
    match random.randint(0, 4):
        case 0: kesenggol(config)
        case 1: keguling(config)
        case 2: ketukar(config)
        case 3: kelempar(config)
        case 4: kejengkang(config)

# objective function
def kesejahteraan(config: IsekaiConfiguration, check_validity: bool = False):
    total = 0
    for truck in config.isekai.trucks:
        if not check_validity or config.is_truck_valid(truck):
            for package in config.isekai.packages:
                if config.get_package_truck(package) is truck:
                    total += package.value
    return total

# penalty
def kesengsaraan(config: IsekaiConfiguration):
    total = 0
    for truck in config.isekai.trucks:
        if (config.total_weight(truck) > truck.max_capacity):
            total -= config.total_weight(truck) - truck.max_capacity
```

```
    total_value = 0
    for package in config.isekai.packages:
        total_value += package.value
    avg_value = round(math.sqrt(total_value // len(config.isekai.packages)))

    for package in config.isekai.packages:
        if config.get_package_truck(package) is None:
            continue
        if config.package_is_outside_truck(package):
            total -= avg_value // 2
        if config.package_is_intersecting(package):
            total -= avg_value // 2
        if not config.package_is_supported(package):
            total -= avg_value // 2
        if package.is_fragile and config.package_is_supporting(package):
            total -= avg_value // 2

    return total

# main function
def dolan(isekai: Isekai, generation=1000, jumlah_penduduk=100,
peluang_kelainan=0.05, persen_kaum_elit=0.05, debug=False, seed:int=None) -> dict:
    random.seed(seed)

    start_time = time.perf_counter()

    orde_lama: list[IsekaiOrder] = []
    for i in range(jumlah_penduduk):
        orde_lama.append(ngasal(isekai))

    the_first_world: IsekaiConfiguration # best initial state
    the_first_kesejahteraan = 0 # best initial value
    the_perfect_world: IsekaiConfiguration # best final state
    the_perfect_kesejahteraan = 0
    list_termiskin = []
    list_termakmur = []
    list_terbiasa = []
    list_perfect = []

    the_perfect_world = IsekaiConfiguration(isekai)
    the_perfect_kesejahteraan = 0
    kesejahteraan_rakyat = dict[IsekaiOrder, int]()

    for i in range(generation):
        # Perhitungan statistik
        termiskin = 0
        termakmur = 0
        terbiasa = 0
        total_kesejahteraan = 0

        first = True
        for rakyat in orde_lama:
```

```
            if rakyat not in kesejahteraan_rakyat:
                rakyat.recalculate_position()
                ks = kesejahteraan(rakyat) + kesengsaraan(rakyat)
                kesejahteraan_rakyat[rakyat] = ks
            else:
                ks = kesejahteraan_rakyat[rakyat]
            total_kesejahteraan += ks
            if rakyat.is_valid() and ks > the_perfect_kesejahteraan:
                the_perfect_world = rakyat.copy()
                the_perfect_kesejahteraan = ks
            if first:
                first = False
                termiskin = ks
                termakmur = ks
            else:
                termiskin = min(termiskin, ks)
                termakmur = max(termakmur, ks)
        terbiasa = total_kesejahteraan / jumlah_penduduk

        # Buat initial state
        if i == 0:
            the_first_world = the_perfect_world
            the_first_kesejahteraan = the_perfect_kesejahteraan

        list_termiskin.append(termiskin)
        list_termakmur.append(termakmur)
        list_terbiasa.append(terbiasa)
        list_perfect.append(the_perfect_kesejahteraan)

        if debug:
            print(f"Generation {i}:")
            print(f" -> termakmur =", termakmur)
            print(f" -> terbiasa =", terbiasa)
            print(f" -> termiskin =", termiskin)
            print(f" -> varietas =", len(kesejahteraan_rakyat))

        # Anggapannya seperti menggeser semua kesejahteraan biar tidak ada yang
negatif dan nol
        if termiskin <= 0:
            total_kesejahteraan += (-termiskin + 1) * jumlah_penduduk

        # Generate offspring
        orde_baru = list[IsekaiOrder]()
        for _ in range(jumlah_penduduk):
            rakyat1: IsekaiConfiguration
            rakyat2: IsekaiConfiguration

            random_pilih1 = random.randint(0, total_kesejahteraan - 1)
            random_pilih2 = random.randint(0, total_kesejahteraan - 1)
            for rakyat in orde_lama:
                if random_pilih1 < kesejahteraan_rakyat[rakyat] + max(0, -termiskin
+ 1):
                    rakyat1 = rakyat
```

```
                    break
                random_pilih1 -= kesejahteraan_rakyat[rakyat] + max(0, -termiskin +
1)
            for rakyat in orde_lama:
                if random_pilih2 < kesejahteraan_rakyat[rakyat] + max(0, -termiskin
+ 1):
                    rakyat2 = rakyat
                    break
                random_pilih2 -= kesejahteraan_rakyat[rakyat] + max(0, -termiskin +
1)

            bocah = kawin(rakyat1, rakyat2)
            orde_baru.append(bocah[0])
            orde_baru.append(bocah[1])

        # Mutasi populasi lama dan offspring, kecuali kaum elit dari populasi lama
        # Sekalian hitung ulang kesejahteraan populasi lama jika perlu
        # Hitung juga kesejahteraan offspring karena belum
        orde_lama.sort(key = lambda rakyat: kesejahteraan_rakyat[rakyat],
reverse=True)
        for i in range(math.ceil(persen_kaum_elit * len(orde_lama)),
len(orde_lama)):
            if random.random() < peluang_kelainan:
                kelainan(orde_lama[i])
                orde_lama[i].recalculate_position()
                kesejahteraan_rakyat[orde_lama[i]] = kesejahteraan(orde_lama[i]) +
kesengsaraan(orde_lama[i])
        for i in range(len(orde_baru)):
            if random.random() < peluang_kelainan:
                kelainan(orde_baru[i])
                orde_baru[i].recalculate_position()
            kesejahteraan_rakyat[orde_baru[i]] = kesejahteraan(orde_baru[i]) +
kesengsaraan(orde_baru[i])

        # Hitung ulang kesejahteraan, buat seleksi nanti
        total_kesejahteraan = 0
        first = True
        for rakyat in orde_lama:
            total_kesejahteraan += kesejahteraan_rakyat[rakyat]
            if first:
                first = False
                termiskin = kesejahteraan_rakyat[rakyat]
            else:
                termiskin = min(termiskin, kesejahteraan_rakyat[rakyat])
        for rakyat in orde_baru:
            total_kesejahteraan += kesejahteraan_rakyat[rakyat]
            if first:
                first = False
                termiskin = kesejahteraan_rakyat[rakyat]
            else:
                termiskin = min(termiskin, kesejahteraan_rakyat[rakyat])

        # Anggapannya seperti menggeser semua kesejahteraan biar tidak ada yang
```

```
negatif dan nol
        if termiskin <= 0:
            total_kesejahteraan += (-termiskin + 1) * 3 * jumlah_penduduk

        # Seleksi dari populasi lama dan baru, kaum elit ngebayar biar langsung
lolos seleksi
        orde_terbaru = []
        for i in range(0, math.ceil(persen_kaum_elit * len(orde_lama))):
            orde_terbaru.append(orde_lama[i])
        while len(orde_terbaru) < jumlah_penduduk:
            sudah_pilih = False
            random_pilih = random.randint(0, total_kesejahteraan - 1)
            for rakyat in orde_lama:
                if random_pilih < kesejahteraan_rakyat[rakyat] + max(0, termiskin +
1):
                    orde_terbaru.append(rakyat)
                    sudah_pilih = True
                    break
                random_pilih -= kesejahteraan_rakyat[rakyat] + max(0, termiskin + 1)
            if sudah_pilih: continue
            for rakyat in orde_baru:
                if random_pilih < kesejahteraan_rakyat[rakyat] + max(0, termiskin +
1):
                    orde_terbaru.append(rakyat)
                    break
                random_pilih -= kesejahteraan_rakyat[rakyat] + max(0, termiskin + 1)

        # Hasil yang betulan valid, ga cuma dari fitness
        for rakyat, ks in kesejahteraan_rakyat.items():
            if rakyat.is_valid() and ks > the_perfect_kesejahteraan:
                the_perfect_world = rakyat.copy()
                the_perfect_kesejahteraan = ks

        # Hapus konfigurasi yang ga masuk jadi generasi berikutnya
        rakyat_mati = []
        for rakyat in kesejahteraan_rakyat.keys():
            if rakyat not in orde_terbaru:
                rakyat_mati.append(rakyat)

        for rakyat in rakyat_mati:
            kesejahteraan_rakyat.pop(rakyat)

        orde_lama = orde_terbaru

    # Hitung statistik akhir
    termiskin = 0
    termakmur = 0
    terbiasa = 0
    total_kesejahteraan = 0

    first = True
    for rakyat in orde_lama:
        if rakyat not in kesejahteraan_rakyat:
```

```
            rakyat.recalculate_position()
            ks = kesejahteraan(rakyat) + kesengsaraan(rakyat)
            kesejahteraan_rakyat[rakyat] = ks
        else:
            ks = kesejahteraan_rakyat[rakyat]
        total_kesejahteraan += ks
        if rakyat.is_valid() and ks > the_perfect_kesejahteraan:
            the_perfect_world = rakyat.copy()
            the_perfect_kesejahteraan = ks
        if first:
            first = False
            termiskin = ks
            termakmur = ks
        else:
            termiskin = min(termiskin, ks)
            termakmur = max(termakmur, ks)
    terbiasa = total_kesejahteraan / jumlah_penduduk

    list_termiskin.append(termiskin)
    list_termakmur.append(termakmur)
    list_terbiasa.append(terbiasa)
    list_perfect.append(the_perfect_kesejahteraan)

    if debug:
        print(f"Generation {generation}:")
        print(f" -> termakmur =", termakmur)
        print(f" -> terbiasa =", terbiasa)
        print(f" -> termiskin =", termiskin)
        print(f" -> varietas =", len(kesejahteraan_rakyat))

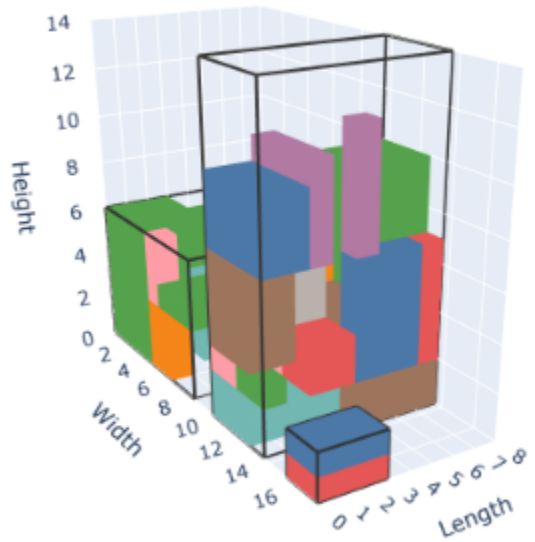
    return  {
        "initial_state": the_first_world,
        "final_state": the_perfect_world,
        "initial_value": the_first_kesejahteraan,
        "final_value": the_perfect_kesejahteraan,
        "best_value_history": list_perfect,
        "max_value_history": list_termakmur,
        "avg_value_history": list_terbiasa,
        "min_value_history": list_termiskin,
        "iterations": generation,
        "populations": jumlah_penduduk,
        "duration": time.perf_counter() - start_time
    }
```

Hasil dan Analisis Eksperimen

Eksperimen akan dilakukan pada testcase data/3_truck_50_packages.json

1. Hill Climbing

Hasil Eksperimen

Test Case	Algoritma	Hasil
3_truck_50_packages	Hill Climbing Steepest Ascend	  ===== Ringkasan hasil ===== Algoritma : Hill Climbing (HC) Testcase : 3_trucks_50_packages.json Mode HC : Steepest Nilai terbaik yang mungkin : 20000 Nilai awal : 11193 Nilai akhir : 12842 State akhir : valid

Iterasi/generasi: 9/100
Durasi : 65.763 detik
Alasan berhenti : sudah Local Optimum

Ringkasan hasil

Algoritma	Nilai awal	Nilai akhir	State akhir valid	Iterasi/generasi	Durasi
hc	7852	12887	Ya	12	104.205 detik

State awal

Truk Truck1

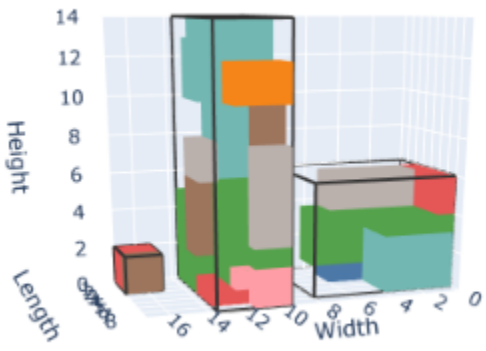
- **Package5** — posisi (0, 0, 4); ukuran (5 × 1 × 3); nilai 310, berat 29
- **Package7** — posisi (0, 1, 0); ukuran (1 × 3 × 1); nilai 88, berat 8
- **Package12** — posisi (5, 0, 4); ukuran (1 × 1 × 2); nilai 83, berat 7
- **Package13** — posisi (7, 2, 0); ukuran (1 × 1 × 5); nilai 200, berat 18
- **Package16** — posisi (3, 2, 0); ukuran (4 × 2 × 1); nilai 312, berat 26
- **Package18** — posisi (0, 4, 0); ukuran (5 × 2 × 1); nilai 298, berat 27
- **Package30** — posisi (0, 1, 1); ukuran (3 × 3 × 5); nilai 1475, berat 116
- **Package39** — posisi (3, 0, 0); ukuran (5 × 2 × 4); nilai 1210, berat 99
- **Package48** — posisi (0, 0, 0); ukuran (3 × 1 × 4); nilai 332, berat 26

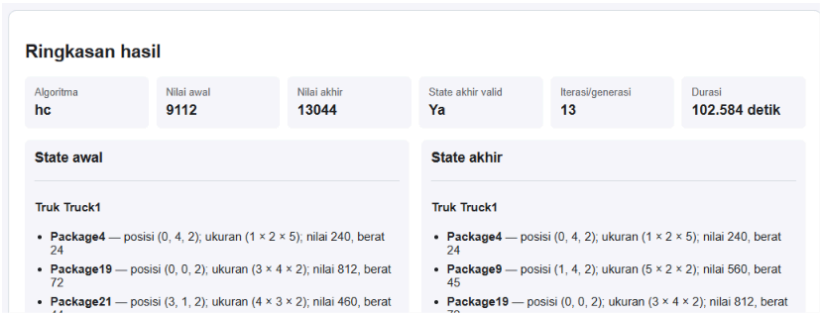
State akhir

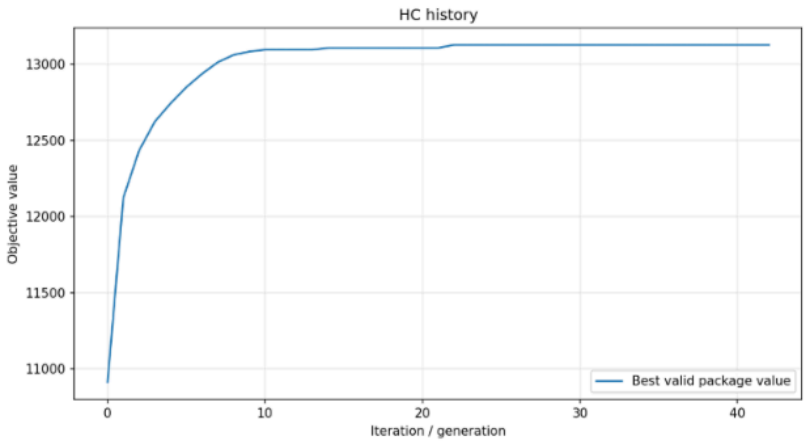
Truk Truck1

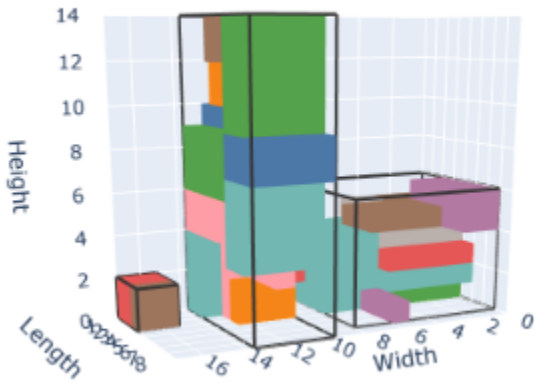
- **Package5** — posisi (0, 0, 4); ukuran (5 × 1 × 3); nilai 310, berat 29
- **Package7** — posisi (0, 1, 0); ukuran (1 × 3 × 1); nilai 88, berat 8
- **Package13** — posisi (7, 2, 0); ukuran (1 × 1 × 5); nilai 200, berat 18
- **Package16** — posisi (3, 2, 0); ukuran (4 × 2 × 1); nilai 312, berat 26
- **Package18** — posisi (0, 4, 0); ukuran (5 × 2 × 1); nilai 298, berat 27
- **Package26** — posisi (5, 0, 4); ukuran (2 × 4 × 3); nilai 634, berat 51
- **Package30** — posisi (0, 1, 1); ukuran (3 × 3 × 5); nilai 1475, berat 116
- **Package39** — posisi (3, 0, 0); ukuran (5 × 2 × 4); nilai 1210, berat 99
- **Package48** — posisi (0, 0, 0); ukuran (3 × 1 × 4); nilai 332, berat 26

===== Ringkasan hasil =====
Algoritma : Hill Climbing (HC)
Testcase : 3_trucks_50_packages.json
Mode HC : Steepest
Nilai terbaik yang mungkin : 20000
Nilai awal : 7852
Nilai akhir : 12887
State akhir : valid
Iterasi/generasi: 12/100
Durasi : 104.205 detik
Alasan berhenti : sudah Local Optimum



		 ===== Ringkasan hasil ===== Algoritma : Hill Climbing (HC) Testcase : 3_trucks_50_packages.json Mode HC : Steepest Nilai terbaik yang mungkin : 20000 Nilai awal : 9112 Nilai akhir : 13044 State akhir : valid Iterasi/generasi: 13/100 Durasi : 102.584 detik Alasan berhenti : sudah Local Optimum
	Hill Climbing Sideway Move	===== Ringkasan hasil ===== Algoritma : Hill Climbing (HC) Testcase : 3_trucks_50_packages.json Mode HC : Sideway Sideway : 30/20 Nilai terbaik yang mungkin : 20000 Nilai awal : 10913 Nilai akhir : 13127 State akhir : valid Iterasi/generasi: 42/100 Durasi : 246.892 detik Alasan berhenti : sideway limit tercapai





Ringkasan hasil

Algoritma	Nilai awal	Nilai akhir	State akhir valid	Iterasi/generasi	Durasi
hc	10913	13127	Ya	42	246.892 detik

State awal

Truk Truck1

- **Package7** — posisi (4, 0, 0); ukuran (1 × 3 × 1); nilai 88, berat 8
- **Package8** — posisi (5, 0, 0); ukuran (3 × 1 × 4); nilai 340, berat 32
- **Package9** — posisi (0, 1, 0); ukuran (2 × 5 × 2); nilai 560, berat 45

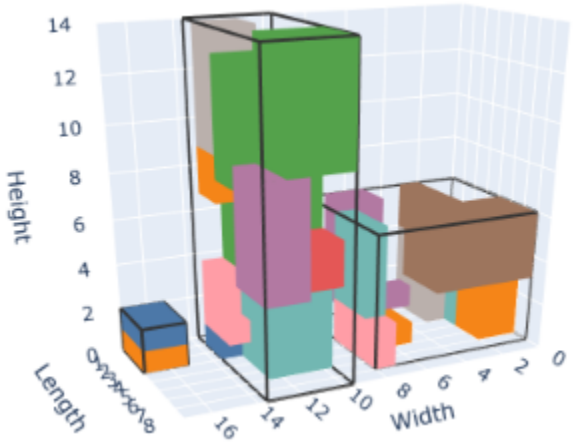
State akhir

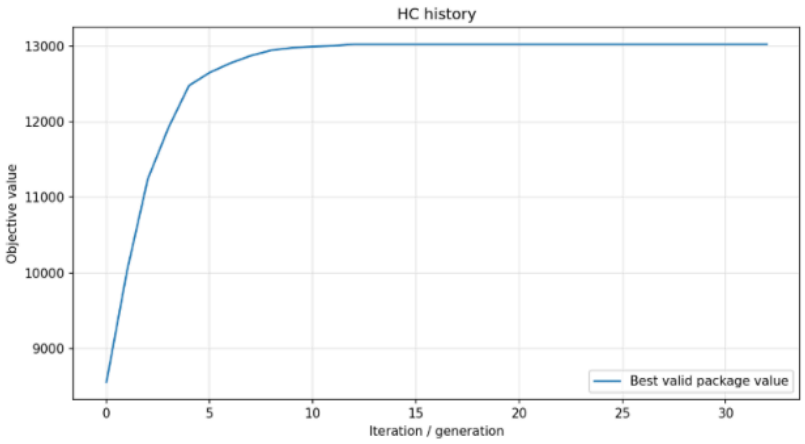
Truk Truck1

- **Package5** — posisi (0, 1, 2); ukuran (5 × 1 × 3); nilai 310, berat 29
- **Package6** — posisi (5, 0, 4); ukuran (1 × 1 × 3); nilai 110, berat 9
- **Package8** — posisi (2, 4, 2); ukuran (4 × 1 × 3); nilai 340, berat 32

```
====== Ringkasan hasil ======
Algoritma       : Hill Climbing (HC)
Testcase        : 3_trucks_50_packages.json
Mode HC         : Sideway
Sideway         : 20/20
Nilai terbaik yang mungkin : 20000
Nilai awal      : 8554
Nilai akhir     : 13027
State akhir     : valid
Iterasi/generasi: 32/100
```

Durasi : 197.870 detik
Alasan berhenti : sideway limit tercapai





Ringkasan hasil

Algoritma	Nilai awal	Nilai akhir	State akhir valid	Iterasi/generasi	Durasi
hc	8554	13027	Ya	32	197.870 detik

State awal

Truk Truck1

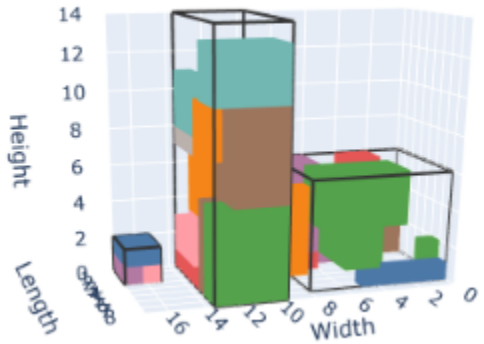
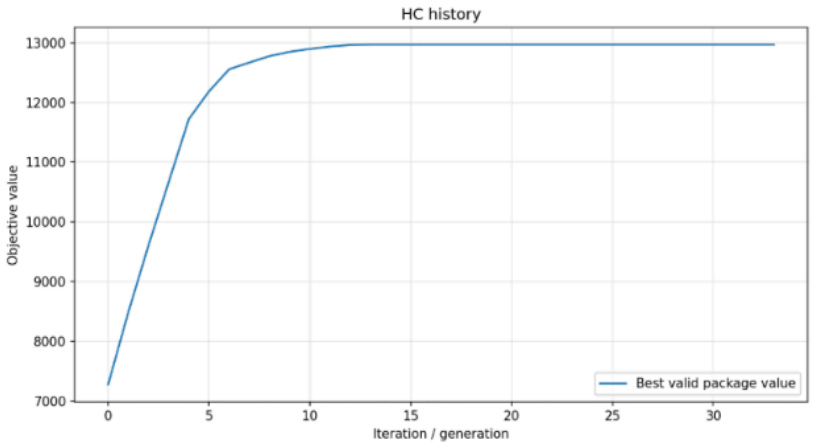
- **Package6** — posisi (1, 4, 0); ukuran (1 × 1 × 3); nilai 110, berat 9
- **Package10** — posisi (6, 0, 0); ukuran (2 × 1 × 4); nilai 160, berat 20

State akhir

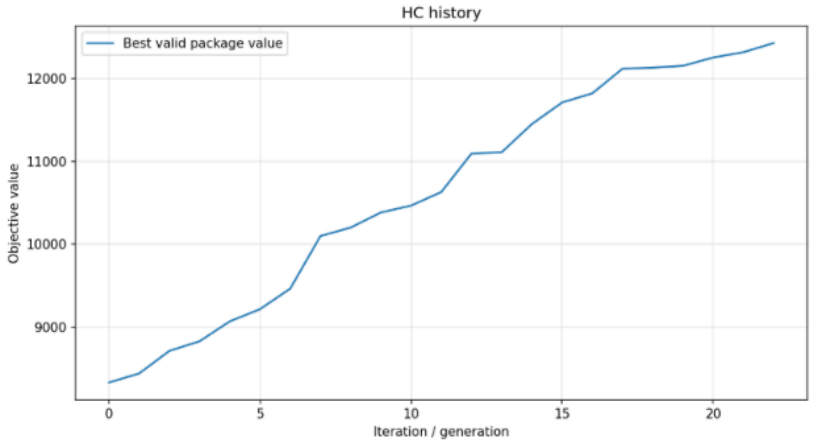
Truk Truck1

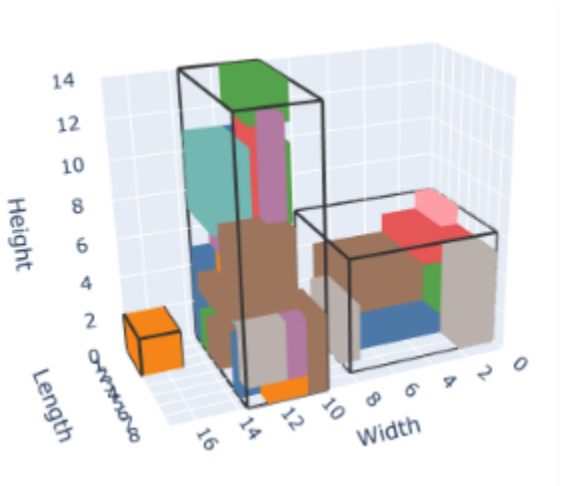
- **Package6** — posisi (4, 1, 2); ukuran (3 × 1 × 1); nilai 110, berat 9
- **Package11** — posisi (5, 0, 0); ukuran (1 × 1 × 5); nilai 182, berat 13

===== Ringkasan hasil =====
Algoritma : Hill Climbing (HC)
Testcase : 3_trucks_50_packages.json
Mode HC : Sideway
Sideway : 20/20
Nilai terbaik yang mungkin : 20000
Nilai awal : 7276
Nilai akhir : 12970

		State akhir : valid Iterasi/generasi: 33/100 Durasi : 191.081 detik Alasan berhenti : sideway limit tercapai  **Ringkasan hasil** Algoritma: hc; Nilai awal: 7276; Nilai akhir: 12970; State akhir valid: Ya; Iterasi/generasi: 33; Durasi: 191.081 detik **State awal** Truk Truck1 • Package15 — posisi (6, 1, 0); ukuran (2 × 5 × 3); nilai 732, berat 66 • Package20 — posisi (0, 0, 2); ukuran (1 × 1 × 2); nilai 76, berat 7 • Package21 — posisi (2, 0, 0); ukuran (2 × 4 × 3); nilai 460, berat **State akhir** Truk Truck1 • Package3 — posisi (4, 1, 3); ukuran (2 × 2 × 4); nilai 464, berat 37 • Package11 — posisi (7, 0, 3); ukuran (1 × 5 × 1); nilai 182, berat 13 
	Hill Climbing Stochastic	===== Ringkasan hasil ===== Algoritma : Hill Climbing (HC) Testcase : 3_trucks_50_packages.json Mode HC : Stochastic Nilai terbaik yang mungkin : 20000 Nilai awal : 8333

Nilai akhir : 12427
State akhir : valid
Iterasi/generasi: 22/100
Durasi : 327.745 detik
Alasan berhenti : sudah Local Optimum





Ringkasan hasil

Algoritma	Nilai awal	Nilai akhir	State akhir valid	Iterasi/generasi	Durasi
hc	8333	12427	Ya	22	327.745 detik

State awal

Truk Truck1

- **Package7** — posisi (0, 5, 0); ukuran (1 × 1 × 3); nilai 88, berat 8
- **Package13** — posisi (0, 4, 0); ukuran (1 × 1 × 5); nilai 200, berat 18
- **Package16** — posisi (2, 0, 0); ukuran (4 × 1 × 2); nilai 312, berat 26
- **Package21** — posisi (0, 0, 5); ukuran (3 × 4 × 2); nilai 460, berat

State akhir

Truk Truck1

- **Package7** — posisi (0, 5, 0); ukuran (1 × 1 × 3); nilai 88, berat 8
- **Package9** — posisi (0, 0, 5); ukuran (2 × 5 × 2); nilai 560, berat 45
- **Package13** — posisi (0, 4, 0); ukuran (1 × 1 × 5); nilai 200, berat 18
- **Package16** — posisi (2, 0, 0); ukuran (4 × 1 × 2); nilai 312, berat

===== Ringkasan hasil =====
Algoritma : Hill Climbing (HC)
Testcase : 3_trucks_50_packages.json
Mode HC : Stochastic
Nilai terbaik yang mungkin : 20000

Nilai awal : 6939
Nilai akhir : 12665
State akhir : valid
Iterasi/generasi: 33/100
Durasi : 327.080 detik
Alasan berhenti : sudah Local Optimum

Ringkasan hasil

Algoritma	Nilai awal	Nilai akhir	State akhir valid	Iterasi/generasi	Durasi
hc	6939	12665	Ya	33	327.080 detik

State awal

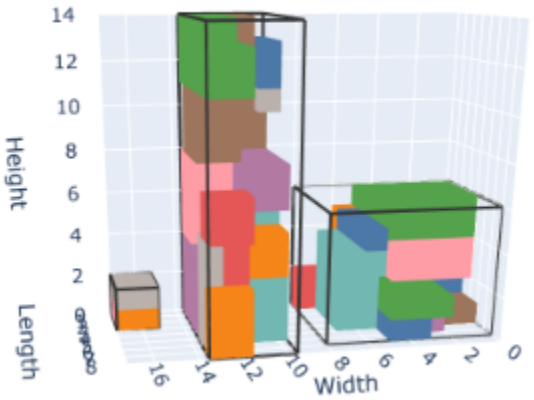
Truk Truck1

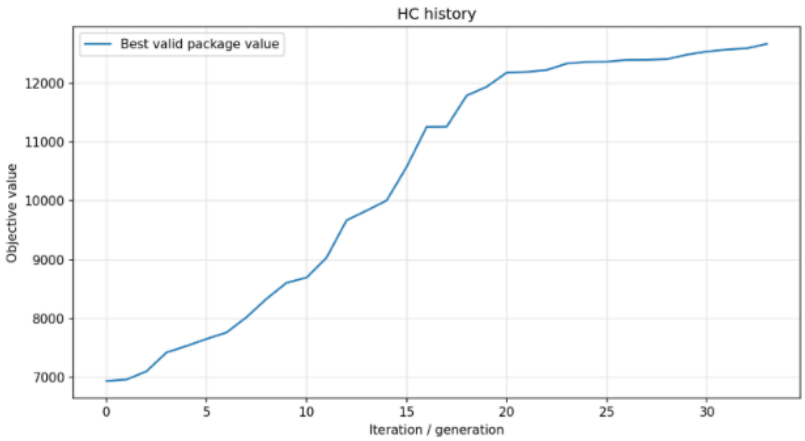
- **Package11** — posisi (5, 0, 0); ukuran (1 × 5 × 1); nilai 182, berat 13
- **Package12** — posisi (0, 2, 0); ukuran (1 × 2 × 1); nilai 83, berat 7
- **Package22** — posisi (0, 0, 0); ukuran (5 × 2 × 1); nilai 310, berat 29
- **Package31** — posisi (0, 0, 3); ukuran (2 × 2 × 1); nilai 116, berat 12

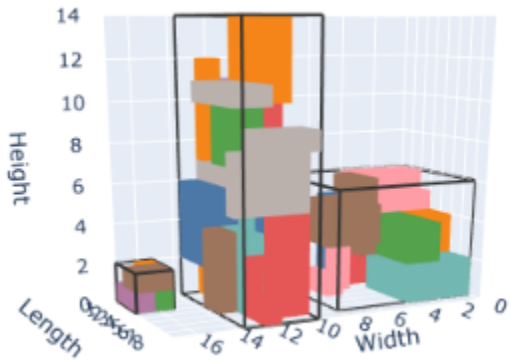
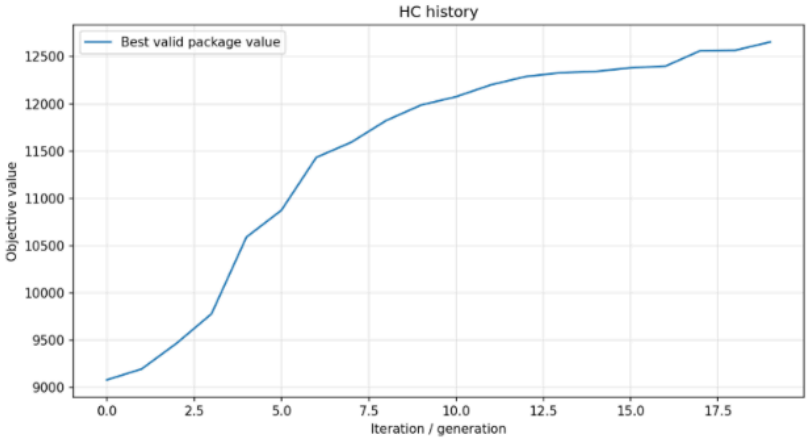
State akhir

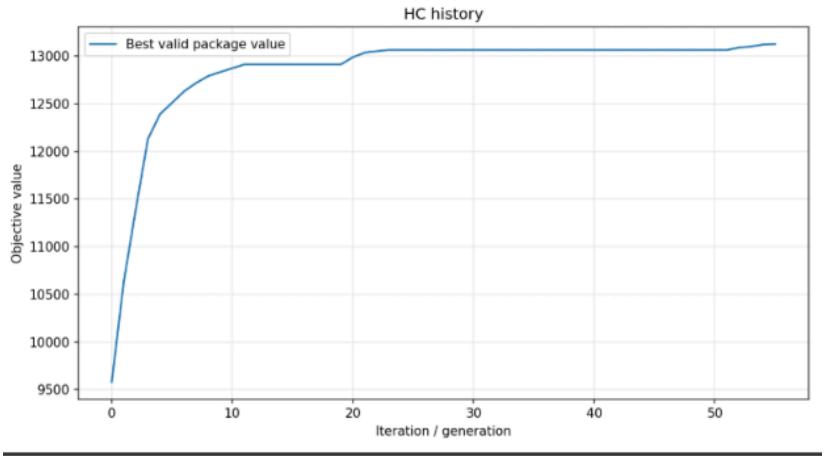
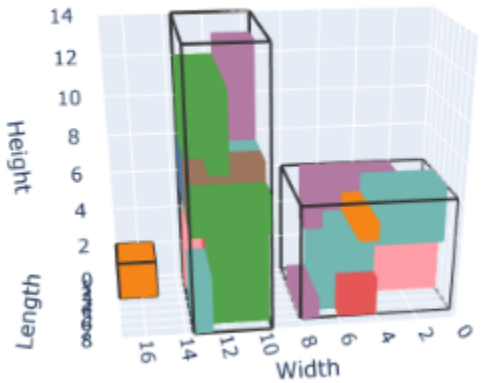
Truk Truck1

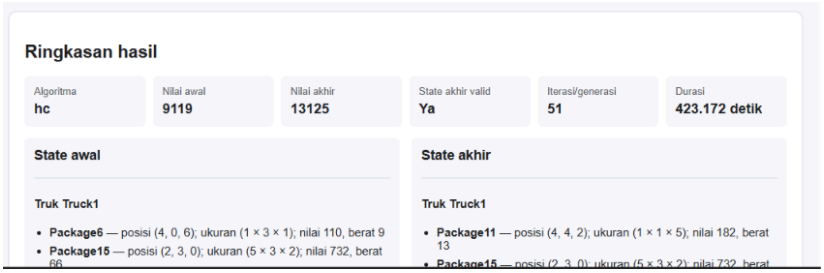
- **Package1** — posisi (3, 0, 3); ukuran (2 × 1 × 4); nilai 120, berat 15
- **Package6** — posisi (2, 0, 3); ukuran (1 × 1 × 3); nilai 110, berat 9
- **Package11** — posisi (5, 0, 0); ukuran (1 × 5 × 1); nilai 182, berat 13
- **Package12** — posisi (0, 2, 0); ukuran (1 × 2 × 1); nilai 83, berat 7



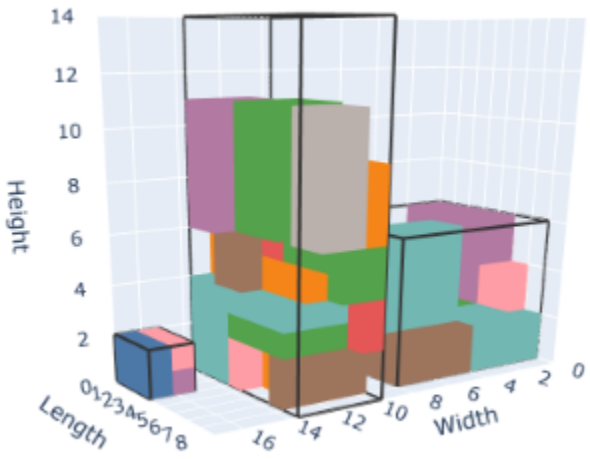


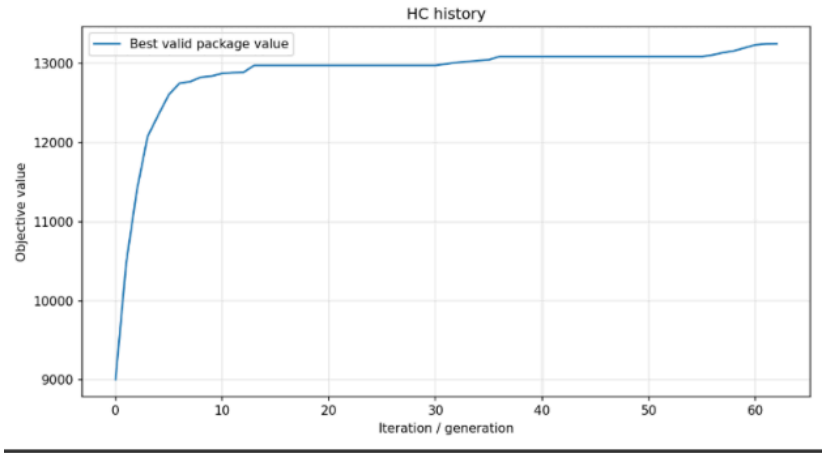
		===== Ringkasan hasil ===== Algoritma : Hill Climbing (HC) Testcase : 3_trucks_50_packages.json Mode HC : Stochastic Nilai terbaik yang mungkin : 20000 Nilai awal : 9077 Nilai akhir : 12655 State akhir : valid Iterasi/generasi: 19/100 Durasi : 222.027 detik Alasan berhenti : sudah Local Optimum  **Ringkasan hasil** Algoritma: **hc** · Nilai awal: **9077** · Nilai akhir: **12655** · State akhir valid: **Ya** · Iterasi/generasi: **19** · Durasi: **222.027 detik** **State awal** **Truk Truck1** • **Package2** — posisi (3, 0, 1); ukuran (2 × 2 × 2); nilai 240, berat 20 • **Package8** — posisi (4, 2, 1); ukuran (1 × 4 × 3); nilai 340, berat 32 • **Package13** — posisi (0, 0, 0); ukuran (1 × 5 × 1); nilai 200, berat **State akhir** **Truk Truck1** • **Package2** — posisi (0, 2, 3); ukuran (2 × 2 × 2); nilai 240, berat 20 • **Package3** — posisi (2, 2, 3); ukuran (2 × 2 × 4); nilai 464, berat 37 • **Package7** — posisi (6, 0, 1); ukuran (1 × 1 × 3); nilai 88, berat 8 

	Hill Climbing Random Restart	===== Ringkasan hasil ===== Algoritma : Hill Climbing (HC) Testcase : 3_trucks_50_packages.json Mode HC : Random Restart Jumlah restart : 5/5 Iterasi/restart : [11, 11, 8, 7, 14] Nilai terbaik yang mungkin : 20000 Nilai awal : 9119 Nilai akhir : 13125 State akhir : valid Iterasi total : 51 (maks. 100 per restart) Durasi : 423.172 detik Alasan berhenti : restart limit tercapai  

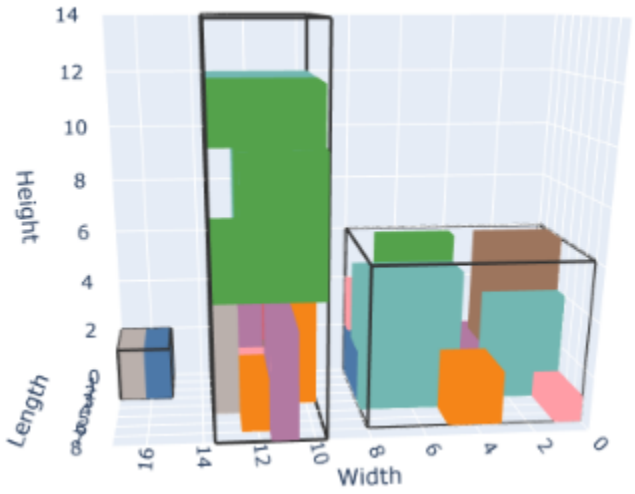


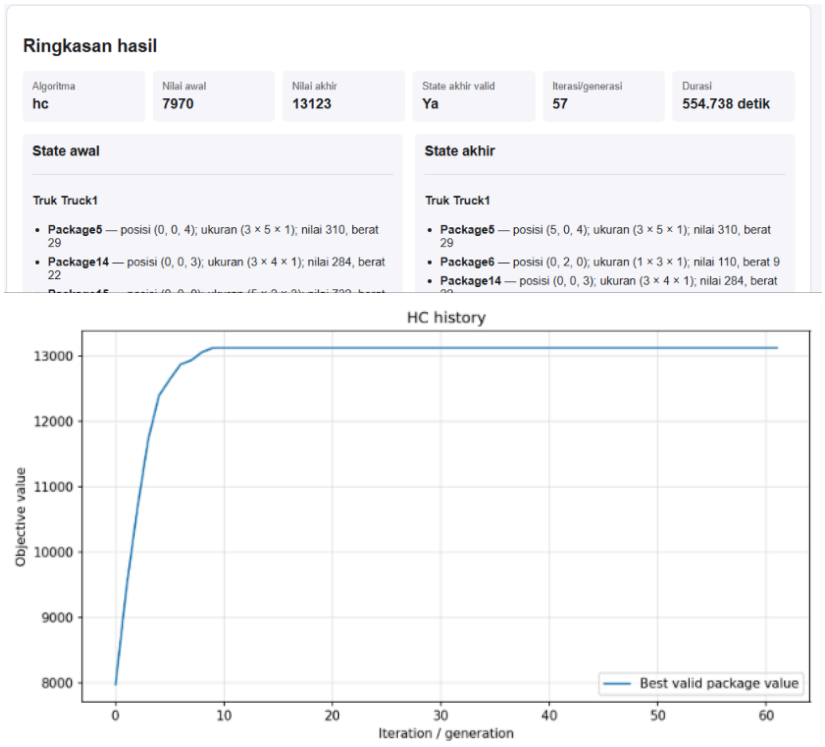
===== Ringkasan hasil =====
Algoritma : Hill Climbing (HC)
Testcase : 3_trucks_50_packages.json
Mode HC : Random Restart
Jumlah restart : 5/5
Iterasi/restart : [13, 9, 12, 10, 14]
Nilai terbaik yang mungkin : 20000
Nilai awal : 11010
Nilai akhir : 13253
State akhir : valid
Iterasi total : 58 (maks. 100 per restart)
Durasi : 518.304 detik





===== Ringkasan hasil =====
Algoritma : Hill Climbing (HC)
Testcase : 3_trucks_50_packages.json
Mode HC : Random Restart
Jumlah restart : 5/5
Iterasi/restart : [10, 13, 12, 11, 11]
Nilai terbaik yang mungkin : 20000
Nilai awal : 7970
Nilai akhir : 13123
State akhir : valid
Iterasi total : 57 (maks. 100 per restart)
Durasi : 554.738 detik
Alasan berhenti : restart limit tercapai



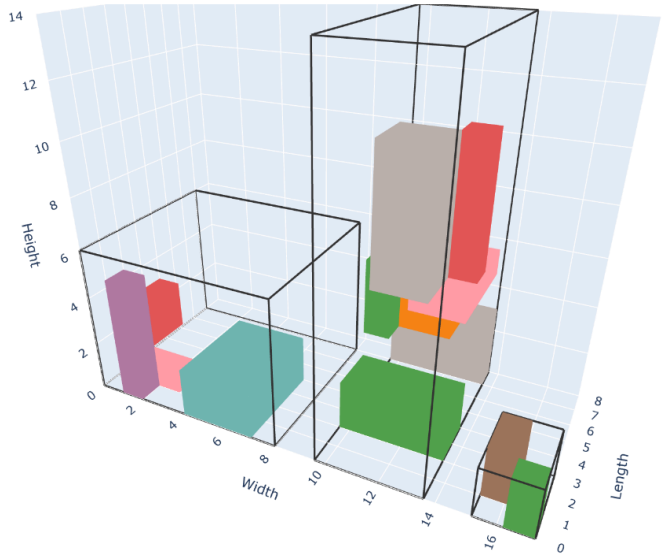
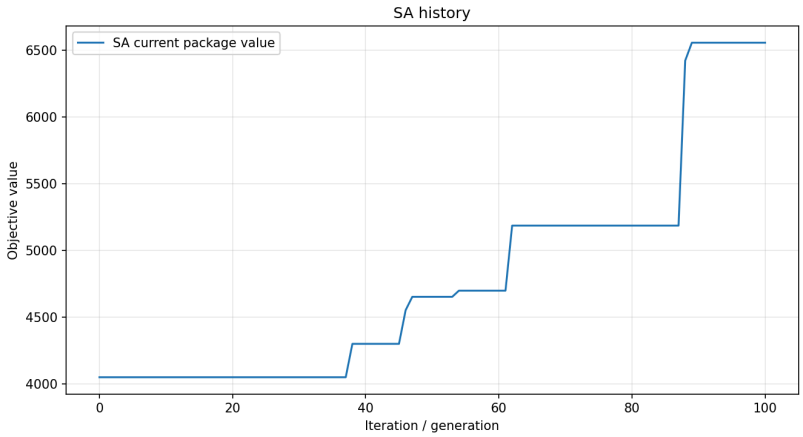
Analisis Hasil

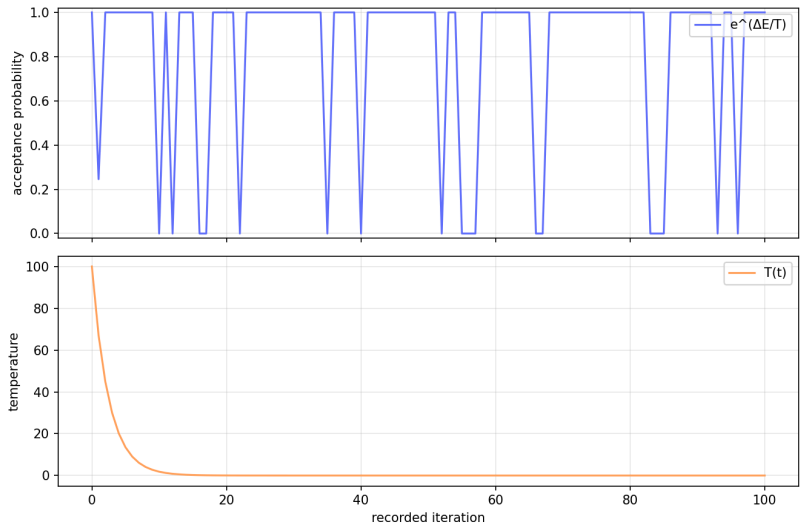
Mode	Rata-rata nilai akhir	Rata-rata iterasi (dibulatkan)	Rata-rata durasi (detik)
Steepest	12.924	11	90,9
Stochastic	12.582	25	292,3
Sideway	13.041	36	211,9
Random Restart	13.167	55(total)	489,738

Pada eksperimen ini, Steepest paling cepat, tetapi rata-rata nilai akhirnya paling rendah. Sideway dan Random Restart memperoleh nilai rata-rata lebih tinggi. Dengan Random Restart paling tinggi, tetapi juga paling lama. Stochastic memerlukan waktu lebih lama daripada Steepest dan nilai akhirnya lebih rendah di percobaan-percobaan ini.

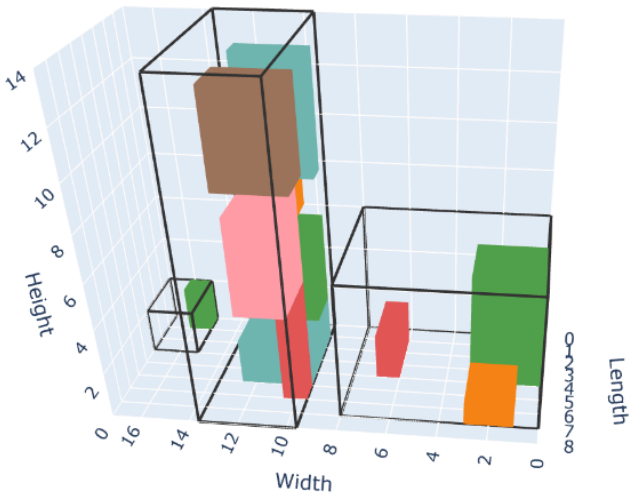
2. Simulated Annealing

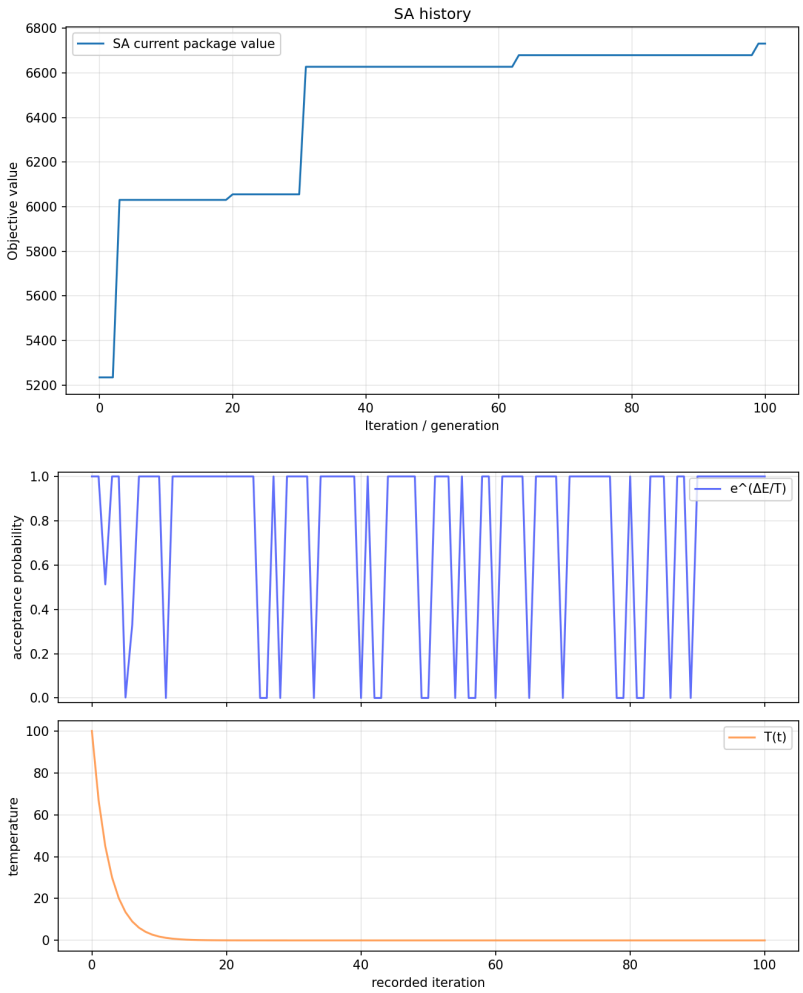
Hasil Eksperimen

Test Case	Algoritma	Hasil
3_truck_50_packages	Simulated Annealing	===== Ringkasan hasil ===== Algoritma : Simulated Annealing (SA) Testcase : 3_trucks_50_packages.json Nilai terbaik yang mungkin : 20000 Nilai awal : 4051 Nilai akhir : 4051 State akhir : valid Iterasi/generasi: 100/100 Durasi : 0.103 detik  

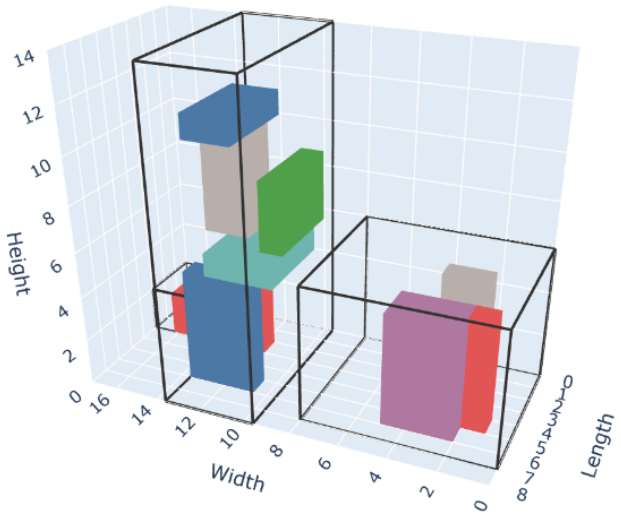


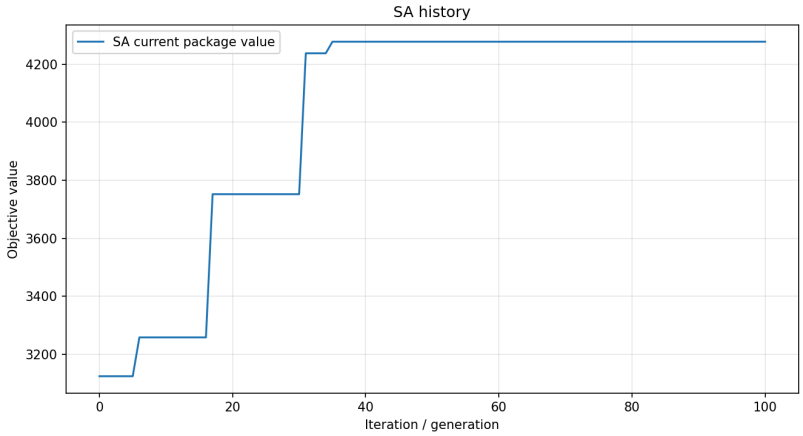
====== Ringkasan hasil ======
Algoritma : Simulated Annealing (SA)
Testcase : 3_trucks_50_packages.json
Nilai terbaik yang mungkin : 20000
Nilai awal : 5234
Nilai akhir : 5234
State akhir : valid
Iterasi/generasi: 100/100
Durasi : 0.033 detik

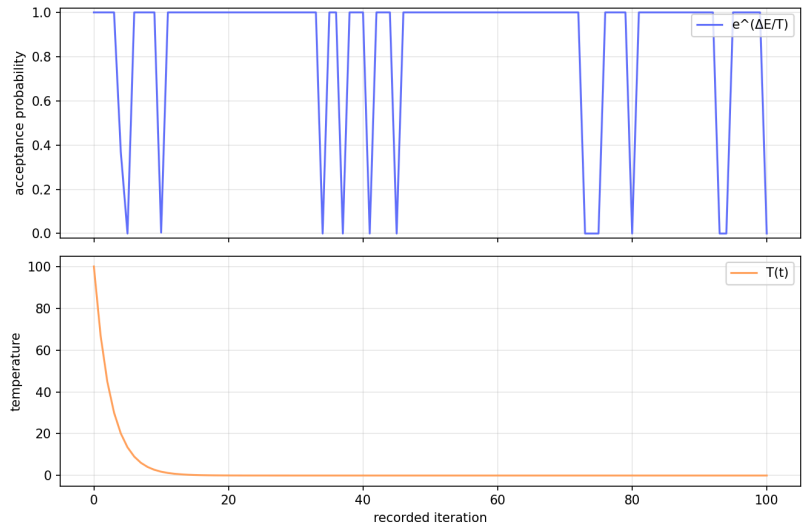




===== Ringkasan hasil =====
Algoritma : Simulated Annealing (SA)
Testcase : 3_trucks_50_packages.json
Nilai terbaik yang mungkin : 20000
Nilai awal : 3124
Nilai akhir : 3258
State akhir : valid
Iterasi/generasi: 100/100
Durasi : 0.026 detik


Height
Width
Length


SA history
SA current package value
Objective value
Iteration / generation

Analisis Hasil

Run 3 merupakan satu-satunya eksekusi yang menunjukkan peningkatan. Meskipun demikian, nilai akhirnya masih lebih rendah dibandingkan dengan nilai pada Run 1 dan Run 2. Nilai yang tidak berubah pada dua eksekusi pertama tidak serta-merta berarti bahwa tidak ada solusi kandidat yang dieksplorasi. Dalam algoritma Simulated Annealing, solusi kandidat dapat diterima atau ditolak selama proses eksekusi berlangsung, dan nilai objektif akhir bisa saja tetap sama meskipun intermediate states di sepanjang proses tersebut berbeda

Selain itu, semua eksekusi menggunakan batas yang sama yaitu 100 iterasi. Batas pencarian yang relatif kecil ini dapat membatasi kemampuan algoritma dalam menjelajahi ruang solusi dan menemukan solusi yang lebih baik.

3. Genetic Algorithm

Hasil Eksperimen

Test Case	Algoritma	Parameter	Hasil
3_truck_50_packages	Genetic Algorithm v1	Populasi=100 Iterasi=100	===== Ringkasan hasil ===== Algoritma : Genetic Algorithm v1 (GA) Testcase : 3_trucks_50_packages.json Ukuran populasi : 100 Nilai terbaik yang mungkin : 20000 Nilai awal : 8446 Nilai akhir : 9224

State akhir : valid
Iterasi/generasi: 100/100
Durasi : 63.875 detik



Ringkasan hasil

Algoritma	Nilai awal	Nilai akhir	State akhir valid	Iterasi/generasi	Durasi
ga	8446	9224	Ya	100	63.875 detik

State awal

Truk Truck1

- **Package7** — posisi (3, 0, 0); ukuran (1 × 3 × 1); nilai 88, berat 8
- **Package10** — posisi (0, 2, 0); ukuran (2 × 1 × 4); nilai 160, berat 20
- **Package13** — posisi (3, 0, 5); ukuran (1 × 5 × 1); nilai 200, berat 18
- **Package22** — posisi (6, 0, 0); ukuran (1 × 5 × 2); nilai 310, berat 29
- **Package29** — posisi (3, 4, 1); ukuran (2 × 2 × 3); nilai 370, berat 35
- **Package34** — posisi (2, 0, 3); ukuran (2 × 4 × 3); nilai 570, berat 55
- **Package40** — posisi (0, 0, 3); ukuran (2 × 2 × 2); nilai 272, berat 21

Truk Truck2

- **Package1** — posisi (0, 0, 3); ukuran (4 × 1 × 2); nilai 120, berat 15
- **Package6** — posisi (0, 0, 5); ukuran (1 × 3 × 1); nilai 110, berat 9
- **Package8** — posisi (1, 9, 4); ukuran (3 × 1 × 4); nilai 340, berat 32
- **Package12** — posisi (2, 0, 5); ukuran (1 × 2 × 1); nilai 83, berat 7
- **Package21** — posisi (1, 5, 1); ukuran (3 × 4 × 2); nilai 450, berat 44
- **Package24** — posisi (0, 5, 4); ukuran (4 × 4 × 1); nilai 530, berat 51
- **Package26** — posisi (1, 10, 6); ukuran (3 × 4 × 2); nilai 634, berat 51
- **Package30** — posisi (0, 9, 0); ukuran (3 × 5 × 3); nilai 1475, berat 116
- **Package39** — posisi (0, 0, 0); ukuran (4 × 5 × 2); nilai 1210, berat 99
- **Package41** — posisi (0, 3, 4); ukuran (4 × 2 × 4); nilai 1060, berat 80
- **Package42** — posisi (0, 9, 4); ukuran (1 × 5 × 1); nilai 150, berat 13
- **Package49** — posisi (0, 5, 1); ukuran (1 × 2 × 3); nilai 140, berat 12

Truk Truck3

- **Package44** — posisi (0, 0, 1); ukuran (1 × 2 × 2); nilai 164, berat 12

State akhir

Truk Truck1

- **Package9** — posisi (0, 0, 5); ukuran (2 × 5 × 2); nilai 560, berat 45
- **Package16** — posisi (0, 5, 0); ukuran (2 × 1 × 4); nilai 312, berat 26
- **Package17** — posisi (4, 0, 3); ukuran (1 × 2 × 4); nilai 272, berat 21
- **Package25** — posisi (2, 0, 5); ukuran (2 × 1 × 2); nilai 110, berat 9
- **Package30** — posisi (5, 0, 4); ukuran (3 × 5 × 3); nilai 1475, berat 116
- **Package37** — posisi (4, 0, 2); ukuran (1 × 5 × 1); nilai 172, berat 17
- **Package39** — posisi (1, 0, 0); ukuran (4 × 5 × 2); nilai 1210, berat 99
- **Package46** — posisi (5, 0, 0); ukuran (2 × 3 × 1); nilai 248, berat 19

Truk Truck2

- **Package3** — posisi (0, 11, 6); ukuran (4 × 2 × 2); nilai 464, berat 37
- **Package10** — posisi (1, 10, 4); ukuran (2 × 1 × 4); nilai 160, berat 20
- **Package11** — posisi (3, 0, 1); ukuran (1 × 5 × 1); nilai 182, berat 13
- **Package12** — posisi (2, 0, 2); ukuran (1 × 2 × 1); nilai 83, berat 7
- **Package15** — posisi (0, 3, 0); ukuran (3 × 5 × 2); nilai 732, berat 66
- **Package29** — posisi (0, 11, 2); ukuran (2 × 2 × 3); nilai 370, berat 35
- **Package32** — posisi (1, 0, 4); ukuran (3 × 4 × 3); nilai 1056, berat 86
- **Package38** — posisi (0, 4, 3); ukuran (3 × 5 × 3); nilai 1120, berat 110
- **Package43** — posisi (0, 9, 5); ukuran (3 × 1 × 1); nilai 126, berat 9
- **Package44** — posisi (0, 0, 0); ukuran (1 × 2 × 2); nilai 164, berat 12
- **Package48** — posisi (1, 0, 0); ukuran (1 × 3 × 4); nilai 332, berat 26

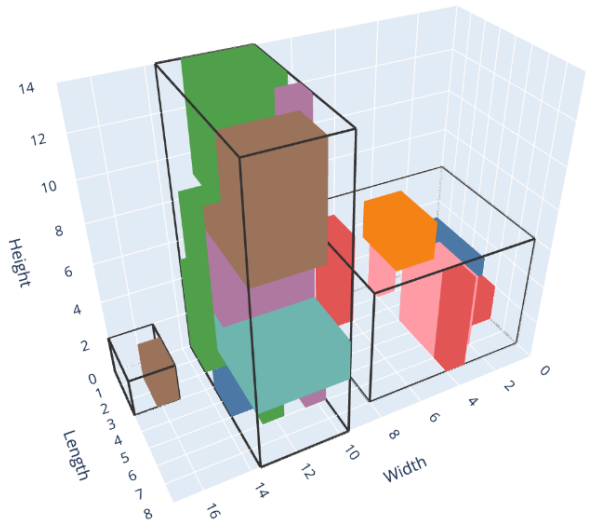
Truk Truck3

- **Package20** — posisi (1, 0, 0); ukuran (1 × 1 × 2); nilai 76, berat 7

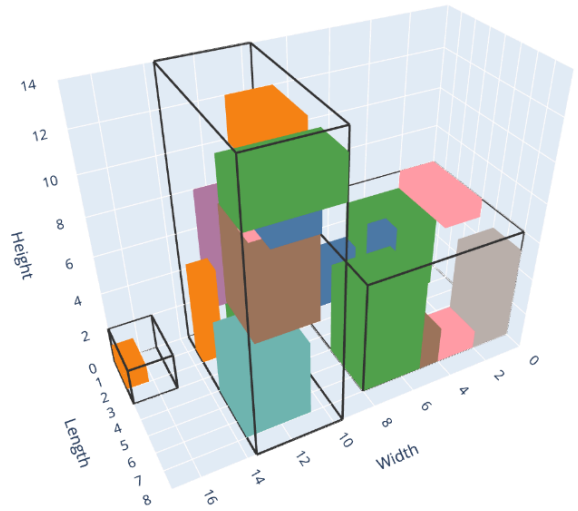
Grafik:



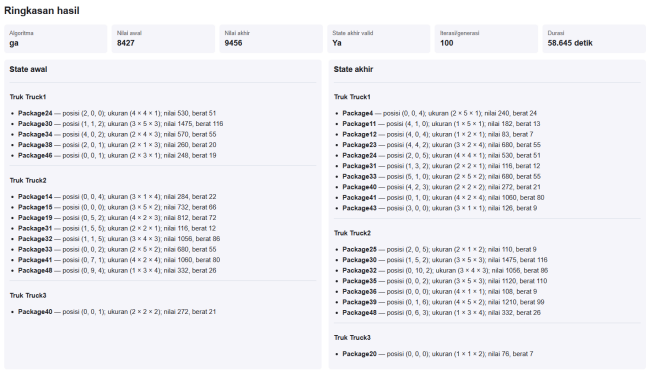
Initial State



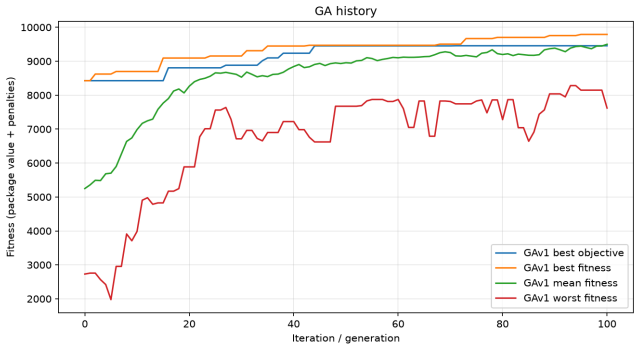
Final State



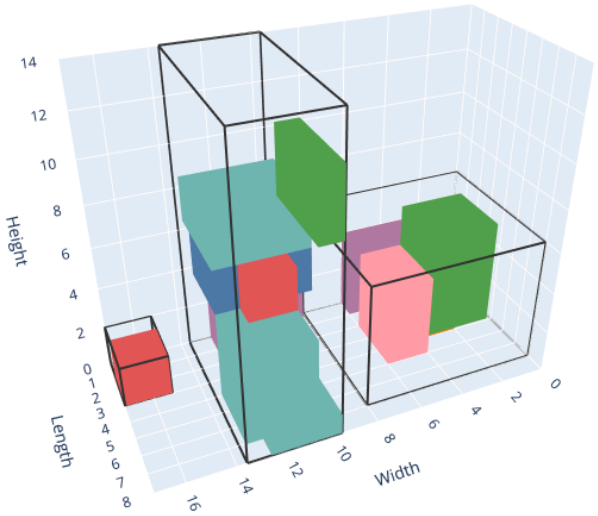
===== Ringkasan hasil =====
Algoritma : Genetic Algorithm v1 (GA)
Testcase : 3_trucks_50_packages.json
Ukuran populasi : 100
Nilai terbaik yang mungkin : 20000
Nilai awal : 8427
Nilai akhir : 9456
State akhir : valid
Iterasi/generasi: 100/100
Durasi : 58.645 detik



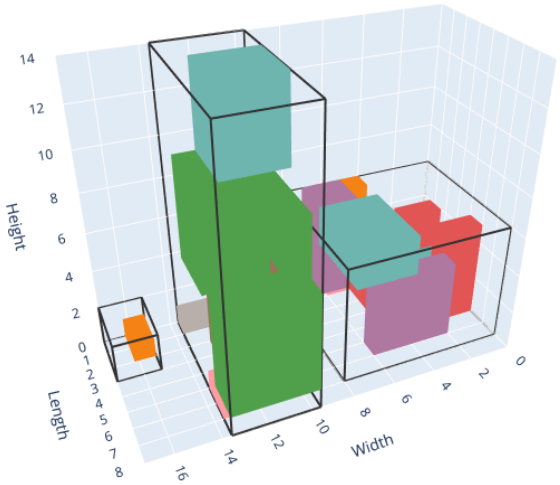
Grafik



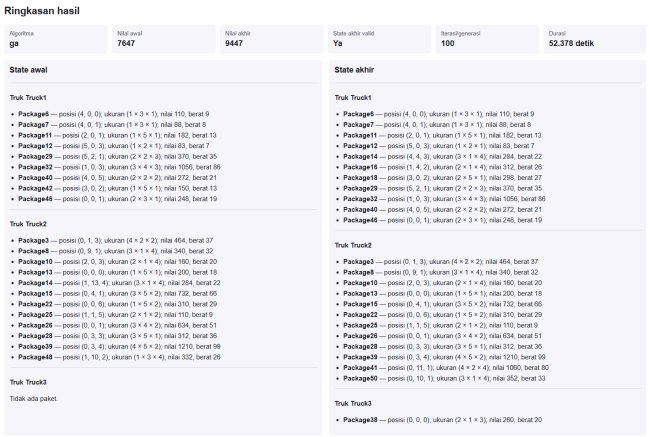
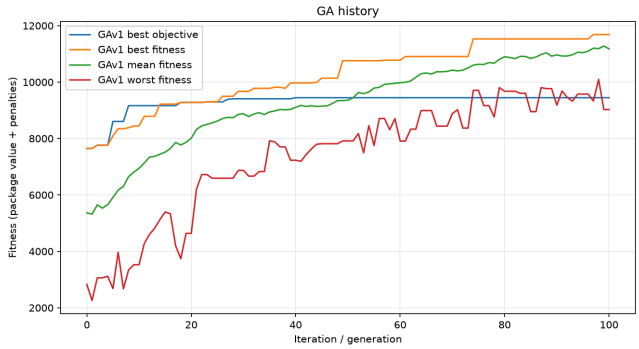
Initial State

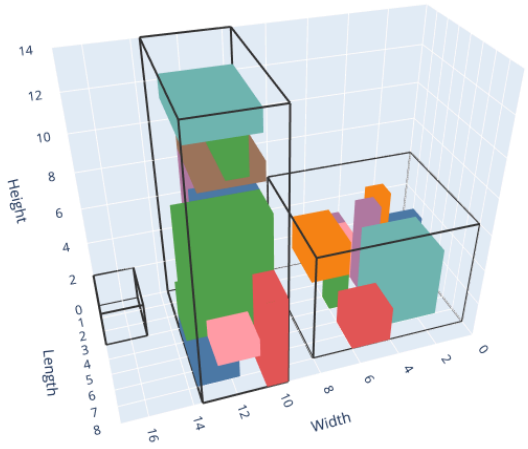
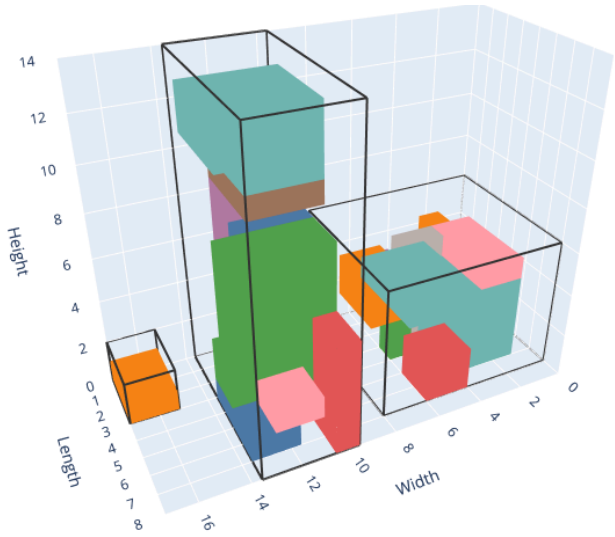


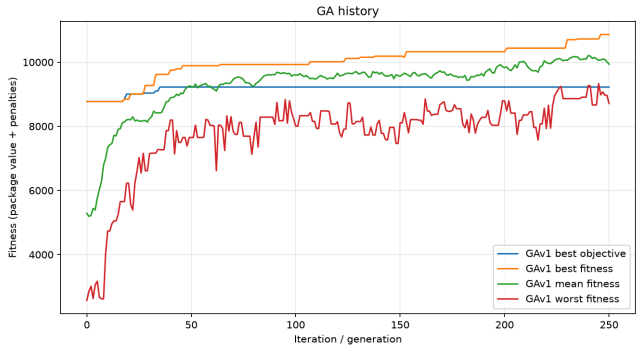
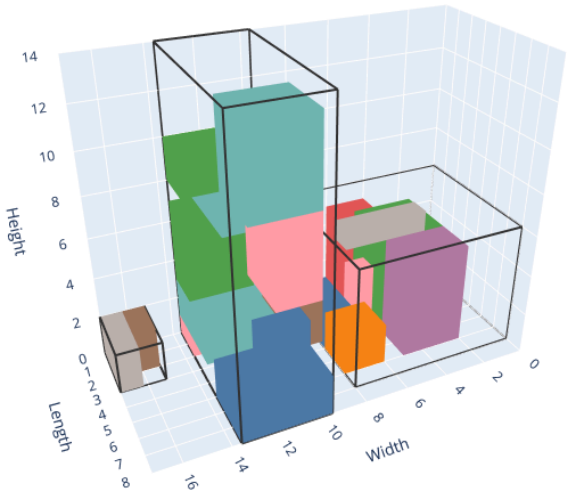
Final State

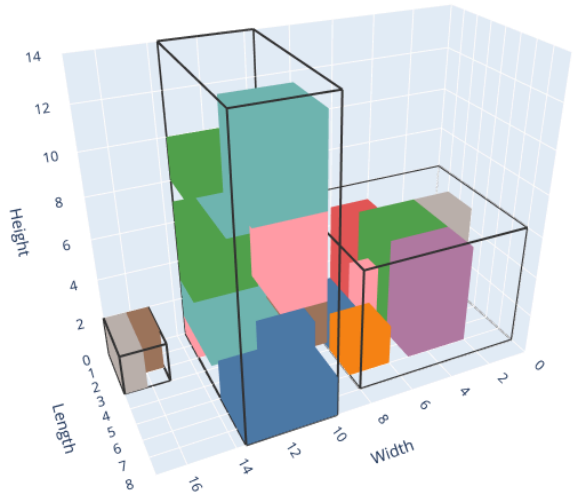


===== Ringkasan hasil =====
Algoritma : Genetic Algorithm v1 (GA)

			Testcase : 3_trucks_50_packages.json Ukuran populasi : 100 Nilai terbaik yang mungkin : 20000 Nilai awal : 7647 Nilai akhir : 9447 State akhir : valid Iterasi/generasi: 100/100 Durasi : 52.378 detik  Grafik  Initial state

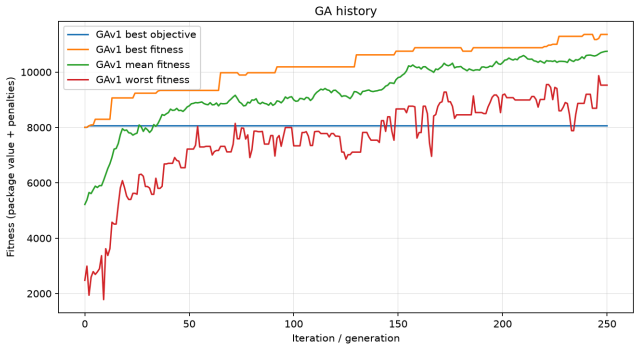
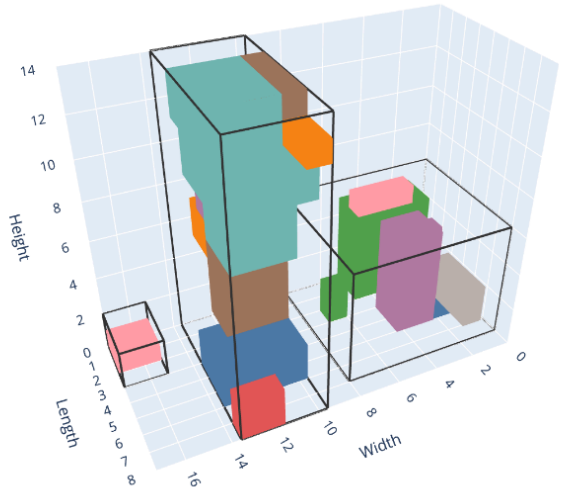
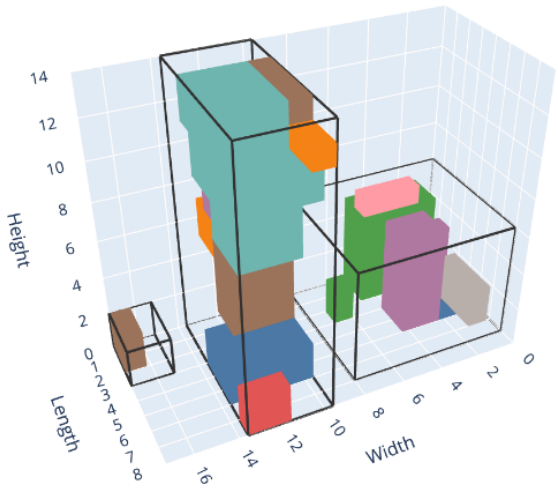
			 Final state 
	Genetic Algorithm v1	Populasi=100 Iterasi=250	===== Ringkasan hasil ===== Algoritma : Genetic Algorithm v1 (GA) Testcase : 3_trucks_50_packages.json Ukuran populasi : 100 Nilai terbaik yang mungkin : 20000 Nilai awal : 8773 Nilai akhir : 9225 State akhir : valid Iterasi/generasi: 250/250 Durasi : 132.024 detik

			Ringkasan hasil Algoritma: ga Nilai awal: 8773 Nilai akhir: 9225 State akhir valid: Ya Iterasi/generasi: 250 Durasi: 132.024 detik **State awal** Truk Truck1 • Package2 — posisi (6, 0, 4); ukuran (2 × 2 × 2); nilai 240, berat 20 • Package4 — posisi (4, 0, 0); ukuran (2 × 5 × 1); nilai 240, berat 24 • Package7 — posisi (5, 0, 2); ukuran (1 × 3 × 1); nilai 88, berat 8 • Package10 — posisi (6, 0, 0); ukuran (2 × 1 × 4); nilai 160, berat 20 • Package15 — posisi (2, 0, 4); ukuran (3 × 5 × 2); nilai 732, berat 66 • Package30 — posisi (2, 0, 1); ukuran (3 × 5 × 3); nilai 1475, berat 116 • Package36 — posisi (3, 5, 3); ukuran (4 × 1 × 1); nilai 108, berat 9 Truk Truck2 • Package3 — posisi (0, 8, 0); ukuran (4 × 2 × 2); nilai 464, berat 37 • Package14 — posisi (0, 7, 0); ukuran (3 × 1 × 4); nilai 284, berat 22 • Package17 — posisi (0, 3, 3); ukuran (1 × 2 × 4); nilai 272, berat 21 • Package19 — posisi (0, 0, 5); ukuran (4 × 2 × 3); nilai 812, berat 72 • Package21 — posisi (1, 3, 0); ukuran (3 × 4 × 2); nilai 460, berat 44 • Package25 — posisi (2, 0, 0); ukuran (2 × 1 × 2); nilai 110, berat 9 • Package32 — posisi (0, 9, 4); ukuran (3 × 4 × 3); nilai 1056, berat 86 • Package34 — posisi (0, 5, 4); ukuran (2 × 4 × 3); nilai 570, berat 55 • Package41 — posisi (0, 1, 0); ukuran (4 × 2 × 4); nilai 1060, berat 80 • Package46 — posisi (1, 2, 7); ukuran (2 × 3 × 1); nilai 248, berat 19 Truk Truck3 • Package27 — posisi (1, 0, 0); ukuran (1 × 2 × 3); nilai 230, berat 21 • Package44 — posisi (0, 0, 0); ukuran (1 × 2 × 2); nilai 164, berat 12 **State akhir** Truk Truck1 • Package2 — posisi (6, 0, 4); ukuran (2 × 2 × 2); nilai 240, berat 20 • Package4 — posisi (4, 0, 0); ukuran (2 × 5 × 1); nilai 240, berat 24 • Package7 — posisi (5, 0, 2); ukuran (1 × 3 × 1); nilai 88, berat 8 • Package9 — posisi (0, 0, 1); ukuran (2 × 5 × 2); nilai 560, berat 45 • Package10 — posisi (6, 0, 0); ukuran (2 × 1 × 4); nilai 160, berat 20 • Package15 — posisi (2, 0, 4); ukuran (3 × 5 × 2); nilai 732, berat 66 • Package30 — posisi (2, 0, 1); ukuran (3 × 5 × 3); nilai 1475, berat 116 Truk Truck2 • Package3 — posisi (0, 8, 0); ukuran (4 × 2 × 2); nilai 464, berat 37 • Package14 — posisi (0, 7, 0); ukuran (3 × 1 × 4); nilai 284, berat 22 • Package17 — posisi (0, 3, 3); ukuran (1 × 2 × 4); nilai 272, berat 21 • Package19 — posisi (0, 0, 5); ukuran (4 × 2 × 3); nilai 812, berat 72 • Package21 — posisi (1, 3, 0); ukuran (3 × 4 × 2); nilai 460, berat 44 • Package25 — posisi (2, 0, 0); ukuran (2 × 1 × 2); nilai 110, berat 9 • Package32 — posisi (0, 9, 4); ukuran (3 × 4 × 3); nilai 1056, berat 86 • Package34 — posisi (0, 5, 4); ukuran (2 × 4 × 3); nilai 570, berat 55 • Package41 — posisi (0, 1, 0); ukuran (4 × 2 × 4); nilai 1060, berat 80 • Package46 — posisi (1, 2, 7); ukuran (2 × 3 × 1); nilai 248, berat 19 Truk Truck3 • Package27 — posisi (1, 0, 0); ukuran (1 × 2 × 3); nilai 230, berat 21 • Package44 — posisi (0, 0, 0); ukuran (1 × 2 × 2); nilai 164, berat 12 Grafik  Initial State  Final State

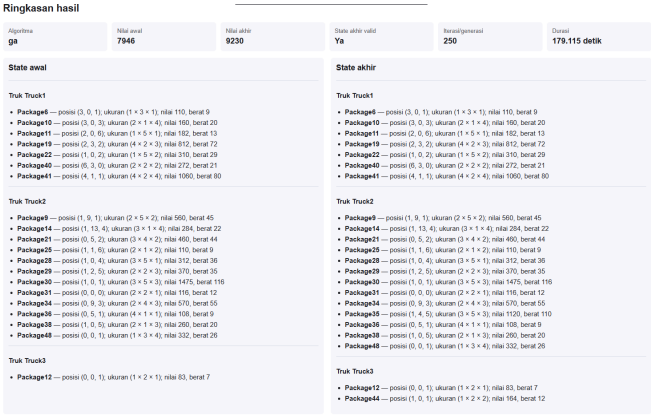


===== Ringkasan hasil =====
Algoritma : Genetic Algorithm v1 (GA)
Testcase : 3_trucks_50_packages.json
Ukuran populasi : 100
Nilai terbaik yang mungkin : 20000
Nilai awal : 8005
Nilai akhir : 8059
State akhir : valid
Iterasi/generasi: 250/250
Durasi : 168.313 detik



			  
			===== Ringkasan hasil ===== Algoritma : Genetic Algorithm v1 (GA)

Testcase : 3_trucks_50_packages.json
Ukuran populasi : 100
Nilai terbaik yang mungkin : 20000
Nilai awal : 7946
Nilai akhir : 9230
State akhir : valid
Iterasi/generasi: 250/250
Durasi : 179.115 detik



Ringkasan hasil

Algoritma	Nilai awal	Nilai akhir	State akhir valid	Iterasi/generasi	Durasi
ga	7946	9230	Ya	250	179.115 detik

State awal

Truk Truck1
- **Package6** — posisi (3, 0, 1); ukuran (1 × 3 × 1); nilai 110, berat 9
- **Package10** — posisi (3, 0, 3); ukuran (2 × 1 × 4); nilai 160, berat 20
- **Package11** — posisi (2, 0, 6); ukuran (1 × 5 × 1); nilai 182, berat 13
- **Package19** — posisi (2, 3, 2); ukuran (4 × 2 × 3); nilai 812, berat 72
- **Package22** — posisi (1, 0, 2); ukuran (1 × 5 × 2); nilai 310, berat 29
- **Package40** — posisi (6, 3, 0); ukuran (2 × 2 × 2); nilai 272, berat 21
- **Package41** — posisi (4, 1, 1); ukuran (4 × 2 × 4); nilai 1060, berat 80

Truk Truck2
- **Package9** — posisi (1, 9, 1); ukuran (2 × 5 × 2); nilai 560, berat 45
- **Package14** — posisi (1, 13, 4); ukuran (3 × 1 × 4); nilai 284, berat 22
- **Package21** — posisi (0, 5, 2); ukuran (3 × 4 × 2); nilai 460, berat 44
- **Package25** — posisi (1, 1, 6); ukuran (2 × 1 × 2); nilai 110, berat 9
- **Package28** — posisi (1, 0, 4); ukuran (3 × 5 × 1); nilai 312, berat 36
- **Package29** — posisi (1, 2, 5); ukuran (2 × 2 × 3); nilai 370, berat 35
- **Package30** — posisi (1, 0, 1); ukuran (3 × 5 × 3); nilai 1475, berat 116
- **Package31** — posisi (0, 0, 0); ukuran (2 × 2 × 1); nilai 116, berat 12
- **Package34** — posisi (0, 9, 3); ukuran (2 × 4 × 3); nilai 570, berat 55
- **Package36** — posisi (0, 5, 1); ukuran (4 × 1 × 1); nilai 108, berat 9
- **Package38** — posisi (1, 0, 5); ukuran (2 × 1 × 3); nilai 260, berat 20
- **Package48** — posisi (0, 0, 1); ukuran (1 × 3 × 4); nilai 332, berat 26

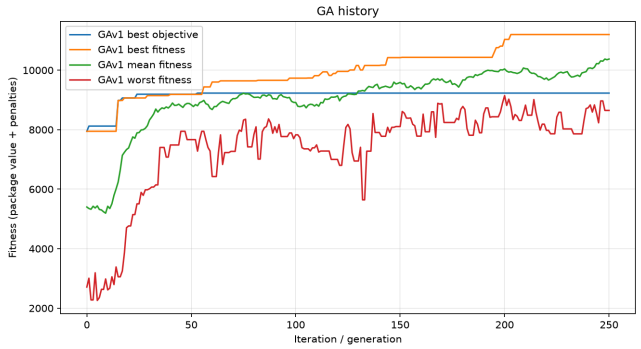
Truk Truck3
- **Package12** — posisi (0, 0, 1); ukuran (1 × 2 × 1); nilai 83, berat 7

State akhir

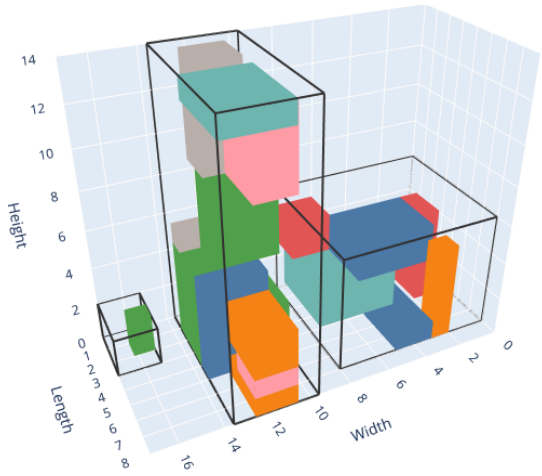
Truk Truck1
- **Package6** — posisi (3, 0, 1); ukuran (1 × 3 × 1); nilai 110, berat 9
- **Package10** — posisi (3, 0, 3); ukuran (2 × 1 × 4); nilai 160, berat 20
- **Package11** — posisi (2, 0, 6); ukuran (1 × 5 × 1); nilai 182, berat 13
- **Package19** — posisi (2, 3, 2); ukuran (4 × 2 × 3); nilai 812, berat 72
- **Package22** — posisi (1, 0, 2); ukuran (1 × 5 × 2); nilai 310, berat 29
- **Package40** — posisi (6, 3, 0); ukuran (2 × 2 × 2); nilai 272, berat 21
- **Package41** — posisi (4, 1, 1); ukuran (4 × 2 × 4); nilai 1060, berat 80

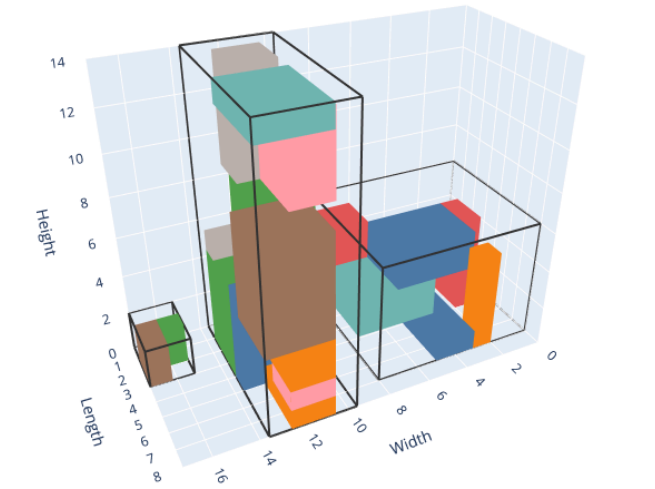
Truk Truck2
- **Package9** — posisi (1, 9, 1); ukuran (2 × 5 × 2); nilai 560, berat 45
- **Package14** — posisi (1, 13, 4); ukuran (3 × 1 × 4); nilai 284, berat 22
- **Package21** — posisi (0, 5, 2); ukuran (3 × 4 × 2); nilai 460, berat 44
- **Package25** — posisi (1, 1, 6); ukuran (2 × 1 × 2); nilai 110, berat 9
- **Package28** — posisi (1, 0, 4); ukuran (3 × 5 × 1); nilai 312, berat 36
- **Package29** — posisi (1, 2, 5); ukuran (2 × 2 × 3); nilai 370, berat 35
- **Package30** — posisi (1, 0, 1); ukuran (3 × 5 × 3); nilai 1475, berat 116
- **Package31** — posisi (0, 0, 0); ukuran (2 × 2 × 1); nilai 116, berat 12
- **Package34** — posisi (0, 9, 3); ukuran (2 × 4 × 3); nilai 570, berat 55
- **Package35** — posisi (1, 4, 5); ukuran (3 × 5 × 3); nilai 1120, berat 110
- **Package36** — posisi (0, 5, 1); ukuran (4 × 1 × 1); nilai 108, berat 9
- **Package38** — posisi (1, 0, 5); ukuran (2 × 1 × 3); nilai 260, berat 20
- **Package48** — posisi (0, 0, 1); ukuran (1 × 3 × 4); nilai 332, berat 26

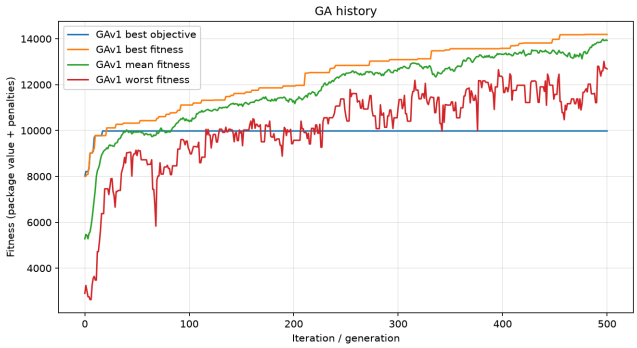
Truk Truck3
- **Package12** — posisi (0, 0, 1); ukuran (1 × 2 × 1); nilai 83, berat 7
- **Package44** — posisi (1, 0, 1); ukuran (1 × 2 × 2); nilai 164, berat 12



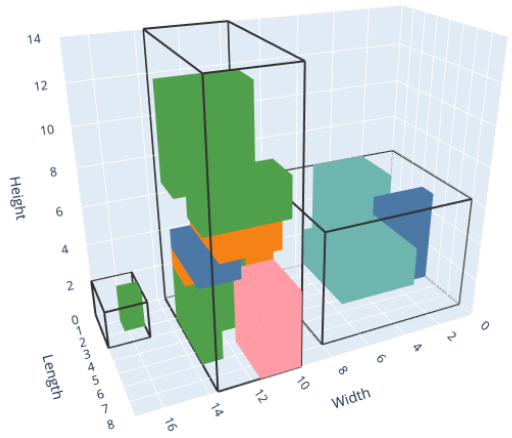
Initial State



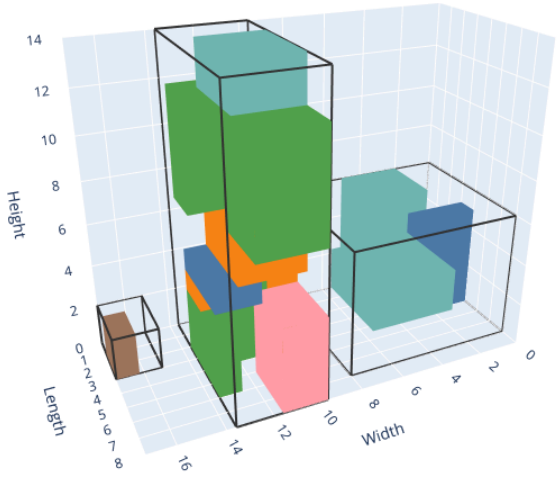
			Final State 
	Genetic Algorithm v1	Populasi=100 Iterasi=500	===== Ringkasan hasil ===== Algoritma : Genetic Algorithm v1 (GA) Testcase : 3_trucks_50_packages.json Ukuran populasi : 100 Nilai terbaik yang mungkin : 20000 Nilai awal : 8007 Nilai akhir : 9981 State akhir : valid Iterasi/generasi: 500/500 Durasi : 236.017 detik  Grafik



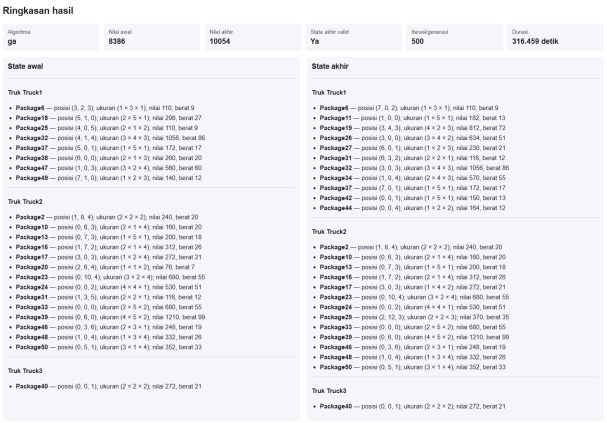
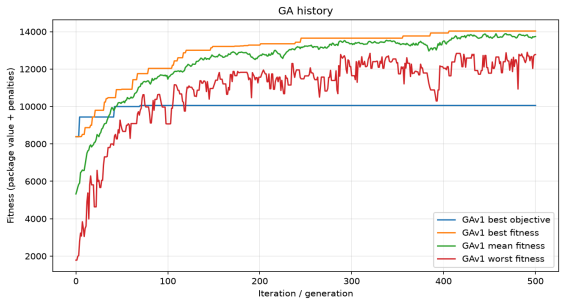
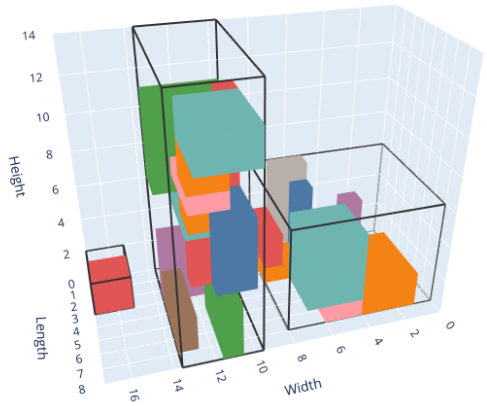
Initial State

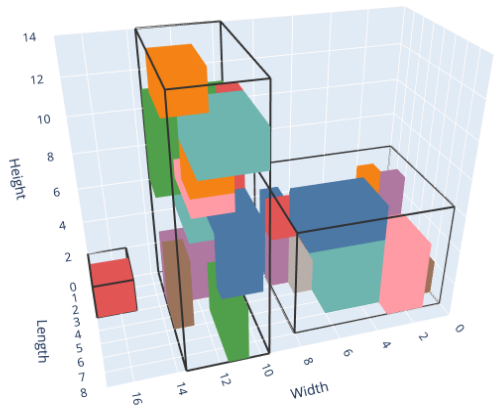


Final State



===== Ringkasan hasil =====
Algoritma : Genetic Algorithm v1 (GA)

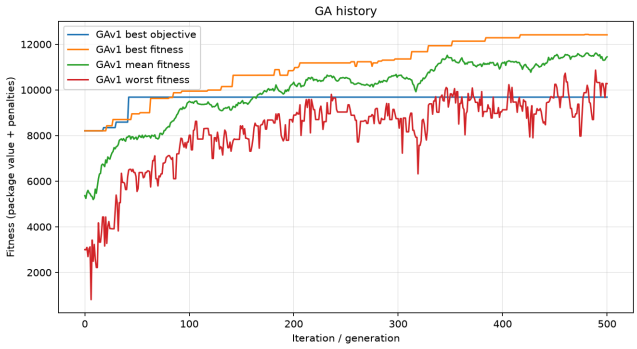
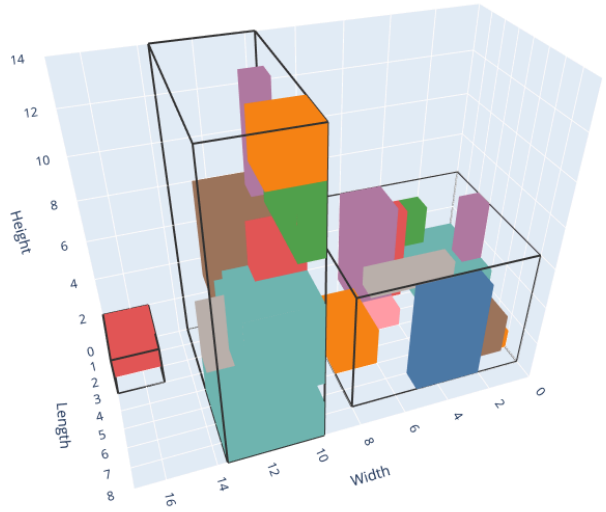
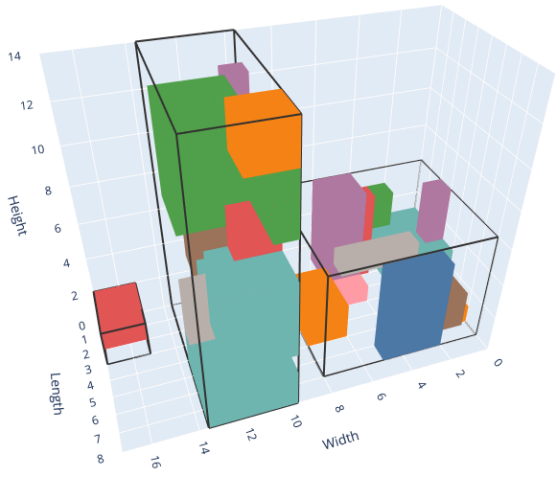
			Testcase : 3_trucks_50_packages.json Ukuran populasi : 100 Nilai terbaik yang mungkin : 20000 Nilai awal : 8386 Nilai akhir : 10054 State akhir : valid Iterasi/generasi: 500/500 Durasi : 316.459 detik   Initial State  Final State

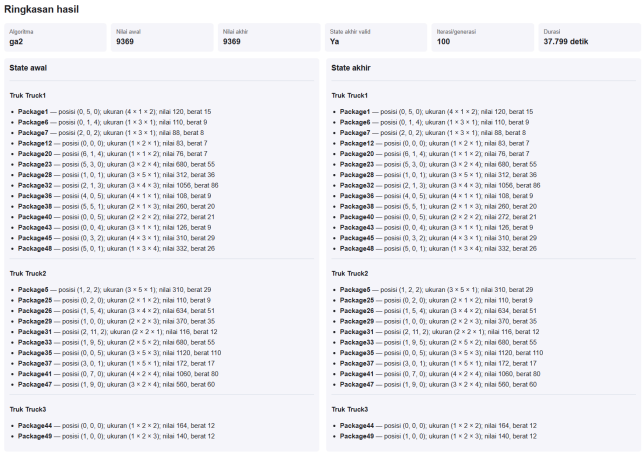
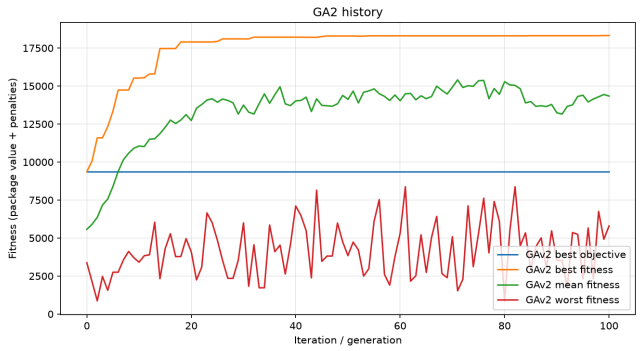


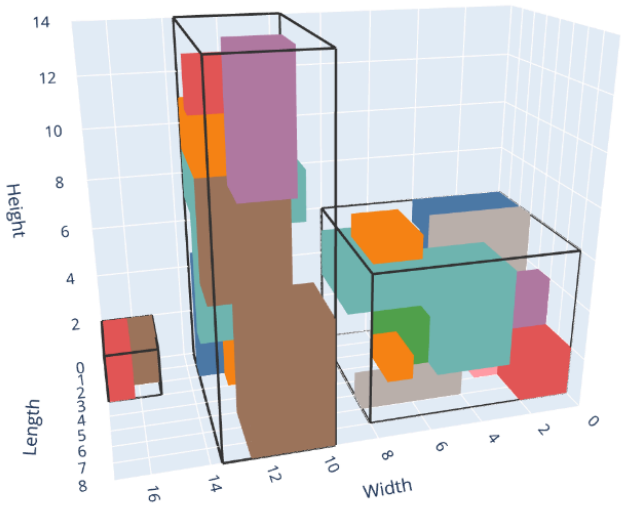
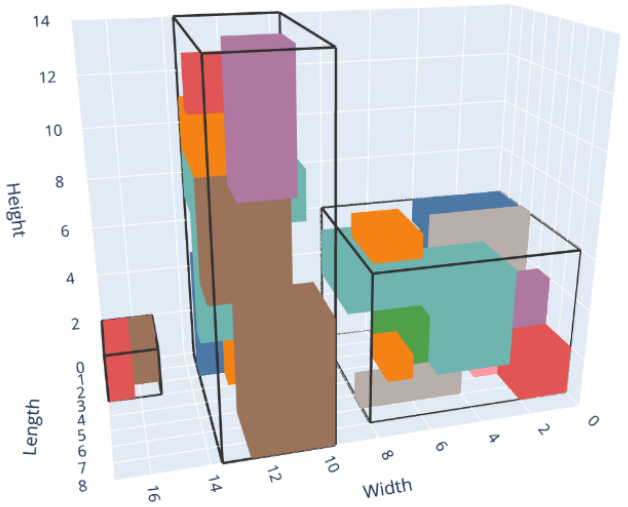
===== Ringkasan hasil =====
Algoritma : Genetic Algorithm v1 (GA)
Testcase : 3_trucks_50_packages.json
Ukuran populasi : 100
Nilai terbaik yang mungkin : 20000
Nilai awal : 8207
Nilai akhir : 9682
State akhir : valid
Iterasi/generasi: 500/500
Durasi : 469.129 detik



Grafik

			 Initial State  Final State 
	Genetic Algorithm v2	Populasi=100 Iterasi=100	===== Ringkasan hasil ===== Algoritma : Genetic Algorithm v2 (GA2)

			Testcase : 3_trucks_50_packages.json Nilai terbaik yang mungkin : 20000 Nilai awal : 9369 Nilai akhir : 9369 State akhir : valid Iterasi/generasi: 100/100 Durasi : 37.799 detik  Grafik:  Initial State:

			 Final State: 
			===== Ringkasan hasil ===== Algoritma : Genetic Algorithm v2 (GA2) Testcase : 3_trucks_50_packages.json Nilai terbaik yang mungkin : 20000 Nilai awal : 8359 Nilai akhir : 8359 State akhir : valid Iterasi/generasi: 100/100 Durasi : 156.670 detik

Ringkasan hasil

Algoritma	Nilai awal	Nilai akhir	State akhir valid	Iterasi/generasi	Durasi
ga2	8359	8359	Ya	100	156.670 detik

State awal

Truk Truck1

- **Package6** — posisi (6, 0, 6), ukuran (1 × 3 × 1), nilai 110, berat 9
- **Package7** — posisi (1, 0, 0), ukuran (1 × 3 × 1), nilai 88, berat 8
- **Package16** — posisi (0, 0, 1), ukuran (2 × 1 × 4), nilai 312, berat 26
- **Package19** — posisi (0, 1, 3), ukuran (4 × 2 × 3), nilai 812, berat 72
- **Package32** — posisi (4, 1, 2), ukuran (3 × 4 × 3), nilai 1056, berat 86
- **Package48** — posisi (3, 3, 0), ukuran (1 × 3 × 4), nilai 332, berat 26
- **Package50** — posisi (4, 0, 0), ukuran (3 × 1 × 4), nilai 352, berat 33

Truk Truck2

- **Package1** — posisi (0, 13, 3), ukuran (4 × 1 × 2), nilai 120, berat 15
- **Package10** — posisi (0, 0, 0), ukuran (2 × 1 × 4), nilai 160, berat 20
- **Package11** — posisi (0, 0, 7), ukuran (1 × 5 × 1), nilai 182, berat 13
- **Package13** — posisi (0, 0, 5), ukuran (1 × 5 × 1), nilai 200, berat 18
- **Package18** — posisi (2, 3, 0), ukuran (2 × 5 × 1), nilai 298, berat 27
- **Package28** — posisi (0, 5, 7), ukuran (3 × 5 × 1), nilai 312, berat 36
- **Package30** — posisi (0, 8, 1), ukuran (3 × 5 × 3), nilai 1475, berat 116
- **Package31** — posisi (1, 1, 0), ukuran (2 × 2 × 1), nilai 116, berat 12
- **Package39** — posisi (0, 1, 1), ukuran (4 × 5 × 2), nilai 1210, berat 99
- **Package41** — posisi (0, 6, 2), ukuran (4 × 2 × 4), nilai 1060, berat 80

Truk Truck3

- **Package44** — posisi (0, 0, 1), ukuran (1 × 2 × 2), nilai 164, berat 12

State akhir

Truk Truck1

- **Package6** — posisi (6, 0, 6), ukuran (1 × 3 × 1), nilai 110, berat 9
- **Package7** — posisi (1, 0, 0), ukuran (1 × 3 × 1), nilai 88, berat 8
- **Package16** — posisi (0, 0, 1), ukuran (2 × 1 × 4), nilai 312, berat 26
- **Package19** — posisi (0, 1, 3), ukuran (4 × 2 × 3), nilai 812, berat 72
- **Package32** — posisi (4, 1, 2), ukuran (3 × 4 × 3), nilai 1056, berat 86
- **Package48** — posisi (3, 3, 0), ukuran (1 × 3 × 4), nilai 332, berat 26
- **Package50** — posisi (4, 0, 0), ukuran (3 × 1 × 4), nilai 352, berat 33

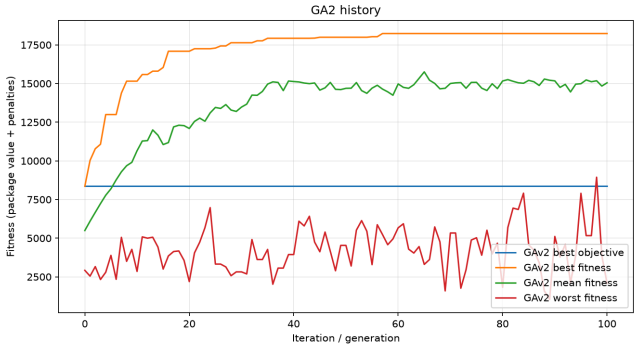
Truk Truck2

- **Package1** — posisi (0, 13, 3), ukuran (4 × 1 × 2), nilai 120, berat 15
- **Package10** — posisi (0, 0, 0), ukuran (2 × 1 × 4), nilai 160, berat 20
- **Package11** — posisi (0, 0, 7), ukuran (1 × 5 × 1), nilai 182, berat 13
- **Package13** — posisi (0, 0, 5), ukuran (1 × 5 × 1), nilai 200, berat 18
- **Package18** — posisi (2, 3, 0), ukuran (2 × 5 × 1), nilai 298, berat 27
- **Package28** — posisi (0, 5, 7), ukuran (3 × 5 × 1), nilai 312, berat 36
- **Package30** — posisi (0, 8, 1), ukuran (3 × 5 × 3), nilai 1475, berat 116
- **Package31** — posisi (1, 1, 0), ukuran (2 × 2 × 1), nilai 116, berat 12
- **Package39** — posisi (0, 1, 1), ukuran (4 × 5 × 2), nilai 1210, berat 99
- **Package41** — posisi (0, 6, 2), ukuran (4 × 2 × 4), nilai 1090, berat 80

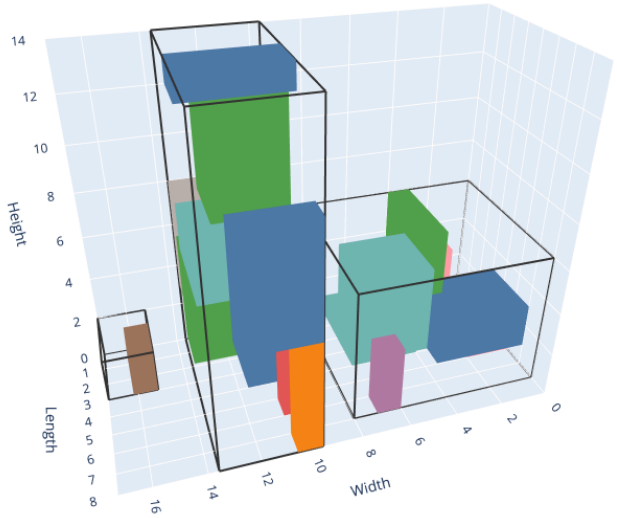
Truk Truck3

- **Package44** — posisi (0, 0, 1), ukuran (1 × 2 × 2), nilai 164, berat 12

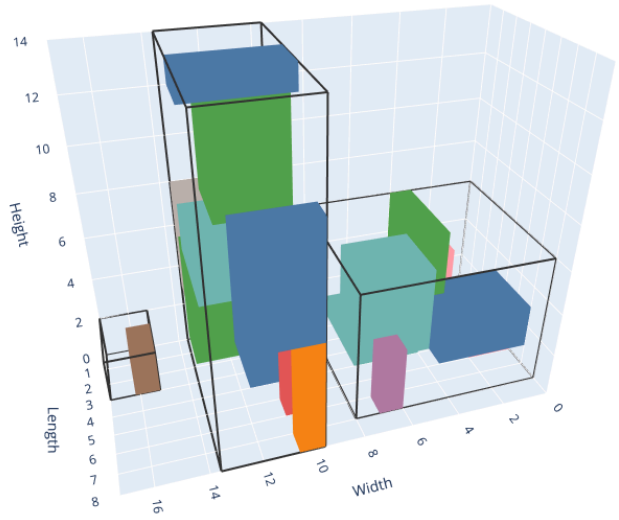
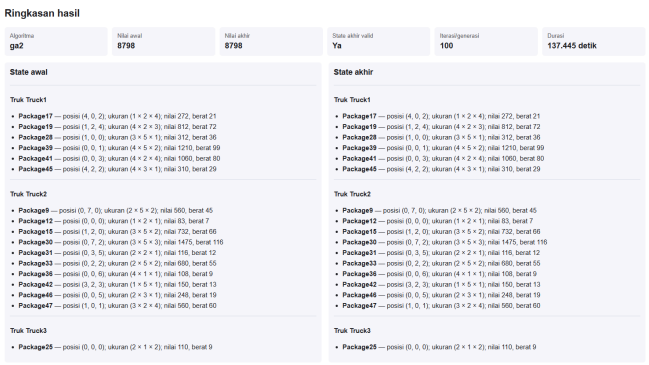
Grafik

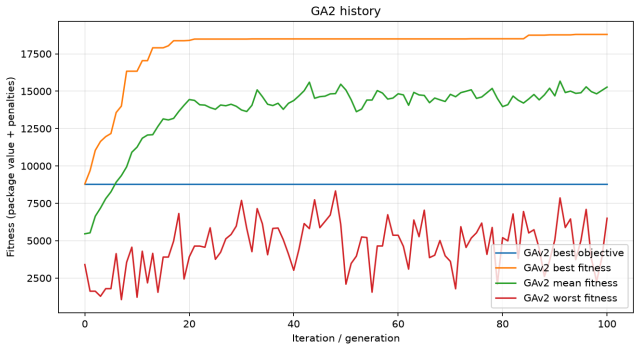
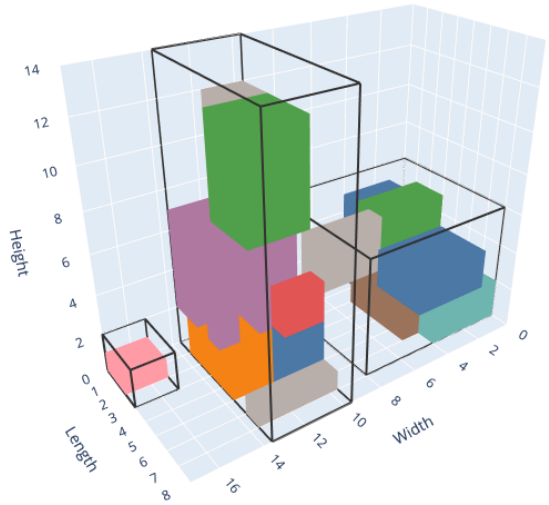
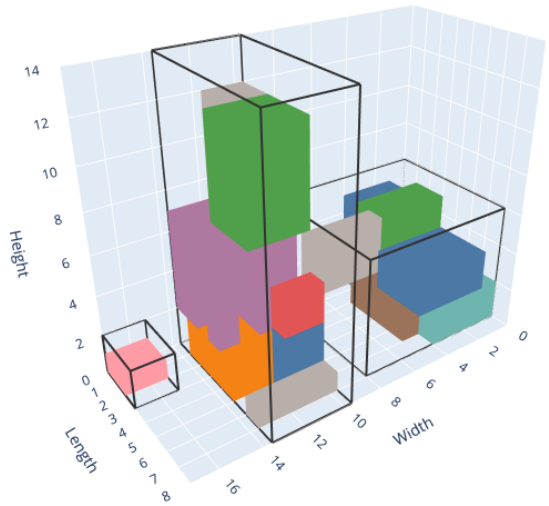


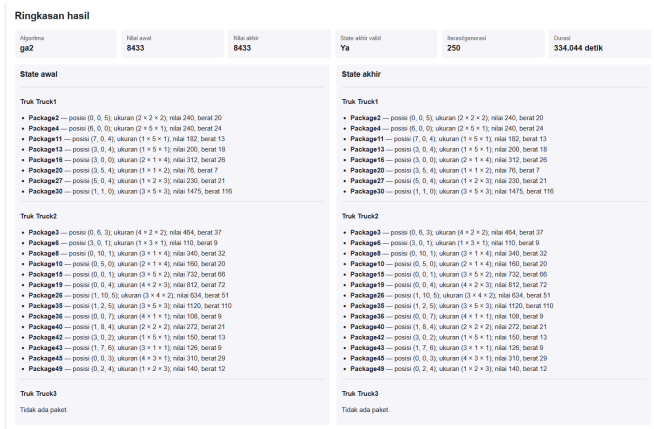
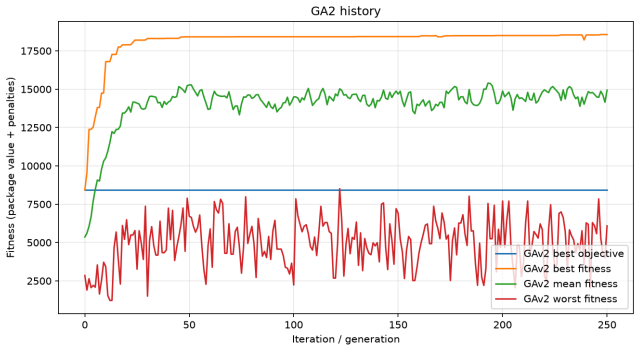
Initial State

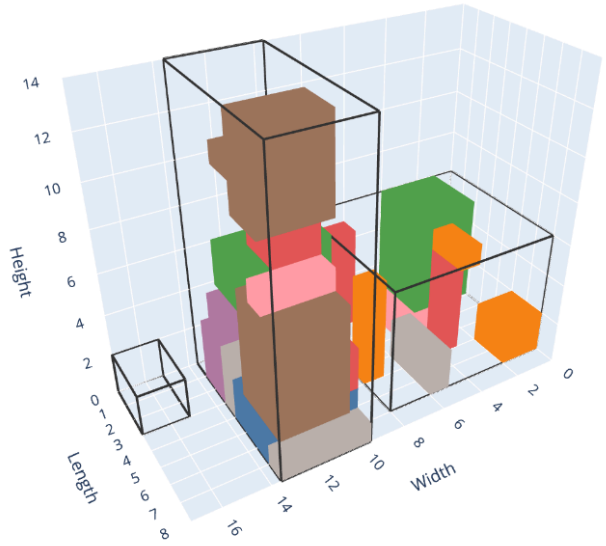


Final State

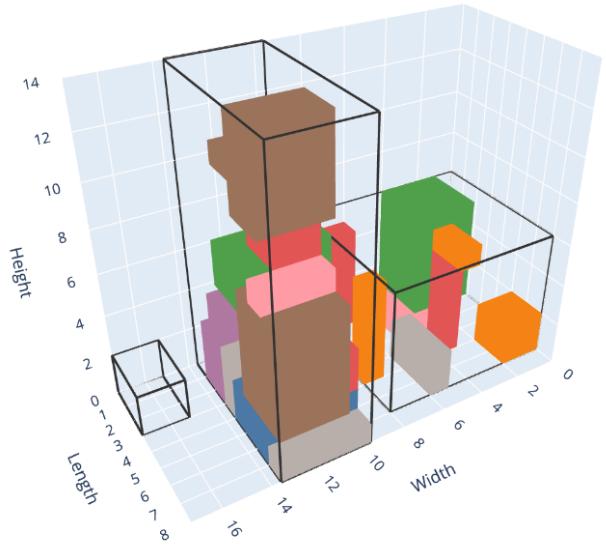
			
			===== Ringkasan hasil ===== Algoritma : Genetic Algorithm v2 (GA2) Testcase : 3_trucks_50_packages.json Nilai terbaik yang mungkin : 20000 Nilai awal : 8798 Nilai akhir : 8798 State akhir : valid Iterasi/generasi: 100/100 Durasi : 137.445 detik  Grafik

			 Initial State  Final State 
	Genetic Algorithm v2	Populasi=100 Iterasi=250	===== Ringkasan hasil ===== Algoritma : Genetic Algorithm v2 (GA2)

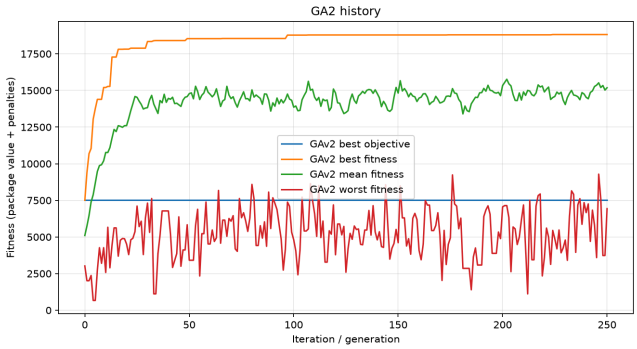
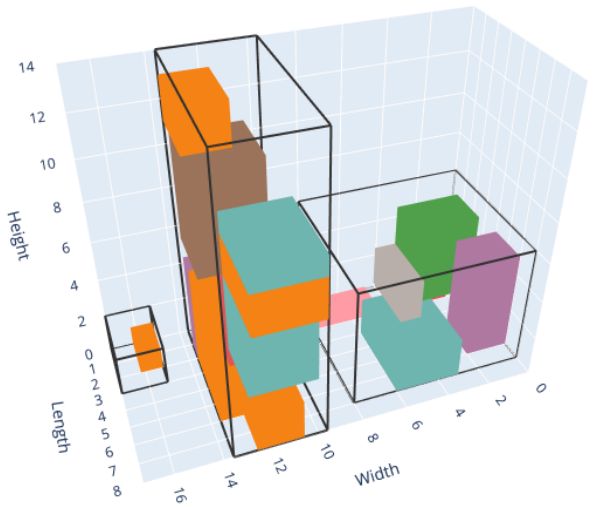
			Testcase : 3_trucks_50_packages.json Nilai terbaik yang mungkin : 20000 Nilai awal : 8433 Nilai akhir : 8433 State akhir : valid Iterasi/generasi: 250/250 Durasi : 334.044 detik  grafik  Initial State

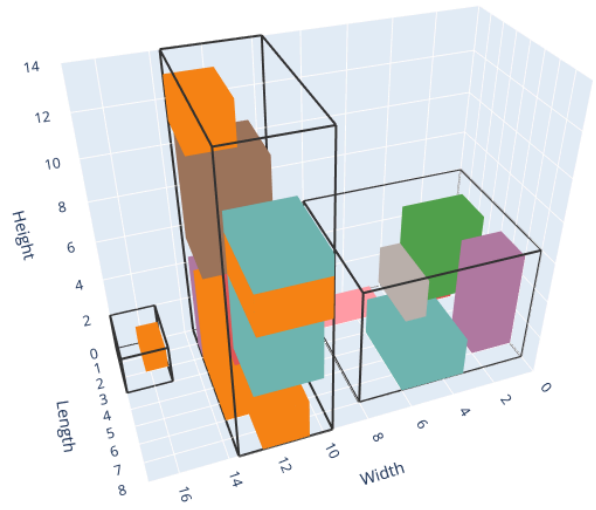


Final State

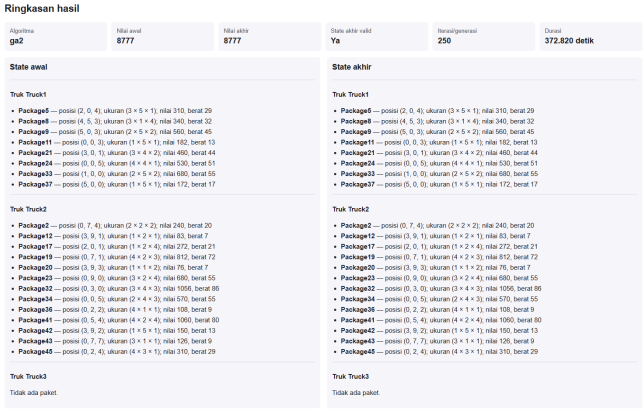


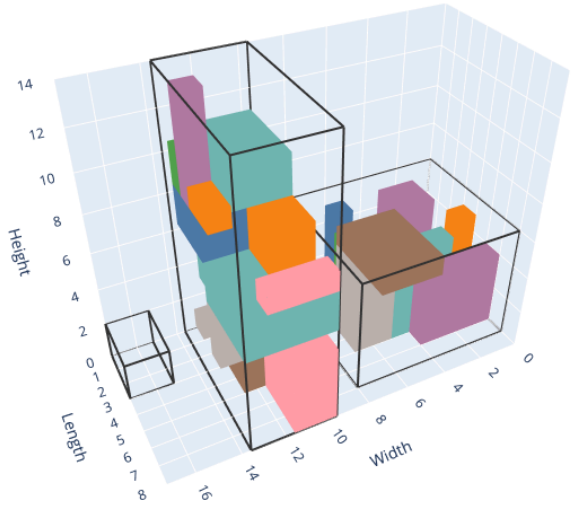
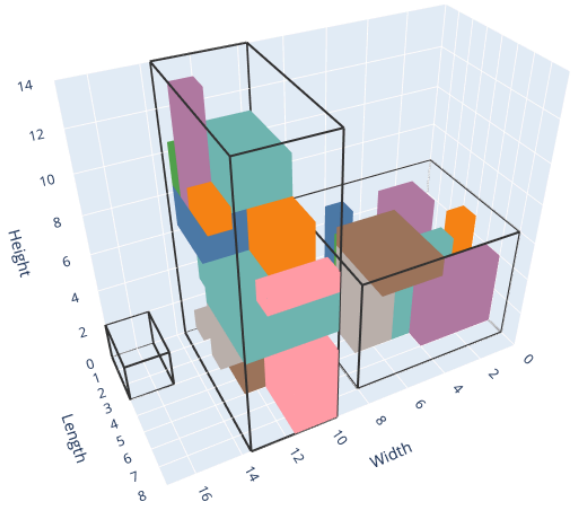
===== Ringkasan hasil =====
Algoritma : Genetic Algorithm v2 (GA2)
Testcase : 3_trucks_50_packages.json
Nilai terbaik yang mungkin : 20000
Nilai awal : 7505
Nilai akhir : 7505
State akhir : valid
Iterasi/generasi: 250/250
Durasi : 344.304 detik

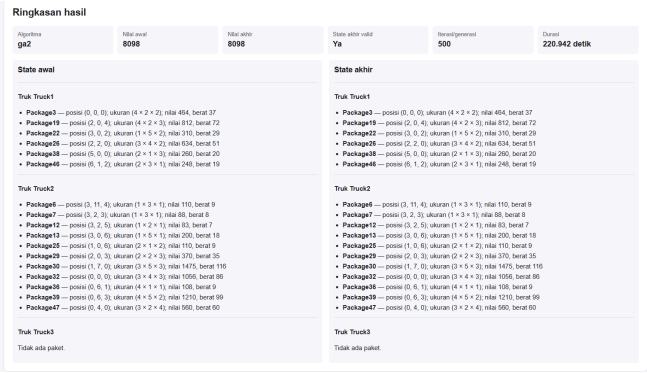
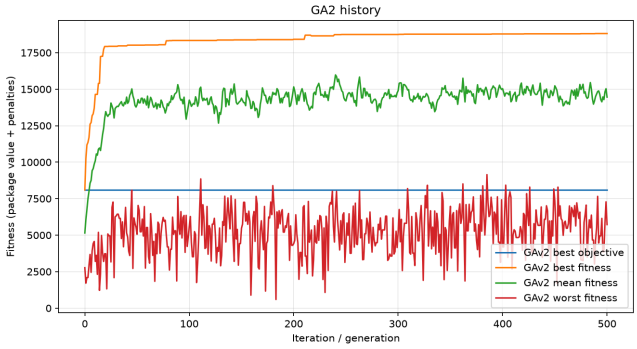
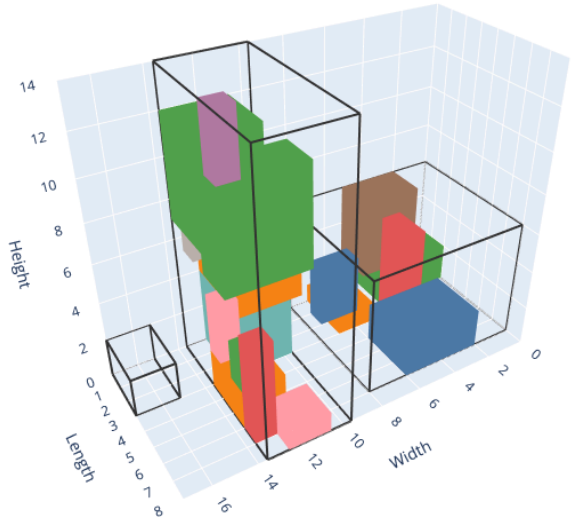
			Ringkasan hasil Algoritma: ga2 | Nilai awal: 7505 | Nilai akhir: 7505 | State akhir valid: Ya | Iterasi/generasi: 250 | Durasi: 344.304 detik **State awal** **Truk Truck1** • **Package12** — posisi (1, 2, 0); ukuran (1 × 2 × 1); nilai 83, berat 7 • **Package21** — posisi (1, 2, 2); ukuran (3 × 4 × 2); nilai 460, berat 44 • **Package23** — posisi (3, 0, 3); ukuran (3 × 2 × 4); nilai 680, berat 55 • **Package27** — posisi (4, 2, 2); ukuran (1 × 2 × 3); nilai 230, berat 21 • **Package33** — posisi (0, 0, 4); ukuran (2 × 5 × 2); nilai 680, berat 55 • **Package40** — posisi (1, 0, 0); ukuran (2 × 2 × 2); nilai 272, berat 21 • **Package43** — posisi (5, 0, 1); ukuran (3 × 1 × 1); nilai 126, berat 9 **Truk Truck2** • **Package2** — posisi (1, 0, 6); ukuran (2 × 2 × 2); nilai 240, berat 20 • **Package11** — posisi (3, 0, 2); ukuran (1 × 5 × 1); nilai 182, berat 13 • **Package18** — posisi (0, 0, 2); ukuran (2 × 5 × 1); nilai 298, berat 27 • **Package22** — posisi (2, 1, 2); ukuran (1 × 5 × 2); nilai 310, berat 29 • **Package29** — posisi (2, 11, 0); ukuran (2 × 2 × 3); nilai 370, berat 35 • **Package32** — posisi (0, 2, 4); ukuran (3 × 4 × 3); nilai 1056, berat 86 • **Package35** — posisi (1, 6, 1); ukuran (3 × 5 × 3); nilai 1120, berat 110 • **Package38** — posisi (2, 0, 3); ukuran (2 × 1 × 3); nilai 260, berat 20 • **Package42** — posisi (3, 0, 1); ukuran (1 × 5 × 1); nilai 150, berat 13 • **Package47** — posisi (0, 6, 4); ukuran (3 × 2 × 4); nilai 560, berat 60 • **Package50** — posisi (0, 8, 4); ukuran (3 × 1 × 4); nilai 352, berat 33 **Truk Truck3** • **Package20** — posisi (0, 0, 0); ukuran (1 × 1 × 2); nilai 76, berat 7 **State akhir** **Truk Truck1** • **Package12** — posisi (1, 2, 0); ukuran (1 × 2 × 1); nilai 83, berat 7 • **Package21** — posisi (1, 2, 2); ukuran (3 × 4 × 2); nilai 460, berat 44 • **Package23** — posisi (3, 0, 3); ukuran (3 × 2 × 4); nilai 680, berat 55 • **Package27** — posisi (4, 2, 2); ukuran (1 × 2 × 3); nilai 230, berat 21 • **Package33** — posisi (0, 0, 4); ukuran (2 × 5 × 2); nilai 680, berat 55 • **Package40** — posisi (1, 0, 0); ukuran (2 × 2 × 2); nilai 272, berat 21 • **Package43** — posisi (5, 0, 1); ukuran (3 × 1 × 1); nilai 126, berat 9 **Truk Truck2** • **Package2** — posisi (1, 0, 6); ukuran (2 × 2 × 2); nilai 240, berat 20 • **Package11** — posisi (3, 0, 2); ukuran (1 × 5 × 1); nilai 182, berat 13 • **Package18** — posisi (0, 0, 2); ukuran (2 × 5 × 1); nilai 298, berat 27 • **Package22** — posisi (2, 1, 2); ukuran (1 × 5 × 2); nilai 310, berat 29 • **Package29** — posisi (2, 11, 0); ukuran (2 × 2 × 3); nilai 370, berat 35 • **Package32** — posisi (0, 2, 4); ukuran (3 × 4 × 3); nilai 1056, berat 86 • **Package35** — posisi (1, 6, 1); ukuran (3 × 5 × 3); nilai 1120, berat 110 • **Package38** — posisi (2, 0, 3); ukuran (2 × 1 × 3); nilai 260, berat 20 • **Package42** — posisi (3, 0, 1); ukuran (1 × 5 × 1); nilai 150, berat 13 • **Package47** — posisi (0, 6, 4); ukuran (3 × 2 × 4); nilai 560, berat 60 • **Package50** — posisi (0, 8, 4); ukuran (3 × 1 × 4); nilai 352, berat 33 **Truk Truck3** • **Package20** — posisi (0, 0, 0); ukuran (1 × 1 × 2); nilai 76, berat 7 Grafik  Initial State  Final State



===== Ringkasan hasil =====
Algoritma : Genetic Algorithm v2 (GA2)
Testcase : 3_trucks_50_packages.json
Nilai terbaik yang mungkin : 20000
Nilai awal : 8777
Nilai akhir : 8777
State akhir : valid
Iterasi/generasi: 250/250
Durasi : 372.820 detik



			 Initial State  Final State 
	Genetic Algorithm v2	Populasi=100 Iterasi=500	===== Ringkasan hasil ===== Algoritma : Genetic Algorithm v2 (GA2) Testcase : 3_trucks_50_packages.json

			Nilai terbaik yang mungkin : 20000 Nilai awal : 8098 Nilai akhir : 8098 State akhir : valid Iterasi/generasi: 500/500 Durasi : 220.942 detik  Grafik  Initial State  Final State

Ringkasan hasil

Algoritma	Nilai awal	Nilai akhir	State akhir valid	Iterasi/generasi	Durasi
ga2	8098	8098	Ya	500	220.942 detik

State awal

Truk Truck1

- **Package3** — posisi (0, 0, 0); ukuran (4 × 2 × 2); nilai 464, berat 37
- **Package19** — posisi (2, 0, 4); ukuran (4 × 2 × 3); nilai 812, berat 72
- **Package22** — posisi (3, 0, 2); ukuran (1 × 5 × 2); nilai 310, berat 29
- **Package26** — posisi (2, 2, 0); ukuran (3 × 4 × 2); nilai 634, berat 51
- **Package38** — posisi (5, 0, 0); ukuran (2 × 1 × 3); nilai 260, berat 20
- **Package46** — posisi (6, 1, 2); ukuran (2 × 3 × 1); nilai 248, berat 19

Truk Truck2

- **Package6** — posisi (3, 11, 4); ukuran (1 × 3 × 1); nilai 110, berat 9
- **Package7** — posisi (3, 2, 3); ukuran (1 × 3 × 1); nilai 88, berat 8
- **Package12** — posisi (3, 2, 5); ukuran (1 × 2 × 1); nilai 83, berat 7
- **Package13** — posisi (3, 0, 6); ukuran (1 × 5 × 1); nilai 200, berat 18
- **Package28** — posisi (1, 0, 6); ukuran (2 × 1 × 2); nilai 110, berat 9
- **Package29** — posisi (2, 0, 3); ukuran (2 × 2 × 3); nilai 370, berat 35
- **Package30** — posisi (1, 7, 0); ukuran (3 × 5 × 3); nilai 1475, berat 116
- **Package32** — posisi (0, 0, 0); ukuran (3 × 4 × 3); nilai 1056, berat 86
- **Package36** — posisi (0, 6, 1); ukuran (4 × 1 × 1); nilai 108, berat 9
- **Package39** — posisi (0, 6, 3); ukuran (4 × 5 × 2); nilai 1210, berat 99
- **Package47** — posisi (0, 4, 0); ukuran (3 × 2 × 4); nilai 560, berat 60

Truk Truck3

Tidak ada paket.

State akhir

Truk Truck1

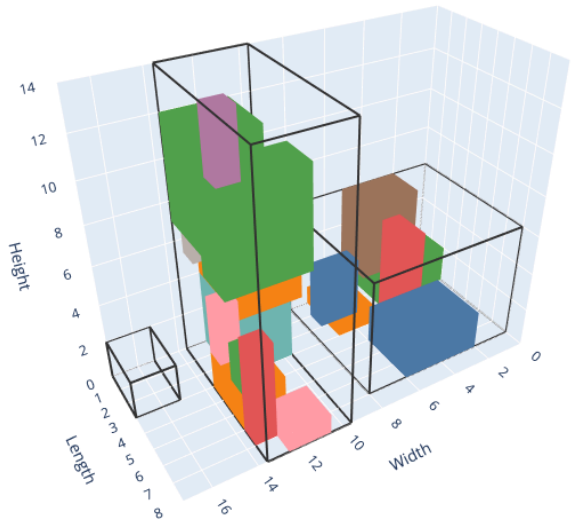
- **Package3** — posisi (0, 0, 0); ukuran (4 × 2 × 2); nilai 464, berat 37
- **Package19** — posisi (2, 0, 4); ukuran (4 × 2 × 3); nilai 812, berat 72
- **Package22** — posisi (3, 0, 2); ukuran (1 × 5 × 2); nilai 310, berat 29
- **Package26** — posisi (2, 2, 0); ukuran (3 × 4 × 2); nilai 634, berat 51
- **Package38** — posisi (5, 0, 0); ukuran (2 × 1 × 3); nilai 260, berat 20
- **Package46** — posisi (6, 1, 2); ukuran (2 × 3 × 1); nilai 248, berat 19

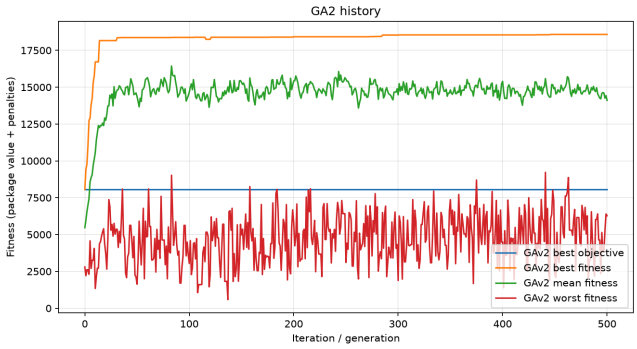
Truk Truck2

- **Package6** — posisi (3, 11, 4); ukuran (1 × 3 × 1); nilai 110, berat 9
- **Package7** — posisi (3, 2, 3); ukuran (1 × 3 × 1); nilai 88, berat 8
- **Package12** — posisi (3, 2, 5); ukuran (1 × 2 × 1); nilai 83, berat 7
- **Package13** — posisi (3, 0, 6); ukuran (1 × 5 × 1); nilai 200, berat 18
- **Package28** — posisi (1, 0, 6); ukuran (2 × 1 × 2); nilai 110, berat 9
- **Package29** — posisi (2, 0, 3); ukuran (2 × 2 × 3); nilai 370, berat 35
- **Package30** — posisi (1, 7, 0); ukuran (3 × 5 × 3); nilai 1475, berat 116
- **Package32** — posisi (0, 0, 0); ukuran (3 × 4 × 3); nilai 1056, berat 86
- **Package36** — posisi (0, 6, 1); ukuran (4 × 1 × 1); nilai 108, berat 9
- **Package39** — posisi (0, 6, 3); ukuran (4 × 5 × 2); nilai 1210, berat 99
- **Package47** — posisi (0, 4, 0); ukuran (3 × 2 × 4); nilai 560, berat 60

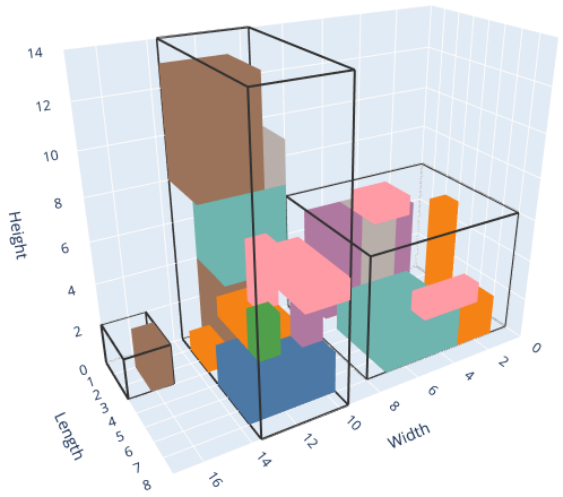
Truk Truck3

Tidak ada paket.

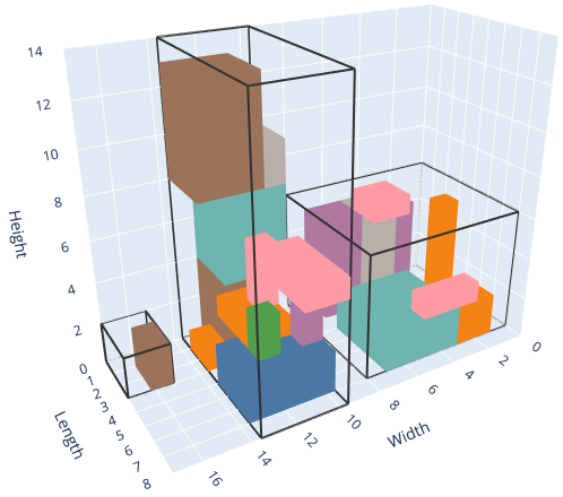
			
			===== Ringkasan hasil ===== Algoritma : Genetic Algorithm v2 (GA2) Testcase : 3_trucks_50_packages.json Nilai terbaik yang mungkin : 20000 Nilai awal : 8053 Nilai akhir : 8053 State akhir : valid Iterasi/generasi: 500/500 Durasi : 254.249 detik  Grafik



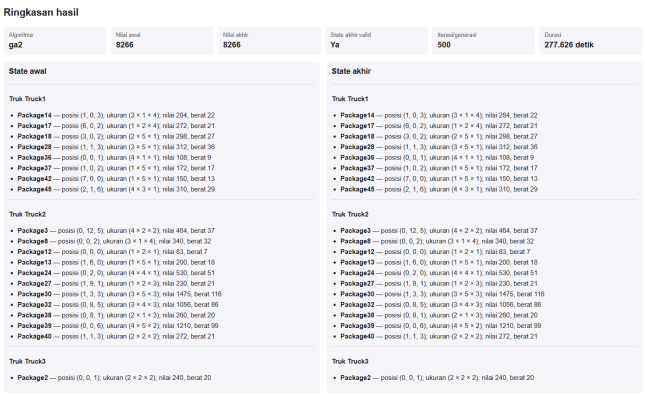
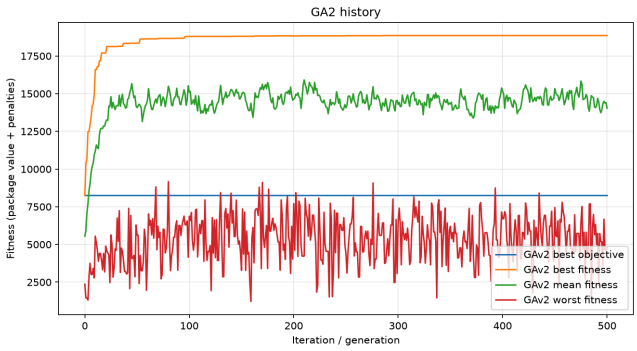
Initial State

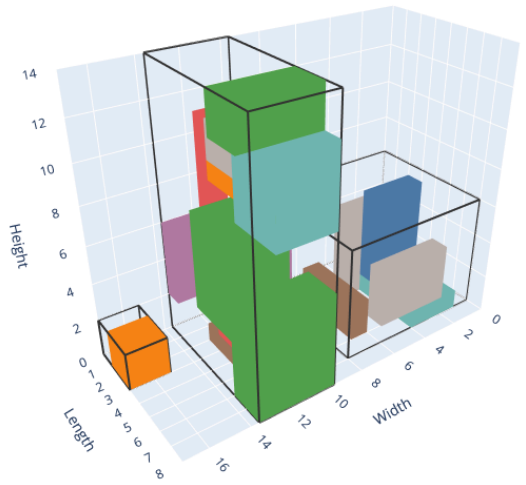
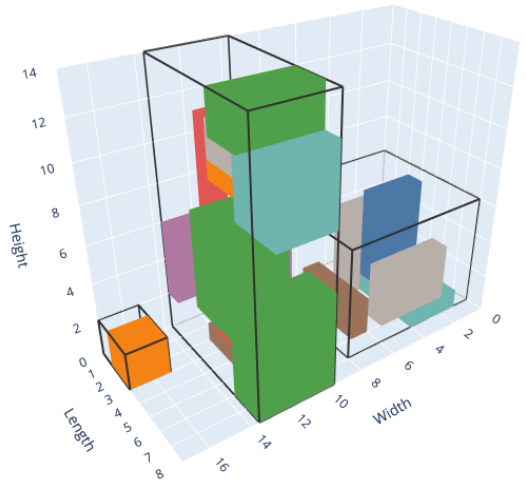


Final State



===== Ringkasan hasil =====

			Algoritma : Genetic Algorithm v2 (GA2) Testcase : 3_trucks_50_packages.json Nilai terbaik yang mungkin : 20000 Nilai awal : 8266 Nilai akhir : 8266 State akhir : valid Iterasi/generasi: 500/500 Durasi : 277.626 detik  Grafik  Initial State

			 Final State 

Analisis Hasil

Nilai objektif cenderung tidak menaik-naik setelah beberapa generasi di awal. Dugaan mengapa bisa terjadi seperti ini yaitu:

- Nilai fitness tidak memperhatikan apakah valid atau tidak. Karena itu, walau fitness menaik, nilai objektif yang harus valid tidak ikut menaik juga.
- Variasi populasi kecil, mungkin dikarenakan:
    - Awalan yang pasti valid mungkin membuat ngestuck di local optima.
    - Bisa juga karena peluang mutasi kurang besar, jadi variasinya kurang banyak. Apalagi dengan banyaknya package.

- Cara seleksi dengan Roulette Wheel Selection dengan peluang terlalu bias terhadap konfigurasi yang fitnessnya besar. Konfigurasi yang fitnessnya lebih kecil tidak sempat memberikan variasi ke generasi berikutnya.

Kesimpulan dan Saran

- Memulai konfigurasi acak dari konfigurasi yang valid bisa mempercepat konvergensi ke solusi yang bagus.
- Persentase populasi elit yang terlalu besar membuat genetic algorithm terlalu cepat konvergen ke local optima. Variasi populasi menjadi sangat kecil.
- Perhitungan nilai fitness perlu memperhatikan apakah mengarah ke fungsi objektif yang tinggi. Dapat dicoba untuk membuat nilai fitness = 0 tanpa penalti jika tidak valid tanpa penalti.

Pembagian Tugas

Nama	NIM	Pembagian Tugas
Muhammad Nafis Habibi	13524018	Struktur data, genetic algorithm
Billy Ontoseno Irawan	13524121	Simulated annealing
Muhammad Rafi Akbar	13524125	Hill Climbing, visualisasi

Referensi

https://algorithmafternoon.com/books/simulated_annealing/chapter02/

https://algorithmafternoon.com/genetic/elitist_genetic_algorithm/

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM :

- Muhammad Nafis Habibi/13524018
- Billy Ontoseno Irawan/13524121
- Muhammad Rafi Akbar/13524125

Fakultas/Sekolah : STEI-K/Teknik Informatika

Kode Mata Kuliah : IF3170

Nama Mata Kuliah : Inteligensi Artifisial

Judul Tugas : Tugas Besar 1

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	**Digunakan?**	**Tingkat Penggunaan**
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	0
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	0
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti	Ya	1 (Menanyakan mengenai

ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.		konsep dari algoritma yang akan dibuat)
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	0
Lainnya: Pembuatan test case	Ya	2 (test case dibuat oleh generative AI dengan perubahan oleh manusia)

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 20 Oktober 2026



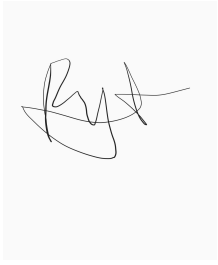
Muhammad Nafis Habibi

NIM 13524018



Billy Ontoseno Irawan

NIM 13524121



Muhammad Rafi Akbar

NIM 13524125